



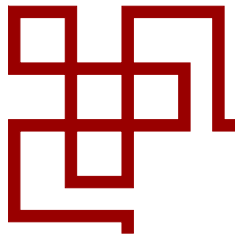
IND219
NOTES

2026

Introduction to Programming in Python

Dr. Simone Santoni

Bayes Business School
106 Bunhill Row, London, EC1Y 8TZ



Copyright

©Dr. Simone Santoni (simone.santoni.1@city.ac.uk)
Do not share or distribute without permission

Version

Current version is 1.1 — last edited on August 30th 2026

Contents

Preface	1
1 Organization of the Notes and IND219	3
1.1 Justification for an Introductory Module on Python	3
1.2 Scope of the Notes	4
1.3 Learning and Teaching Activities	6
2 Getting Started with Python	11
2.1 Installing Python	11
2.2 How Python Runs Programs	14
2.3 How We Run Python Programs	15
2.4 Managing Python Environments	20
3 Python Objects	27
3.1 Number Type Fundamentals	29
3.2 String Type Fundamentals	37
3.3 List and Dictionaries	45
3.4 Dictionaries	48
3.5 Tuples	52
3.6 Sets	54
3.7 Files	56
3.8 Python Statements and Syntax	61
3.9 Control Flow (or If-Then Statements)	63
3.10 While and For Loops	65
3.11 Iterations and Comprehensions	69
4 Technical & Scientific Computation with NumPy	73
4.1 Installing NumPy	73
4.2 NumPy <code>ndarray</code>	74
4.3 Array Creation Routines	80
4.4 Array Manipulation Routines	95
4.5 Universal Functions in NumPy	97
4.6 Mathematical Functions	97
4.7 Statistics	99
4.8 Linear Algebra	106
4.9 Pseudorandom Number Generation	115
4.10 File Input and Output (IO) with <code>ndarrays</code>	124
5 Data Management with Pandas	129
5.1 Pandas 101	129
5.2 The Pandas <code>DataFrame</code>	132
5.3 Checking a <code>DataFrame</code> 's Attributes	136
5.4 The Anatomy of a <code>DataFrame</code>	137
5.5 Querying <code>DataFrames</code>	140

5.6	Manipulating DataFrame Columns	142
5.7	Data Types and Pandas	148
5.8	Handling ‘Time’ Data in Pandas	149
5.9	Shaping and Reshaping DataFrames	153
5.10	Group By Part I: Data Aggregation	159
5.11	Group By Part II: Data Transformation	161
5.12	Working with Multiple DataFrames	162
5.13	File Input and Output (IO) with Pandas	166
6	Coda	173
Appendices		
A	Cheat Sheets	177
A.1	Escapes	177
A.2	String Methods	178
A.3	NumPy Array Manipulation Routines	180
B	GitHub: The World-Leading Collaborative and Versioning Tool	189
B.1	GitHub in a Nuthshell	189
B.2	GitHub in the Education Sector	189
B.3	Getting Started with GitHub	192

List of Figures

1.1	Interest towards data-science related programming languages over time	4
1.2	The suggested workflow to learn to code	8
2.1	A screenshot showing a sample of applications accessible from within Anaconda Navigator	13
2.2	A stylized representation of Python’s execution model	14
2.3	A screenshot showing the execution of the minimal script included in Snippet 1.1 . . .	16
2.4	A screenshot of an interactive Python shell	16
2.5	A screenshot of an interactive IPython shell	17
2.6	A screenshot of a Jupyter notebook session	18
2.7	A screenshot of an interactive IPython shell in VSCode	18
2.8	A screenshot of a Jupyter notebook session in VSCode	19
2.9	A screenshot showing how to create a Python environment using <code>conda</code> from the com- mand line	22
2.10	A screenshot showing how to populate a Python environment with libraries using <code>conda</code> from the command line	22
2.11	A screenshot showing how to create a new environment from within the ‘Environments’ section of Anaconda Navigator	24
2.12	A screenshot showing how to add Python modules to a (newly created) environment from within the ‘Environments’ section of Anaconda Navigator	25
4.1	A contour plot showing the associations among X, Y, and Z	89
4.2	Visual representation of NumPy <code>.sin</code> and <code>.cos</code> functions as per Snippet 5.12.	99
4.3	A visual illustration of the least-square estimation carried out in Snippet 5.17	116
4.4	A visual illustration of the Normal and Poisson data generated in Snippet 5.19	124
5.1	A stylized representation of a case-by-variable data structure	132
5.2	A stylized representation of a ‘wide’ data structure	154
5.3	A stylized representation of a ‘long’ data structure	155
B.1	The distinctive features of GitHub <i>vis a’ vis</i> traditional LMS	190
B.2	A screenshot of the GitHub repository behind SMM638, Network analytics	191
B.3	A screenshot of the GitHub desktop application webpage	192
B.4	A screenshot showing how to login into a GitHub account from within GitHub Desktop	193
B.5	A screenshot showing how to search for a repository name and then clone it	193
B.6	A screenshot showing how to clone a repo using its URL	194
B.7	A screenshot showing the actions available through GitHub Desktop	195

List of Tables

3.1	Built-In Objects in Python	28
3.2	Number Type Objects in Python	29
3.3	A Sample of Functions Provided by the <code>math</code> Module	31
3.4	Operator Precedence Hierarchy (Ascending Order)	33
3.5	Number Formatting Options in Python	35
3.6	Sample of String Literals and Operators	41
3.7	Popular List Methods	47
3.8	Popular Dictionary Methods	51
3.9	Popular File Methods	60
3.10	Python Statements	62
4.1	Common Use Attributes of NumPy <code>array</code>	76
4.2	NumPy Data Types	79
4.3	Routines for Creating Arrays from Shape or Value	82
4.4	Routines for Creating Arrays from Existing Data	85
4.5	Routines for Creating Record Arrays	87
4.6	Routines for Numerical Ranges	90
4.7	Routines for Building Matrices	92
4.8	Routines for the Matrix Class	94
4.9	NumPy Statistical Routines: Order Statistics	101
4.10	NumPy Statistical Routines: Average and Variances	102
4.11	NumPy Statistical Routines: Correlating	103
4.12	NumPy Statistical Routines: Histograms	104
4.13	NumPy Linear Algebra Routines: Matrix and Vector Products	107
4.14	NumPy Linear Algebra Routines: Decompositions	108
4.15	NumPy Linear Algebra Routines: Matrix Eigenvalues	109
4.16	NumPy Linear Algebra Routines: Norms and Other Number	110
4.17	NumPy Linear Algebra Routines: Solving Equations and Inverting Matrices	111
4.18	NumPy Pseudorandom Generators: Simple Random Data	117
4.19	NumPy Pseudorandom Generators: Permutations	118
4.20	NumPy Pseudorandom Generators: Distributions	119
4.21	File Input and Output with NumPy Arrays	126
5.1	Creating a DataFrame from Data Loaded in the Python Session	133
5.2	Routines for Reading and Writing Data with Pandas: Pickles	167
5.3	Routines for Reading and Writing Data with Pandas: Excel Spreadsheets	168
5.4	Routines for Reading and Writing Data with Pandas: JSON Files	169
5.5	Routines for Reading and Writing Data with Pandas: Flat Files	170
A.1	Helpful Escapes	177
A.2	Comprehensive List of String Methods	178
A.3	NumPy Universal Functions: Mathematical Operations	181
A.4	NumPy Universal Functions: Trigonometric Operations	182
A.5	NumPy Universal Functions: Floating Operations	183

A.6 NumPy Universal Functions: Comparison Operations 184

A.7 NumPy Array Manipulation Routines 185

Preface

These notes were prepared for the Summer Module ‘IND219 — Introduction to Programming in Python.’ The materials I present condense more than ten years of experience I developed using Python for industrial applications and research. At the same time, the materials incorporate the feedback from four different cohorts of Bayes MSc students who took IND219 and other modules for which Python is a prerequisite, such as ‘SMM638 — Network Analytics,’ ‘SMM635 — Data Visualization,’ and ‘SMM694 — Natural Language Processing.’

I prepared these notes with two design principles in mind. First, the notes should have mimicked a conversation between a person interested in learning Python and a knowledge recipient (a robot or a human being, it does not matter!). The [Cornell Notes Template](#) helped me emphasize the notes’ conversational structure: the left-hand side of the page is reserved for questions, reported as blue boxes; answers are displayed on the right-hand side of the page.

Second, I designed the notes to be as practical as possible. For this reason, the knowledge recipient’s answer is supported by a Python code snippet. In total, there are 76 snippets the learner may want to scrutinize, reason with, run, and modify.

What to say? Have fun!

—Simone Santoni

Chapter 1

Organization of the Notes and IND219

In this chapter, the reader is supposed to appreciate:

- The justification for an introductory module on Python for Business Analytics students
 - The scope of the teaching materials included in these notes
 - The learning and teaching activities that form ‘IND219 — Introduction to Programming in Python’
-

1.1 Justification for an Introductory Module on Python

Why should I learn Python?

Python is a general-purpose, high-level programming language. So, why should business professionals learn it? Mainly, there are two reasons:

- First, Python is the center of one of the richest ecosystems of modules¹ for technical and scientific computation, which plays a key role in the quantitative analysis of organizations and markets.
- Second, for a large number of developers and users, ‘Python’ and ‘data science’ are largely overlapping skills (as Figure 1.1 shows, the popularity surge of Python is strictly related to the emergence and development of the data science field, YR 2014 →)

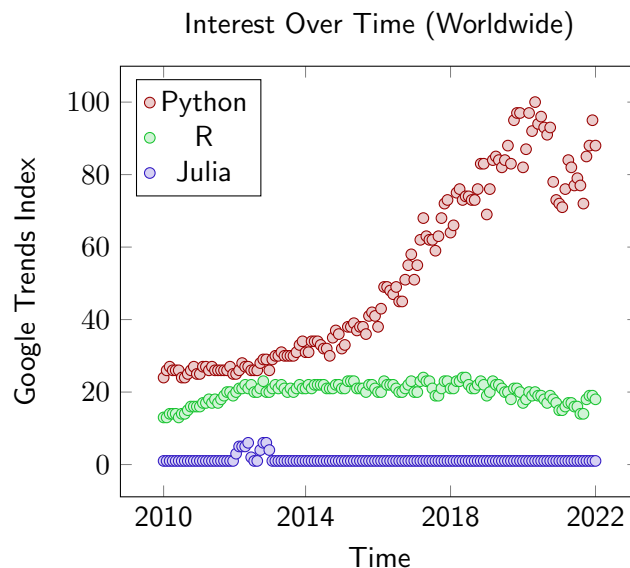


Figure 1.1: Interest towards data-science related programming languages over time
Notes. — Source is Google Trends

Why do I need to take a pre-course module on Python?

There are at least three arguments for taking a module such as IND219:

- Modern quantitative curricula increasingly depart from a spreadsheet-based approach to teaching and learning
- For example, the Bayes Business Analytics MSc builds heavily on the Python programming language
 - There are *four core modules* based on Python: Data Visualization, Deep Learning, Network Analytics, Value Creation in Digital Settings
 - Plus *two elective modules*: Applied Machine Learning and Applied Natural Language Processing
- You may want to get up and running with Python in advance, that is, before the start of Term I

1.2 Scope of the Notes

What is the scope of IND219?

The scope of IND219 has been tailored to the learning needs of the Bayes MSc students. Specifically, these notes were created based on the feedback comments provided by four cohorts of students. Hence, I warmly invite you to focus on these notes instead of learning Python using the many books and resources available on the web — otherwise, you may miss core notions and tools or learn irrelevant ones!

So, do these notes cover the entire spectrum of the Python language?

Got it. Which parts of the Python language will I learn then?

Which technical and scientific modules will I learn in these notes?

Can you summarize the subjects covered in these notes?

NO! Covering the entire spectrum of the language would take substantial time — Python has extensive language references!! Also, some aspects of the Python language reference have limited added value for business professionals closer to Python users than developers.

The attention revolves around the following subjects:

- Variables
- Object
- Some built-in functions and methods for strings, lists, dictionaries, and sets

Learning the basics of the Python language will put you in the position to carry out technical and scientific computation tasks — namely, the essence of data science X BA

Two foundational modules support statistical analysis, Viz, ML, and DL: NumPy and Pandas. These two modules are the focus of IND219's second part.

Of course. These notes cover the following topics:

- Getting started with Python (Chapter 2)
- Python objects (Chapter 3)
- Technical and scientific computation with NumPy (Chapter 4)
- Data management with Pandas (Chapter 5)

1.3 Learning and Teaching Activities

What is the philosophy behind these notes?

Mainly, there are two pillars:

- LESS IS MORE — that is, you may want to focus on a few core Python notions at a time and practice them
- LEARN BY DOING — that is, the best way to learn Python is by addressing concrete problems

I am a Programming Newbie:
How Do I Learn Python?

Here is an approach that proved effective according to the pedagogical literature on learning programming languages:²

- Step 1: Focus on *few core notions* at a time (e.g., ‘string methods’)
- Step 2: Learn the selected core notions ‘pen & paper’ (stay away from the computer!)
- Step 3: Open a Python session and practice the few core notions
- Step 4: Critically self-assess your learning (if your code is not working, Python will force you to understand what’s wrong!)

How do I assess my learning outcome?

You may want to assess your learning frequently, at the end of each chapter or, possibly, every two or three sections of the chapter. As Figure 1.2 suggests, the key idea is to adopt a sequential approach. Subject-by-subject, you may want to:

- Open a learning circle by watching the hook video and study the notes closely
- Test your understanding by taking a quiz
- If your learning is satisfactory, try to solve some practical coding problems (i.e., problem sets)

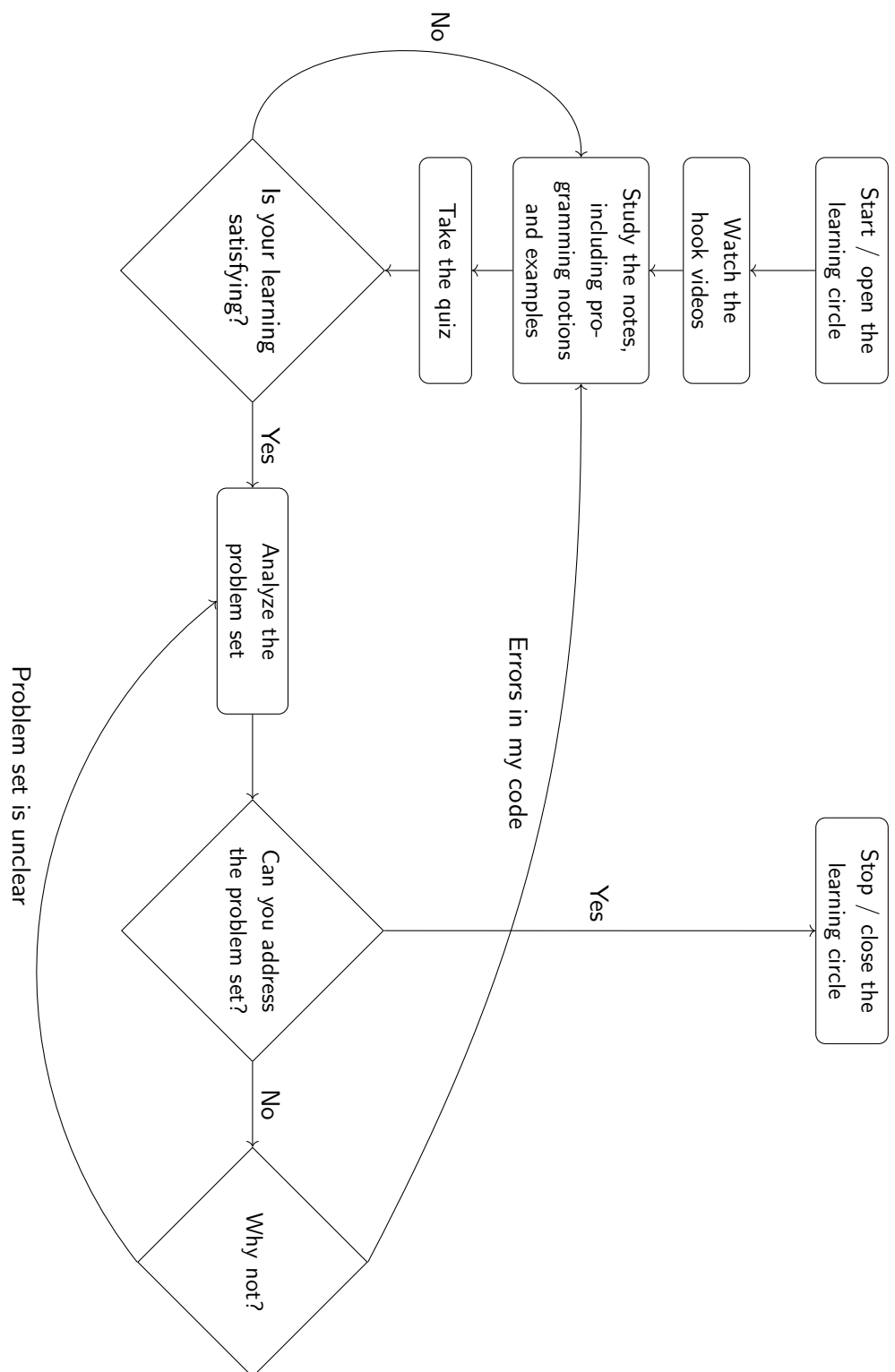


Figure 1.2: The suggested workflow to learn to code

Notes

¹Throughout the various chapters, I will use the terms module, library, and package interchangeably.

²See for example i) Mayer, Richard E. 1981. “The Psychology of how Novices Learn Computer Programming.” *ACM Computing Surveys* (CSUR) 13 (1): 121–141, ii) Robins, Anthony, Janet Rountree, and Nathan Rountree. 2003. “Learning and Teaching Programming: A Review and Discussion.” *Computer Science Education* 13 (2): 137–172, and iii) Soloway, Elliot, and James C Spohrer. 2013. *Studying the Novice Programmer*. Psychology Press.

Chapter 2

Getting Started with Python

At the end of the chapter, you will be able to:

- Get Python work in your machine
 - Run Python code from a script file and interactively
 - Configure a Python environment
-

2.1 Installing Python

How can I install Python?

What are the distinctive features of Anaconda?

There are two options:

- Using the [official installer](#)
- Using the [Anaconda Distribution](#) of Python (preferred option)
- Anaconda is ‘battery-included’ — it comes with a humongous number of modules for data science
 - If you use the official Python installation, you must install the modules you need on your own!
- Anaconda is a bundle of various pieces of software:
 - `conda` is the Swiss army knife to manage Python modules and environments
 - Anaconda Navigator is the graphical interface from within to access Python IDEs and related desktop/web applications

What are the steps to install Anaconda?

Here are the steps:

1. Download the installer for your operating system (unless you have a very old machine running Windows, go for the 64-Bit version)
2. Run the installer
 - For Linux: navigate to the folder where you have downloaded the installer as per step 1, open a shell session, then run `$ bash ./Anaconda3-XXXX.XX-Linux-x86-64.sh`
 - For Windows and Mac OS: just run the graphical installer downloaded in step 1
3. Accept the terms proposed by the Anaconda people to use their software, comprising Python, the `conda` package manager, and a bundle of modules for data science
4. That's it!
 - For Linux users: if you accepted the default installation options, an environmental variable is created either in your `.bashrc` or `.zshrc`. That means you can access the various pieces of software included in the Anaconda installation (e.g., Anaconda Navigator) from a shell session
 - For Windows and Mac OS users: the various pieces of software included in the Anaconda installation are available from the menu of your system

What are the pieces of software included in Anaconda?

There are plenty of applications included in the Anaconda installation. These applications can be accessed from within Anaconda Navigator (see Figure 2.1), available in the launcher of your operating system.

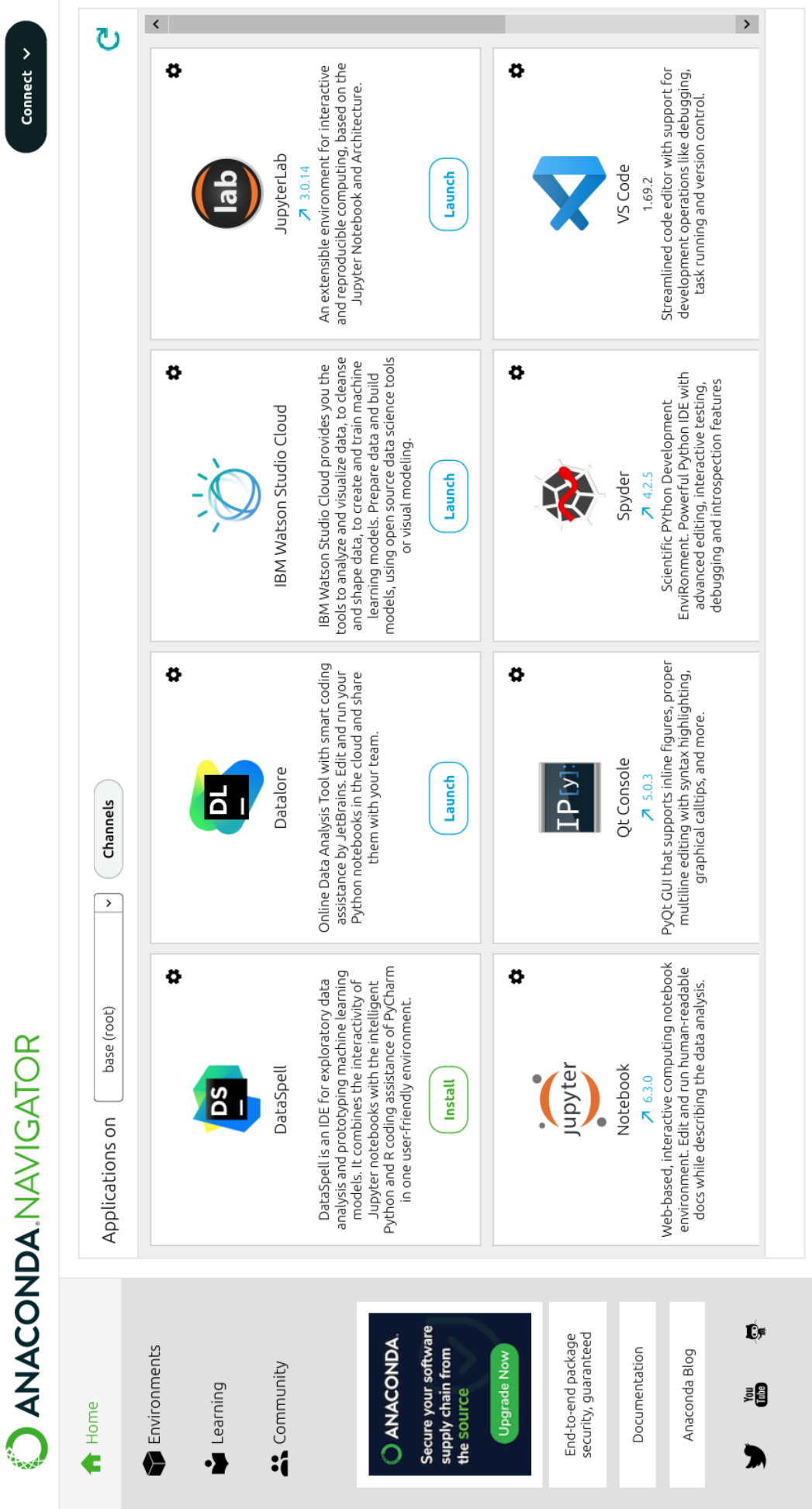


Figure 2.1: A screenshot showing a sample of applications accessible from within Anaconda Navigator

2.2 How Python Runs Programs

Is Python a programming language or an interpreter?

How does an interpreter work?

Short answer: BOTH. Typically, we refer to Python as a programming language. However, Python is also a software package called an interpreter, a kind of program that executes other programs. When you write a Python program, the Python interpreter reads your program and carries out the instructions it contains. In effect, the interpreter is a layer of software logic between your code and the computer hardware on your machine.

- When you instruct Python to run your script (e.g., ‘script.py’), there are a few steps the interpreter carries out before your code starts crunching away (see Figure 2.2)
- First, the code is compiled into something called ‘byte code.’
 - This step happens behind the scenes (there is nothing to do for the programmer!)
 - A file called ‘script.pyc’, automatically generated, contains the translation of your code into lower-level code instructions
- Then, the compiled code is routed to something called a ‘Python Virtual Machine’ (PVM)
 - This step is hidden from the programmer like the previous one
 - Mainly, it iterates through your byte code instructions, one by one, to carry out their operations

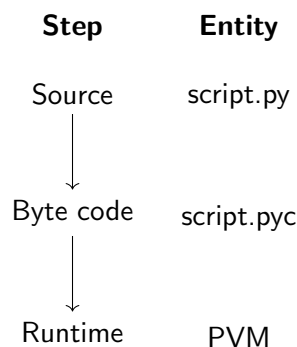


Figure 2.2: A stylized representation of Python’s execution model

2.3 How We Run Python Programs

What are the alternative ways to run some Python code?

How do I run a Python script in a non-interactive way?

There are two alternatives:

- The non-interactive way consists of preparing a Python script and running it from the command line
- The interactive way consists of preparing and running one or a few Python statements from within a Python shell

One has to go through a two-step process:

- First, the script has to be prepared using a text editor (being basic or sophisticated, it is up to you)
- Second, the script has to be executed from the shell

Snippet 1.1 shows a minimal Python script. Such a script can be created using the built-in text editor of your operating system or an advanced text editor such as [Atom](#), [Emacs](#), [Vim/Neovim](#), [Sublime Text](#), and [Visual Studio Code](#).

Once the script is saved to a file with extension `.py`, you can execute it from the command line by typing the following command in the shell session of our choice:

```
$ python filename.py
```

Let us unpack that command. The `$` symbol denotes the statement has to be run in a shell session; the `python` command tells the shell to use the Python interpreter to evaluate the script; `filename.py` is the script's file name. Figure 2.3 shows how to run the Python script containing the code displayed in Snippet 1.1 (named `simple_script.py`).³ It is self-evident the outcome of the script is displayed in the active shell session (we will see the `print` function in action many times over the next few chapters).

Snippet 1.1 — a minimal Python script

```
1 # Print a string object
2 print("Bazinga")
3
4 # Print the result of an algebraic operation
5 print(2 + 4)
```

```
(base) → ~ python simple_script.py
Bazinga
6
(base) → ~
```

Figure 2.3: A screenshot showing the execution of the minimal script included in Snippet 1.1

How do I interactively run a Python script?

How do I run Python code interactively using a Python or IPython shell?

Mainly, there are three ways to do that:

- Running a Python shell in the terminal
- Running an IPython shell in the terminal
- Interacting with a Python or IPython shell through an Integrated Development Environment (IDE)

The first step is opening the terminal emulator of your choice (e.g., Windows Terminal, Cmder, Iterm, Terminator, Kitty). Then, you run the command `$ python` or `$ ipython` to start a Python and IPython shell respectively (see Figures 2.4 and 2.5).

```
(base) → ~ python
Python 3.9.7 (default, Sep 16 2021, 08:50:36)
[Clang 10.0.0 ] :: Anaconda, Inc. on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Bazinga")
Bazinga
>>> print(2 + 4)
6
>>>
```

Figure 2.4: A screenshot of an interactive Python shell


```
(base) ➔ ~ ipython
Python 3.9.7 (default, Sep 16 2021, 08:50:36)
Type 'copyright', 'credits' or 'license' for more information
IPython 7.29.0 -- An enhanced Interactive Python. Type '?' for help.

In [1]: print("Bazinga")
Bazinga

In [2]: print(2 + 4)
6

In [3]:
```

Figure 2.5: A screenshot of an interactive IPython shell

What are the most popular Python IDEs?

There are plenty of Python IDEs in the market, including:

- Colab (online)
- IDLE
- Datalore (online)
- Jupyter/Jupyterlab
- PyCharm
- Qt Console
- Spyder
- Thonny
- Wing

Shall I start using Jupyter?

Based on my experience, Jupyter sustains novices' programming learning in at least two ways. First, the graphical interface of a Jupyter notebook (see Figure 2.6) allows learners to embrace a trial and error approach to coding by dissecting a script into manageable snippets — or even single lines — that can be better examined, understood, and tested. Second, the text boxes and rich media content typically included in a notebook provide learners with explanations to understand a script's logical structure and background.

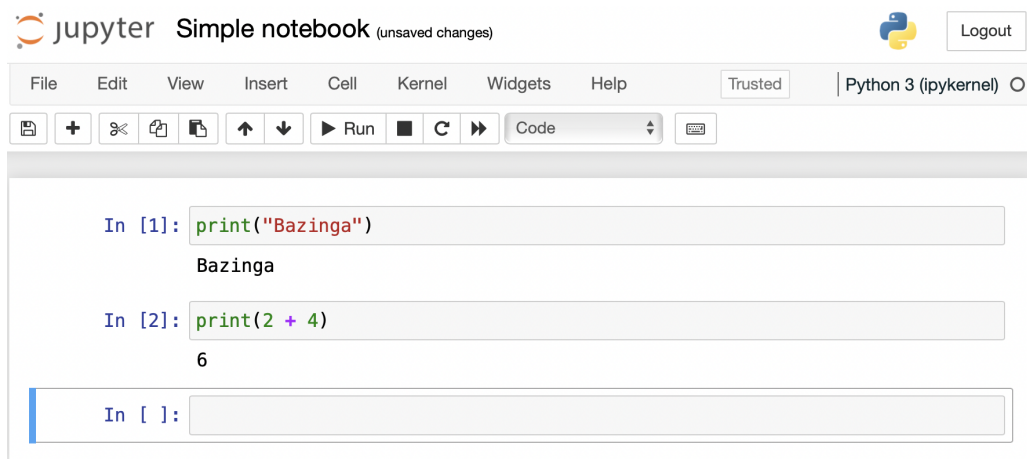


Figure 2.6: A screenshot of a Jupyter notebook session

Can I turn an advanced text editor into a Python IDE?

Of course. By installing a couple of plugins, the following (advanced) text editors can turn into Python IDEs

- Emacs
- Vm/Neovim
- Visual Studio Code (VSCode)

Figures 2.7 and 2.8 show how to run an IPython and Jupyter session in VSCode respectively.⁴

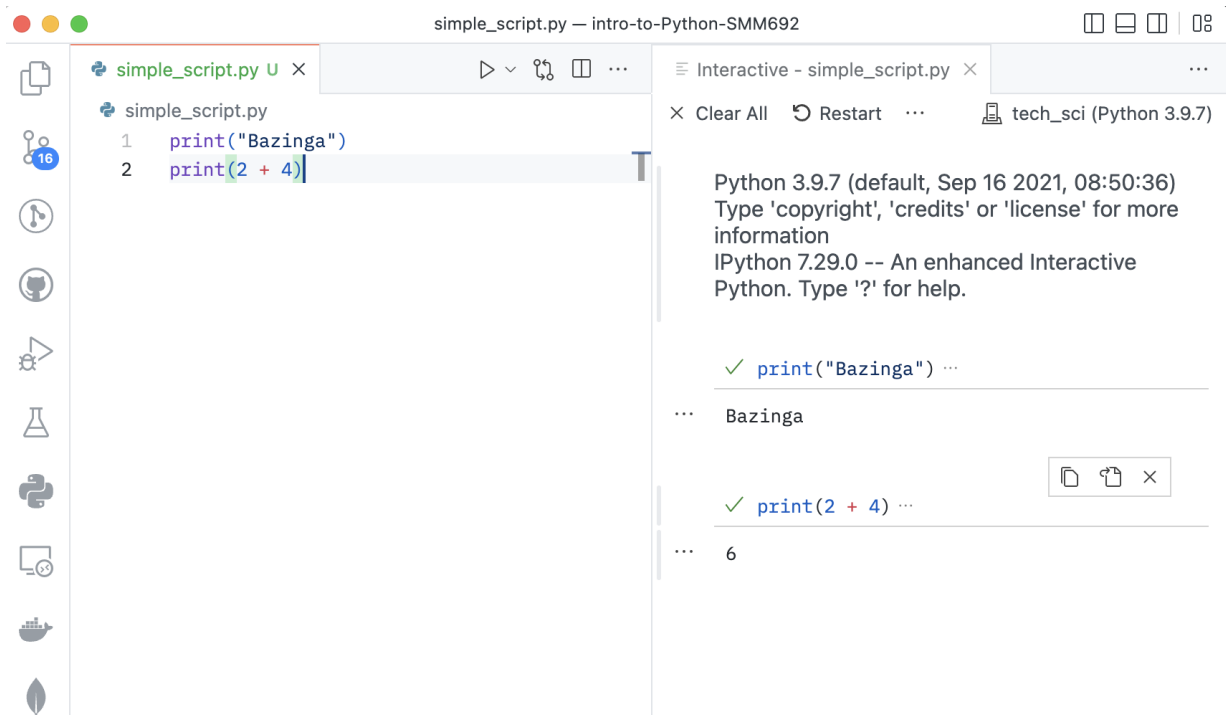


Figure 2.7: A screenshot of an interactive IPython shell in VSCode

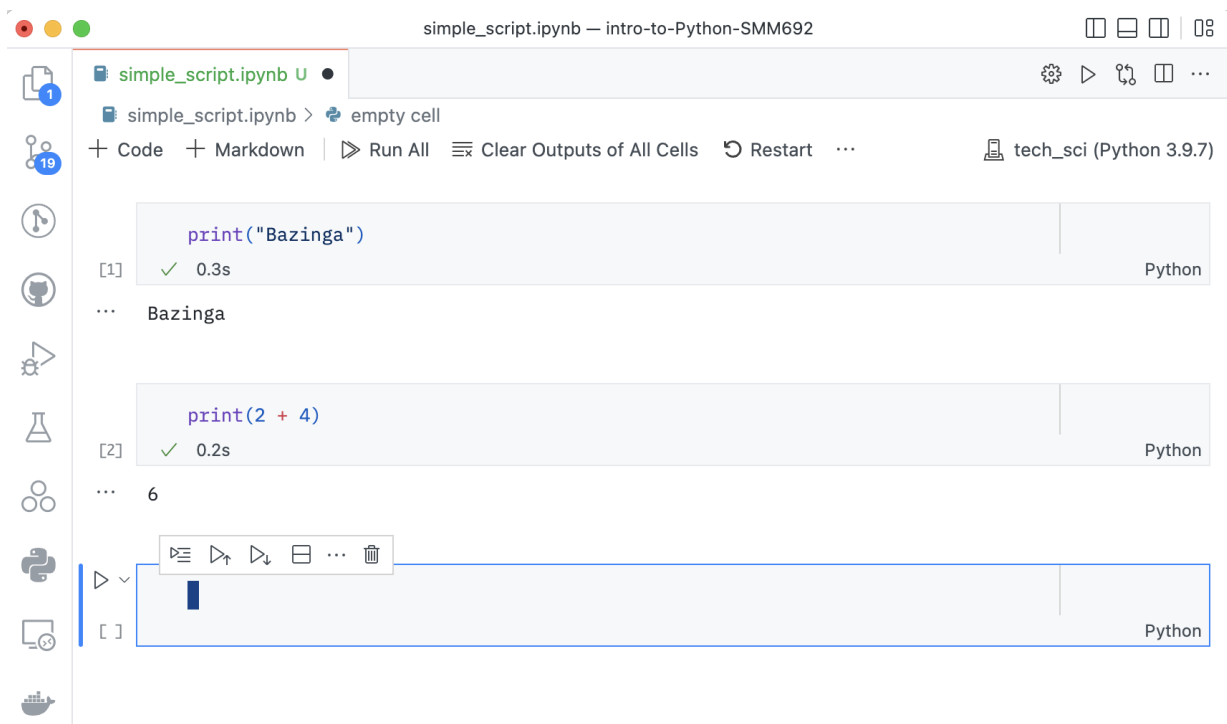


Figure 2.8: A screenshot of a Jupyter notebook session in VSCode

2.4 Managing Python Environments

What is a Python environment?

A Python environment is a self-contained directory that contains a Python interpreter, a set of Python packages, and their dependencies.

Why should I use a Python environment?

Mainly, there are two reasons:

1. To protect the system-wise installation of Python
2. To create collections of Python modules to deploy in specific projects/classes of projects (e.g., Machine Learning projects)

Why should I protect my system Python?

The large majority of operating systems come with Python installed by default. Such an installation is called ‘system Python’ and is responsible for many essential operating systems processes.

Every time we install a new Python module A , the package manager (e.g., conda) checks the libraries on which A depends. If a dependency $D(A)$ is not installed, the package manager will install it for us. The larger the number of libraries:

- The longer it takes to install a library (checking the web of dependencies takes substantial time!)
- The more likely inconsistencies will arise within the web of dependencies; that is
 - The less likely we will be able to install the modules we need
 - The more likely modules previously installed will stop work

Ultimately, if you care about not breaking some essential operating system features, you should avoid using ‘system Python’ for projects requiring Python.

Why Python environments work well for specific projects?

Typically, we create a Python environment to carry out a specific project or a class of projects (e.g., Machine Learning projects). The advantage is twofold:

- Reliability → the web of dependencies is relatively simple insofar as we install only a few modules that are required by the project or project class. Likely as not, we will not come across installation issues!
- Reproducibility → the environment explains the modules necessary to carry out a project. Hence, a user who wants to reproduce our project's results knows which modules to install

How do I create a Python environment?

The procedure to create a Python environment depends on the package manager you use (hence, the Python installation we have). Here, I focus on the case where conda is the adopted package manager.⁵

How do I use conda from the command line to create and populate a Python environment?

To create a Python environment using conda, we can run the following command in the shell:

```
$ conda create --name myenv python=3.X
```

where conda is the name of the package manager, create is the operation to run,⁶ myenv is the name of the environment and python=3.X is the version of Python to use (e.g., 3.10).

To use the newly created environment, one has to activate it by running the following command in the shell:

```
$ conda activate myenv
```

At this point, one can install packages in the activated environment using the following command:

```
$ conda install package.name
```

Figures 2.9 and 2.10 show a concrete example wherein a new Python environment is created and populated using conda from the command line.

```
(base) → ~ conda create -n my_env
Collecting package metadata (current_repodata.json): done
Solving environment: done

## Package Plan ##

  environment location: /Users/sbbk475/opt/anaconda3/envs/my_env

Proceed ([y]/n)? y

Preparing transaction: done
Verifying transaction: done
Executing transaction: done
#
# To activate this environment, use
#
#     $ conda activate my_env
#
# To deactivate an active environment, use
#
#     $ conda deactivate
#
(base) → ~
```

Figure 2.9: A screenshot showing how to create a Python environment using `conda` from the command line

```
(base) → ~ conda activate my_env
(my_env) → ~ conda install ipython jupyter numpy matplotlib
```

Figure 2.10: A screenshot showing how to populate a Python environment with libraries using `conda` from the command line

How do I use Anaconda Navigator to create and populate a Python environment?

It is also possible to create a Python environment from within the Anaconda Navigator (Figures 2.11 and 2.12 provide a pictorial illustration of how to create a new environment and populate it with modules).



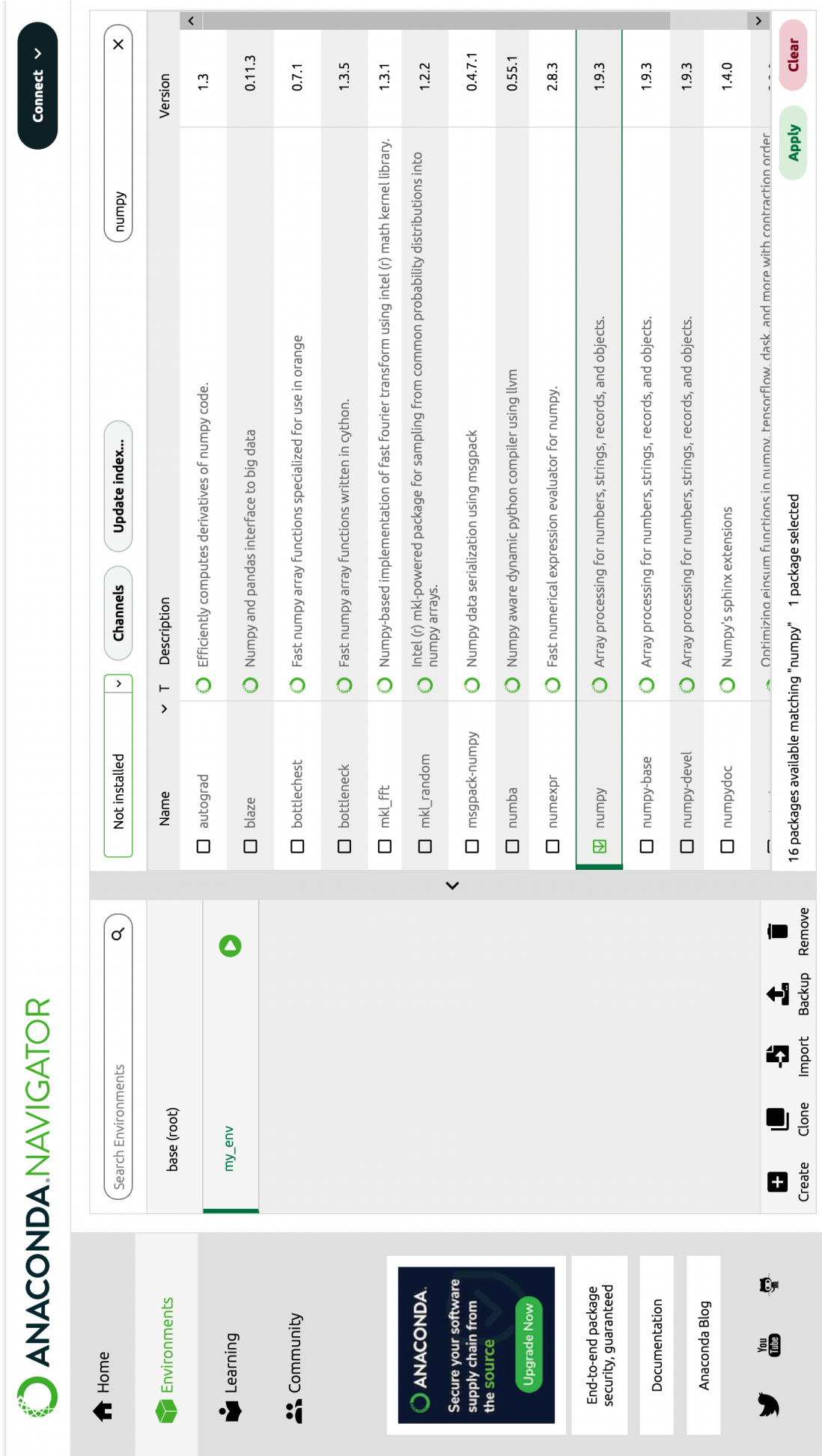


Figure 2.12: A screenshot showing how to add Python modules to a (newly created) environment from within the ‘Environments’ section of Anaconda Navigator

Notes

³The example shown in the screenshot assumes the Python script `sample_script.py` is saved in the current working directory. If the script is saved in a different directory, you have to specify the full path to the script in the command line.

⁴Please refer to the [official documentation](#) on how to use Python in VSCode.

⁵`pip` users are warmly encouraged to read the [Virtual Environments and Packages](#) section of the official Python Tutorial.

⁶The [Conda Command reference](#) illustrates the various commands Conda provides for managing packages and environments.

Chapter 3

Python Objects

At the end of the chapter, you will be able to evaluate the various types of Python objects regarding:

- Key features
- Use cases/roles
- Available methods

What is a Python object?

In essence, Python objects are pieces of data. Mark Lutz, the author of the popular book [Learning Python](#)⁷, points out

“... in Python, we do things with stuff. “Things” take the form of operations like addition and concatenation, and “stuff” refers to the objects on which we perform those operations”.

What are the main families of Python objects?

In Python, there are two families of objects: built-in objects provided by the Python language itself and ad-hoc objects — called [classes](#) — we can create to accomplish specific goals.

Why do built-in Python objects matter?

What are the core built-in Python objects?

Typically, we do not need to create ad-hoc objects. Python provides us with diverse built-in objects that make our job easier:

- Built-in objects make coding efficient and easy. For example, using the `string` object, we can represent and manipulate a piece of text — e.g., a newspaper article — without loading any `module`
- Built-in objects are flexible. For example, we can deploy built-in objects to create a `class`
- Built-in objects have been created and refined over time by a large community of expert developers. Hence, they are often more efficient than ad-hoc objects (unless the creator of the ad-hoc object knows her business!)

Table 3.1 illustrates the types of built-in Python objects. For example, `Numbers` and `strings` objects are used to represent numeric and textual data respectively. `Lists` and `dictionaries` are — likely as not — the two most popular `data structures` in Python. Lists are ordered collections of other objects (any type!!). Dictionaries are pairs of keys (e.g., a product identifier) and objects (e.g., the product’s price). No worries: we will go through each built-in type in the following sections of this document. Caveat: in the interest of logical coherence, the various built-in types will not be presented in the order adopted in Table 3.1.

TABLE 3.1
Built-In Objects in Python

Object type	Example literals/creation
Numbers	1234, 3.1415, 3+4j, 0b111, <code>Decimal()</code> , <code>Fraction()</code>
Strings	<code>'spam'</code> , <code>"Bob's"</code> , <code>b'a\x01c'</code> , <code>u'sp\xc4m'</code>
Lists	<code>[1, [2, 'three'], 4.5]</code> , <code>list(range(10))</code>
Dictionaries	<code>{'food': 'spam', 'taste': 'yum'}</code> , <code>dict(hours=10)</code>
Tuples	<code>(1, 'spam', 4, 'U')</code> , <code>tuple('spam')</code> , <code>namedtuple</code>
Files	<code>open('eggs.txt')</code> , <code>open(r'C:\ham.bin', 'wb')</code>
Sets	<code>set('abc')</code> , <code>{'a', 'b', 'c'}</code>
Other core types	Booleans, <code>types</code> , <code>None</code>
Program unit types	Functions, modules, classes
Implementation types	Compiled code, stack tracebacks

3.1 Number Type Fundamentals

What are the types of ‘number’ objects?

Snippet 4.1, “Doing stuff with numbers,” highlights the two most popular ‘number’ instances in Python: integers and floating-point numbers. Integers are whole numbers such as 0, 4, or -12. Floating-point numbers represent real numbers such as 0.5, 3.1415, or -1.6e-19. However, floating points in Python do not have — in general — the same value as the real number they represent.⁸ It is worth noticing that any single number with a period ‘.’ is considered a floating point in Python. Also, Snippet 4.1 shows that the multiplication of an integer by a floating point yields a floating point. That happens because Python first converts operands to the type of the most complicated operand.

Snippet 4.1 — doing ‘stuff’ with numbers

```

1 # integer addition
2 >>> 1 + 1
3 2
4
5 # floating-point multiplication
6 >>> 10 * 0.5
7 5.0
8
9 # 3 to the power 100
10 >>> 3 ** 100
11 515377520732011331036461129765621272702107522001
12
```

Are there other number types besides integers and floating points?

Besides integers and floating points numbers, Python includes fixed-precision, rational numbers, Booleans, and sets instances — see Table 3.2.

TABLE 3.2
Number Type Objects in Python

Literal	Interpretation
1234, -24, 0, 9999999999999999	Integers (unlimited size)
1.23, 1., 3.14e-10, 4E210, 4.0e+210	Floating-point numbers
0o177, 0x9ff, 0b101010	Octal, hex, and binary literals in 3.X
0177, 0o177, 0x9ff, 0b101010	Octal, octal, hex, and binary literals in 2.X
3+4j, 3.0+4.0j, 3J	Complex number literals
set(‘spam’), {1, 2, 3, 4}	Sets: 2.X and 3.X construction forms
Decimal(‘1.0’), Fraction(1, 3)	Decimal and fraction extension types
bool(X), True, False	Boolean type and constants

How do I carry out basic arithmetic operations in Python?

Numbers in Python support the usual mathematical operations:

- `+` → addition
- `-` → subtraction
- `*` → multiplication
- `\` → floating point division
- `//` → integer division
- `%` → modulus (remainder)
- `**` → exponentiation

To use these operations, it is sufficient to launch a Python or IPython session without any modules loaded (see Snippet 4.1).

How do I carry out advanced mathematical operations?

Besides the mathematical operations shown above, there are many [modules shipped with Python](#) that carry out advanced/specific numerical analysis. For example, the [math](#) module provides access to the mathematical functions defined by the [C standard](#).⁹ Table 3.3 reports a sample of these functions. To use them [math](#), we have to import the module as shown in Snippet 4.2. Another popular module shipped with Python is [random](#), implementing pseudo-random number generators for various distributions (see the lower section of Example 2).

TABLE 3.3
A Sample of Functions Provided by the `math` Module

Function name	Expression
<code>math.sqrt(x)</code>	$\sqrt{x}$
<code>math.exp(x)</code>	e^x
<code>math.log(x)</code>	$\ln x$
<code>math.log(x, b)</code>	$\log_b(x)$
<code>math.log10(x)</code>	$\log_{10}(x)$
<code>math.sin(x)</code>	$\sin(x)$
<code>math.cos(x)</code>	$\cos(x)$
<code>math.tan(x)</code>	$\tan(x)$
<code>math.asin(x)</code>	$\arcsin(x)$
<code>math.acos(x)</code>	$\arccos(x)$
<code>math.atan(x)</code>	$\arctan(x)$
<code>math.sinh(x)</code>	$\sinh(x)$
<code>math.cosh(x)</code>	$\cosh(x)$
<code>math.tanh(x)</code>	$\tanh(x)$
<code>math.asinh(x)</code>	$\operatorname{arsinh}(x)$
<code>math.acosh(x)</code>	$\operatorname{arcosh}(x)$
<code>math.atanh(x)</code>	$\operatorname{artanh}(x)$
<code>math.hypot(x, y)</code>	The Euclidean norm, $\sqrt{x^2 + y^2}$
<code>math.factorial(x)</code>	$x!$
<code>math.erf(x)</code>	The error function at x
<code>math.gamma(x)</code>	The gamma function at x , $\omega(x)$
<code>math.degrees(x)</code>	Converts x from radians to degrees
<code>math.radians(x)</code>	Converts x from degrees to radians

Snippet 4.2 — advanced mathematical operations with the math module

```

1 # import the math module
2 >>> import math
3
4 # base-y log of x
5 >>> math.log(12, 8)
6 1.1949875002403856
7
8 # base-10 log of x
9 >>> math.log10(12)
10 1.0791812460476249
11
12 # import the random module
13 >>> import random
14
15 # a draw from a normal distribution with mean = 0 and standard deviation = 1
16 >>> random.normalvariate(0, 1)
17 -0.136017752991189
18
19 # trigonometric functions
20 >>> math.cos(0)
21 1.0
22
23 >>> math.sin(0)
24 0.0
25
26 >>> math.tan(0)
27 0.0
28
29 # an expression containing a factorial product
30 >>> math.factorial(4) - 4 * 3 * 2 * 1
31 0

```

What is the precedence order among Python operators?

As shown in Snippet 4.2, line 30, Python expressions can string together multiple operators. So, how does Python know which operation to perform first? The answer to this question lies in operator precedence. When you write an expression with more than one operator, Python groups its parts according to what is called precedence rules,¹⁰ and this grouping determines the order in which the expression's parts are computed. Table 3.4 reports the precedence hierarchy concerning the most common operators. Note that operators lower in the table have higher precedence. Parentheses can be used to create sub-expressions that override operator precedence rules.

TABLE 3.4
Operator Precedence Hierarchy
(Ascending Order)

Operator	Description
<code>x + y</code>	Addition, concatenation
<code>x - y</code>	Subtraction, set difference
<code>x * y</code>	Multiplication, repetition
<code>x % y</code>	Remainder, format;
<code>x / y, x // y</code>	Division: true and floor
<code>-x, +x</code>	Negation, identity
<code>~x</code>	Bitwise NOT (inversion)
<code>x ** y</code>	Power (exponentiation)

How do I carry out technical and scientific computation with Python?

Python is at the center of a rich ecosystem of modules for technical and scientific computation. In the following chapter, the attention will revolve around one of the most prominent modules, namely, [NumPy](#). In a nutshell, [NumPy](#) offers the infrastructure for efficiently manipulating data structures. [SciPy](#) builds on [NumPy](#) to implement many algorithms across the fields of statistics, linear algebra, optimization, calculus, signal processing, image processing, and others. Another core module in the technical and scientific domain is [SymPy](#), a library for symbolic mathematics. Note that none of these three modules are shipped with Python and should be installed with the package manager of your choice (e.g., [conda](#)).

What is a Python variable?

Variables are simply names — created by you or Python — that are used to keep track of information in your program. In Python:

- Variables are created when they are first assigned values
- Variables are replaced with their values when used in expressions
- Variables must be assigned before they can be used in expressions
- Variables refer to objects and are never declared ahead of time

As Snippet 4.3 shows, the assignment of `x = 2` causes the variable `x` to come into existence ‘automatically.’ From that point, we can use the variables in the context of expressions such as the ones displayed in lines 8, 12, 16, and 20 or create new variables like in line 24.

Snippet 4.3 — expressions involving arithmetic operations

```

1
2 # let us assign the variables 'x' and 'y' to two number objects
3 >>> x = 2
4
5 >>> y = 4.0
6
7 # subtracting an integer from variable 'x'
8 >>> x - 1
9 1
10
11 # dividing the variable 'y' by an integer
12 >>> y / 73
13 0.0547945205479452
14
15 # integer-dividing the variable 'y' by an integer
16 >>> y // 73
17 0.0
18
19 # getting a linear combination of 'x' and 'y'
20 >>> 3 * x - 5 * y
21 -14.0
22
23 # assigning the variable 'z' to the linear combination of 'x' and 'y'
24 >>> z = 3 * x - 5 * y
25

```

How do I display number objects in a readable way?

Snippet 4.3 includes some expressions whose result is not passed to a new variable (e.g., lines 8, 12, 16, 20). In those cases, the IPython session displays the expression's outcome as is (e.g., 0.0547945205479452). However, a number with more than three or four decimals may not suit the table or report we must prepare. Python has powerful [string formatting](#) capabilities to display number objects in a readable and nice manner. Table 3.5 illustrates various number formatting options with concrete cases. Format strings contain 'replacement fields' surrounded by curly braces `{}`. Anything not contained in braces is considered literal text, copied unchanged to the output. Snippet 4.4 presents a fully-fledged number formatting case. First, we assign the variable `a` to a floating-point number (line 2). Then, we pass the formatting option `{:.2f}` over the variable `a` using the Python built-in function `format`.

TABLE 3.5
Number Formatting Options in Python

Number	Format	Output	Description
3.1415926	<code>{:.2f}</code>	3.14	Format float 2 decimal places
3.1415926	<code>{:+.2f}</code>	+3.14	Format float 2 decimal places with sign
-1	<code>{:+.2f}</code>	-1.00	Format float 2 decimal places with sign
2.71828	<code>{:.0f}</code>	3	Format float with no decimal places
5	<code>{:0>2d}</code>	05	Pad number with zeros (left padding, width 2)
5	<code>{:x<4d}</code>	5xxx	Pad number with x's (right padding, width 4)
10	<code>{:x<4d}</code>	10xx	Pad number with x's (right padding, width 4)
1000000	<code>{:,}</code>	1,000,000	Number format with comma separator
0.25	<code>{:.2%}</code>	25.00%	Format percentage
1000000000	<code>{:.2e}</code>	1.00e+09	Exponent notation
13	<code>{:10d}</code>	13	Right aligned (default, width 10)
13	<code>{:<10d}</code>	13	Left aligned (width 10)
13	<code>{:~10d}</code>	13	Center aligned (width 10)

Snippet 4.4 — number formatting examples

```

1 # assign the variable 'a' to a floating-point number
2 >>> a = 0.67544908755
3
4 # displaying 'a' with the first two decimals only
5 >>> "{:.2f}".format(a)
6 "0.68"
7
8 # displaying 'a' with the first three decimals only
9 >>> "{:.3f}".format(a)
10 "0.675"

```

How do I compare number objects?

Comparisons are used frequently to create control [flows](#), a topic we will discuss later in this chapter. Normal comparisons in Python regard two number objects and return a Boolean result. Chained comparisons concern three or more objects and, like normal comparisons, yield a Boolean result. Snippet 4.5 provides a sample of normal comparisons (between lines 1 and 15) and chained comparisons (between lines 21 and 30). As evident in the example, comparisons can regard both numbers and variables assigned to numbers. Chained comparisons can take the form of a range test (see line 21), a joined, ‘AND’ test of the truth of multiple expressions (see line 25), or a disjoined, ‘OR’ test of the truth of multiple expressions (see line 29).

Snippet 4.5 — comparing numeric objects

```

1  # less than
2  >>> 3 < 2
3  False
4
5  # greater than or equal
6  >>> 1 <= 2
7  True
8
9  # equal
10 >>> 2 == 2
11 True
12
13 # not equal
14 >>> 4 != 4
15 False
16
17 # range test
18 >>> x = 3
19 >>> y = 5
20 >>> z = 4
21 >>> x < y < z
22 False
23
24 # joined test
25 >>> x < y and y > z
26 True
27
28 # disjoined test
29 >>> x < y or y < z
30 True
31

```

3.2 String Type Fundamentals

What is a string?

A Python string is a positionally ordered collection of other objects. Sequences maintain a left-to-right order among the items they contain: their items are stored and fetched by their relative positions. Strictly speaking, strings are *immutable sequences* of one-character strings; other, more general sequence types include lists and tuples, covered later.

How do I use strings?

Strings are used to record words, contents of text files loaded into memory, Internet addresses, Python source code, and so on. Strings can also hold the raw bytes used for media files and network transfers and the encoded and decoded forms of non-ASCII Unicode text used in internationalized programs.

Is `abc` a Python string?

Nope. Python strings are enclosed in single quotes (`'...'`) or double quotes (`"..."`) with the same result. Hence, `"abc"` can be Python string, while `abc` cannot. `abc` can be a variable name, though.

How do I manipulate string objects?

The fact that strings are immutable sequences affects how we manipulate textual data in Python. In Snippet 4.6, we fetch the individual elements of `S`, a variable assigned to “Python 3.X.” As per the built-in function `len`, `S` contains six unitary strings. That means that each element in `S` is associated with a position in the numerical progression $\{0, 1, 2, 3, 4, 5\}$.

Now, you may be surprised that the list’s first element is 0 instead of 1. The reason is that Python is a zero-based indexed programming language: the first element of a series has an index of 0, while the last part has an index `len(obj) - 1`.

Fetching the individual elements of a string, such as `S`, requires passing the desired index between brackets, as shown in line 9 (where we get the first unitary string, namely, “P”), line 13 (where we get the last unitary string, namely, “X”), and line 21 (where we get the unitary string with index 3, i.e., the fourth unitary string appearing in `S`, “h”). Note that line 17 is an alternative indexing strategy to the one presented in line 13: it is possible to retrieve the last unitary string by counting ‘backward’; that is, getting the first element starting from the right-hand side of the string, which equates to index `-1`.

In lines 26 and 31, we exploit the indices of `S` to retrieve multiple unitary strings in a row. What we pass among brackets is not a single index. Instead, we specify a range of indices `i:j`. It is worth noticing that, in Python, the element associated with the lower bound index is returned. In contrast, the element associated with the upper bound index `j` is not. In line 26, we fetch the unitary strings between index 2 — equating to third unitary string of `S` — and index 5 excluded — namely, the fifth unitary string of `S`. In line 30, we adopt the ‘backward’ approach to retrieve the unitary string with index `-3` — the third string counting from the right-hand side of `S` — as well as any other unitary strings following index `-3`. To do that, we leave the upper bound index blank.

Snippet 4.6 — Python strings as sequences

```
1 # let us assign the string "Python 3.X" to the variable S
2 >>> S = "Python 3.X"
3
4 # check the length of S
5 >>> len(S)
6 6
7
8 # access the first unitary string in the sequence behind S
9 >>> S[0]
10 "P"
11
12 # access the last unitary string in the sequence behind S
13 >>> S[len(S)-1]
14 "X"
15
16 # or, equivalently
17 >>> S[-1]
18 "X"
19
20 # access the i-th, e.g., 3rd, unitary string in the sequence behind S
21 >>> S[3]
22 "h"
23
24 # access the unitary strings between the i-th and j-th positions in the
25 # sequence behind S
26 >>> S[2:5]
27 "tho"
28
29 # access the unitary strings following the i-th position in the sequence
30 # behind S
31 >>> S[-3:]
32 "3.X"
33
```

What are the most common string literals and operators?

Can you share some concrete examples of string manipulation tasks?

Snippet 4.6 deals with string indexing and slicing, two of the many operations we can carry out on strings. Table 3.6 reports a sample of common string literals and operators.

The first two lines of Table 3.6 remind us that single and double quotes are equivalent when assigning a variable to a string object. However, we must refrain from mixing and matching single and double quotes. In other words, a string object requires the leading and trailing quotes are of the same type (i.e., double-double or single-single).

In the interest of consistency, it is a good idea to make a policy choice, such as “in my Python code, I use double quotes only”, and to stick with that throughout the various lines of the script. I prefer using double quotes because the single quote symbol is relatively popular in natural language (consider, for example, the Saxon genitive).

As shown in the third line of Table 3.6, the single quote is treated as a unitary string insofar as double quotes are used to delimit the string object. Should the string object be delimited by single quotes, we should tell Python not to treat the single quote symbol after `m` as a Python special character but as a unitary string. To do that, we use the escape symbol `\` as shown in the fourth line of Table 3.6. Table A.1 provides several escaping examples.

Snippet 4.7 presents a sample of ‘common’ miscellaneous string manipulation tasks. In lines 6 and 9, we check the length of the variables `S1` and `S2`. In line 13, we display five repetitions of `S1`. In line 17, we use the algebraic operator `+` to concatenate `S1` and `S2`. In line 21, we expand on the previous input by separating `S1` and `S2` by a white-space. In line 25, we carry out the same task as line 17 — however, we rely on the built-in `join` function to join `S1` and `S2` with whitespace. The argument taken by `join` is a Python `list`, the subject of paragraph 5.3. In line 29, `S1` and `S2` are joined with a custom string object, namely, `" Vs. "`. Finally, in line 33, we use the built-in `format` function (see also Snippet 4.4) to display a string object including `S1` and `S2`. For a comprehensive list of string methods, see Table A.2.

TABLE 3.6
Sample of String Literals and Operators

Literal/operation	Interpretation
<code>S = ""</code>	Empty string
<code>S = ‘ ’</code>	Single quotes, same as double quotes
<code>S = "spam's"</code>	Single quote as a string
<code>S = 'spam\'s'</code>	Escape symbol
<code>length(S)</code>	Length
<code>S[i]</code>	Index
<code>S[i:j]</code>	Slice
<code>S1 + S2</code>	Concatenate
<code>S * 3</code>	Repeat S <i>n</i> times (e.g., three times)
<code>"text".join(strlist)</code>	Join multiple strings on a character (e.g., "text")
<code>"{}".format()</code>	String formatting expression
<code>S.strip()</code>	Remove white spaces
<code>S.replace("pa", "xx")</code>	Replacement
<code>S.split(",")</code>	Split on a character (e.g., ",")
<code>S.lower()</code>	Case conversion — to lower case
<code>S.upper()</code>	Case conversion — to upper case
<code>S.find("text")</code>	Search substring (e.g., "text")
<code>S.isdigit()</code>	Test if the string is a digit
<code>S.endswith("spam")</code>	End test
<code>S.startswith("spam")</code>	Start test
<code>S = """...multiline..."""</code>	Triple-quoted block strings

Snippet 4.7 — miscellaneous string manipulation tasks

```

1 # let us assign S1 and S2 to two strings
2 >>> S1 = "Python 3.X"
3 >>> S2 = "Julia"
4
5 # check the length of S1 and S2
6 >>> len(S1)
7 10
8
9 >>> len(S2)
10 5
11
12 # display the S1 repeated five times
13 >>> S1 * 5
14 "Python 3.XPython 3.XPython 3.XPython 3.XPython 3.X"
15
16 # display the concatenation of S1 and S2
17 >>> S1 + S2
18 "Python 3.XJulia"
19
20 # display the concatenation of S1, whitespace, and S2
21 >>> S1 + " " + S2
22 "Python 3.X Julia"
23
24 # display the outcome of joining S1 and S2 with a whitespace
25 >>> " ".join([S1, S2])
26 "Python 3.X Julia"
27
28 # display the outcome of joining S1 and S2 with an arbitrary string object
29 >>> " Vs. ".join([S1, S2])
30 "Python 3.X Vs. Julia"
31
32 # string formatting
33 >>> "Both {} and {} have outstanding ML modules".format(S1, S2)
34 "Both Python 3.X and Julia have outstanding ML modules"

```

Can you share some concrete examples of string editing tasks?

Snippet 4.8 illustrates some string editing tasks. In line 5, we use `rstrip` — a variation of the built-in function `strip` — that returns a copy of the string with leading characters removed. In line 9, we use the built-in `replace` to return a copy of the string with all occurrences of substring `old` (first argument taken by the function) replaced by `new` (second argument taken by the function). Finally, in line 17, we use the built-in function `lower` to return a copy of the string with all the cased characters converted to lowercase.

Snippet 4.8 — miscellaneous string editing tasks

```
1 # let us assign S to a string object
2 >>> S = "Both Python 3.X and Julia have outstanding ML modules"
3
4 # strip target leading characters
5 >>> S.lstrip("Both ")
6 "Python 3.X and Julia have outstanding ML modules"
7
8 # replace target characters
9 >>> S.replace("Python 3.X", "R")
10 "Both R and Julia have outstanding ML modules"
11
12 # split string on target characters
13 >>> S.split(" and ")
14 ["Both Python 3.X", "Julia have outstanding ML modules"]
15
16 # make the string lower case
17 >>> S.lower()
18 "both python 3.x and julia have outstanding ml modules"
```

How do I test or search string attributes?

Snippet 4.9 presents a series of string test and search tasks. The built-in function `find` (see lines 5 and 9) returns the lowest index in the string where substring `sub` is found within the slice `S[start:end]` or `-1` if substring is not found. The built-in function `isdigit` return `True` if all characters in the string are digits and there is at least one character, `False` otherwise. Finally, the built-in function `endswith` returns `True` if the string ends with the specified suffix, otherwise returns `False`.

Snippet 4.9 — miscellaneous string test and search tasks

```
1 # let us assign S to a string object
2 >>> S = "The first version of Python was released in 1991"
3
4 # search for "Python" in S
5 >>> S.find("Python")
6 21
7
8 # search for "Julia" in S
9 >>> S.find("Julia")
10 -1
11
12 # slice the string the get Python's release year information
13 >>> SS = S[-4:]
14
15 # display SS
16 >>> SS
17 "1991"
18
```

```

19 # test if all characters in SS are digits
20 >>> SS.isdigit()
21 True
22
23 # test if all characters in SS are digits
24 >>> SS.isdigit()
25 True
26
27 # test if all characters in SS are digits
28 >>> SS.isdigit()
29 True
30
31 # test if S ends with "1991" / SS
32 >>> S.endswith(SS)
33 True

```

Can I display 'complex' string objects?

We just came across the built-in function `print`. Such a function can print both number- and string-type objects. Sometimes, what we want to print fits into a single line. In other circumstances, we are interested in visualizing rich data which can span multiple lines. Snippet 4.9 how to print objects across multiple lines with the triple-quoted block string (see line). As evident from the Python code in lines 8-13, any line between triple quotes is considered part of the same string object.

Snippet 4.9 — multiline string printing

```

1 # single-line print
2 >>> print("Hello world!")
3 Hello world!
4
5 # multi-line print
6 >>> print(
7 ... """
8 ... =====
9 ... COL A      | COL B      | ...      | COL K
10 ... -----
11 ... Sheldon    | Cooper    | ...      | bazinga.com
12 ... -----
13 ... NOTES: this table has fake data
14 ... """
15 ... )
16
17 =====
18 COL A      | COL B      | ...      | COL K
19 -----
20 Sheldon    | Cooper    | ...      | bazinga.com
21 -----
22 NOTES: this table has fake data
23

```

3.3 List and Dictionaries

What is a list?

A Python `list` is an *ordered, mutable* array of objects. A list is constructed by specifying the objects, separated by commas, between square brackets, `[]`.

Why should I use lists?

Lists are just places to collect other objects so you can treat them as groups.

What type of objects can I include in a list?

Lists can contain any sort of object: numbers, strings, and even other lists. See Snippet 4.10.

Snippet 4.10 — sample lists with different items

```
1 # an empty list
2 >>> L = []
3
4 # a list with an integer, a float, and a string
5 >>> L = [2, -3.56, "XyZ"]
6
7 # a list with an integer and a list
8 >>> L = [4, ["abc", 8.98]]
9
```

How can I fetch a list's elements?

We can retrieve one or more list component objects via indexing. That is possible because list items are ordered by their position (similarly to strings). Since Python is a zero-based indexed programming language, to fetch the first item of a list, we have to call the index 0 (see Snippet 4.11, line 5). A list nested in another list can be fetched using multiple indices (line 13). The first index refers to the outer list, while any subsequent index refers to an inner list. In our case, we have two indices, one for each list; that is, `L` and its sub-list `["abc", 8.98]`.

Snippet 4.11 — list indexing and slicing

```

1 # the list
2 >>> L = [4, ["abc", 8.98]]
3
4 # get the first item of L
5 >>> L[0]
6 4
7
8 # get the second element of L
9 >>> L[1]
10 ["abc", 8.98]
11
12 # get the first item of L's second item
13 >>> L[1][0]
14 "abc"
15

```

Can I edit a list's elements?

Lists are mutable objects, which may be changed in place by assignment to offsets and slices, list method calls, deletion statements, and more. Snippet 4.12 illustrates some snippets to change a list's items. In the first part of the example, we change the items using indexing (line 5) and slicing (line 10). In the second half, we use Python's `del` statement to delete the items using indexing (line 15) and slicing (line 20).

Snippet 4.12 — changing and deleting list items in place

```

1 # the list
2 >>> L = ["Leonard", "Penny", "Sheldon"]
3
4 # change the second item of L via indexing
5 >>> L[1] = "Raj"
6 >>> print(L)
7 ["Leonard", "Raj", "Sheldon"]
8
9 # change multiple items of L via slicing
10 >>> L[0:2] = ["Amy", "Howard"]
11 >>> print(L)
12 ["Amy", "Howard", "Sheldon"]
13
14 # delete the first item of L via indexing and using the 'del' statement
15 >>> del L[0]
16 >>> print(L)
17 ["Howard", "Sheldon"]
18
19 # delete multiple items of L via slicing and using the 'del' statement
20 >>> del L[0:2]
21 >>> print(L)
22 []
23

```

What are the built-in methods to manipulate a list?

Python offers many [methods to manipulate and test list objects](#). Table 3.7 reports some of the most popular methods and synopses. The first three methods — `.append()`, `.insert()`, and `.extend()` — expand an existing list. The fourth method, `.index` test for the presence of an item in an existing list. Note this method raises a [ValueError](#) if there is no such item. The remaining methods produce in-place changes in an existing list's items.

TABLE 3.7
Popular List Methods

Method	Synopsis
<code>L.append(X)</code>	Append an item to an existing list
<code>L.insert(i, X)</code>	Append an item to an existing list in position <i>i</i>
<code>L.extend([X0, X1, X2])</code>	Extend an existing list with the items from another list
<code>L.index(X)</code>	Get the index of the first instance of the argument in an existing list
<code>L.count(X)</code>	Get the cardinality of an item in an existing list
<code>L.sort()</code>	Sort the items in an existing list
<code>L.reverse()</code>	Reverse the order of the items in an existing list
<code>L.copy()</code>	Get a copy of an existing list
<code>L.pop(i)</code>	Remove the item at the given position in the list, and return it
<code>L.remove(X)</code>	Remove the first instance of an item in an existing list
<code>L.clear()</code>	Remove all items in an existing list

How can I expand an existing list?

Both the `.append()` and `.extend()` methods can be used to expand an existing list as per Snippet 4.13. However, they accomplish different goals and should not be confused: `.append()` adds a new item (of any type) to the end of the list (see line 6); `.extend()` extend the list by appending all the items from another iterable object (e.g., another list, see line 11).

Snippet 4.13 — methods for expanding an existing list

```

1  # create two lists
2  >>> L1 = ["Leonard", "Penny", "Sheldon"]
3  >>> L2 = ["Howard", "Raj", "Amy", "Bernadette"]
4
5  # expand an existing list with .append()
6  >>> L2.append("Priya")
7  >>> print(L2)
8  ["Howard", "Raj", "Amy", "Bernadette", "Priya"]
9
10 # concatenate L1 and L2 with .extend()
11 >>> L1.extend(L2)
12 >>> print(L1)
13 ["Leonard", "Penny", "Sheldon", "Howard", "Raj", "Amy", "Bernadette", "Priya"]
14

```

Can I change the order of a list's elements?

One of the most common list manipulation tasks consists of changing the order of an item's list. As shown in Snippet 4.14, it is possible to use the `.reverse()` method to reverse the elements of the list in place (see line 5) while sorting an item's list can be carried out with the `.sort()` method.

Snippet 4.14 — methods for changing list items in place

```

1 # create a list
2 >>> L = ["Howard", "Raj", "Amy", "Bernadette", "Priya"]
3
4 # reverse the list's item positions
5 >>> L.reverse()
6 >>> print(L)
7 ["Priya", "Bernadette", "Amy", "Raj", "Howard"]
8
9 # sort the list's items
10 >>> L.sort()
11 >>> print(L)
12 ["Amy", "Bernadette", "Howard", "Priya", "Raj"]
13

```

3.4 Dictionaries

What is a dictionary?

Along with lists, dictionaries are one of the most flexible built-in data types in Python. If you think of lists as ordered collections of objects, you can think of dictionaries as unordered collections; the chief distinction is that in dictionaries, items are stored and fetched by *key* instead of by *positional offset*.

Why should I use a dictionary?

Dictionaries take the place of records, search tables, and any other sort of aggregation where item names are more meaningful than item positions.

What type of objects can I include in a dictionary?

Like lists, dictionaries can contain objects of any type, and they support nesting to any depth(they can contain lists, other dictionaries, and so on). Each key can have just one associated value, but that value can be a collection of multiple objects if needed, and a given value can be stored under any number of keys.

How do I create a dictionary?

Snippet 4.15 shows two different ways to create a dictionary. A dictionary can be created by including key-value pairs among curly braces (see line 2). In the example, there are three keys associated with Marvel characters and as many values, which can be thought of as the characters' position in an ideal power rank. A colon separates a key and its associated value. The second way to create a dictionary is based on Python's builtin `dict`, mapping key onto values, and `zip`, which iterates over two elements in parallel. Specifically, `zip` creates the one-to-one correspondence between keys (characters) and values (character's power) that is passed as the argument of `dict`. We will analyze the topic of iterations extensively in sections 3.10 and 3.11.

Snippet 4.15 — initializing a new dictionary object

```

1 # method 1
2 >>> D = {"Captain Marvel": 3, "Living Tribunal": 2, "One-Above-All": 1}
3
4 # method 2
5 >>> CHARACTERS = ["Captain Marvel", "Living Tribunal", "One-Above-All"]
6 >>> RANK = [3, 2, 1]
7 >>> D = dict(zip(CHARACTERS, RANK))
8 >>> print(D)
9 {"Captain Marvel": 3, "Living Tribunal": 2, "One-Above-All": 1}

```

How do I fetch a dictionary's values?

Dictionaries' items cannot be accessed via positional offsets — like lists. Instead, we fetch the individual items using the dictionary keys shown in Snippet 4.16 (see line 5). The reference key is passed among brackets. When the dictionary at hand contains nested dictionaries (see line 9), it is possible to concatenate multiple queries, namely, sequences of keys between brackets (see line 21).

Snippet 4.16 — fetching dictionary items

```

1 # the dictionary
2 >>> D = {"Captain Marvel": 3, "Living Tribunal": 2, "One-Above-All": 1}
3
4 # let's fetch Captain Marvel's position in the Marvel characters' power rank
5 >>> D["Captain Marvel"]
6 3
7
8 # a dictionary of dictionaries
9 >>> D = {
10     "Dr. Strange": {
11         "first_appearance": 1963,
12         "created_by": "Lee & Ditko"
13     },
14     "Iron Man": {
15         "first_appearance": 1963,
16         "created_by": "Lee, Lieber, Heck & Kirby"
17     },
18 }
19
20 # let us fetch the creator of Dr. Strange
21 >>> D["Dr. Strange"]["created_by"]
22 "Lee & Ditko"

```

Are dictionaries mutable?

Dictionaries, like lists, are mutable. Thus, we can change, expand, and shrink them in place without making new dictionaries: simply assign a value to a key to change or create an entry. The `del` statement works here, too; it deletes the entry associated with the key specified as an index (see Snippet 4.17).

Snippet 4.17 — dictionary mutability examples

```

1 # the dictionary
2 >>> D = {"Captain Marvel": 3, "Living Tribunal": 2, "One-Above-All": 1}
3
4 # let us change the power rank for Captain Marvel
5 >>> D["Captain Marvel"] = 12
6 >>> print(D)
7 {"Captain Marvel": 12, "Living Tribunal": 2, "One-Above-All": 1}
8
9 # let us eliminate the character Living Tribunal
10 >>> del D["Living Tribunal"]
11 >>> print(D)
12 {"Captain Marvel": 12, "One-Above-All": 1}
13
14 # let us add a further character
15 >>> D["Wanda Maximoff"] = 4
16 >>> print(D)
17 {"Captain Marvel": 12, "One-Above-All": 1, "Wanda Maximoff": 4}
18

```

What are the most common methods to manipulate a dictionary?

Like for lists, Python offers many [methods to manipulate dictionary objects](#). Table 3.8 reports some of the most common methods and synopses. The first three methods, `.keys()`, `.values()`, `.items()`, get the constitutive elements of dictionaries: keys, values, and key-value pairs, respectively. The fourth method, `.get(key, default?)` gets the value for a specific key. The fifth method, `.update()`, updates the value for a specific key. Like `.update()`, `.popitem()`, `.pop()`, and `d.clear()` alter the information of a dictionary in place. The first removes the value of a certain key; the second removes the item (a key-value pair) for a certain key; the latter deletes all dictionary items. Finally, `.copy()` creates a [shallow copy](#) of an existing dictionary.

TABLE 3.8
Popular Dictionary Methods

Method	Synopsis
<code>D.keys()</code>	Get all dictionary keys
<code>D.values()</code>	Get all dictionary values
<code>D.items()</code>	Get all dictionary key-value pairs as tuples
<code>D.get(key, default?)</code>	Query a dictionary element by key
<code>D.update(D2)</code>	Update a dictionary key's value
<code>D.popitem()</code>	Remove the value corresponding to a certain key
<code>D.pop(key, default?)</code>	Remove the item at the given position in the list,
<code>D.clear()</code>	Delete all dictionary items
<code>D.copy()</code>	Copy the target dictionary

How do I access the information in a dictionary?

Snippet 4.18 shows how to use built-in methods to carry out three fundamental tasks: accessing dictionary keys (see line 5), values (see line 9), and items (i.e., key-value pairs, see line 13). It is worth noticing that the three methods illustrated in the example yield specific [dictionary objects](#) such as `dict.keys`, `dict.values`, and `dict.items`. Translating one of these dictionary objects into a list — if needed — is straightforward (see line 17).

Snippet 4.18 — accessing the information included in a dictionary

```

1 # the dictionary
2 >>> D = {"Captain Marvel": 3, "Living Tribunal": 2, "One-Above-All": 1}
3
4 # get the keys
5 >>> D.keys()
6 dict_keys(["Captain Marvel", "Living Tribunal", "One-Above-All"])
7
8 # get the values
9 >>> D.values()
10 dict_values([3, 2, 1])
11
12 # get the items
13 >>> D.items()
14 dict_items([("Captain Marvel", 3), ("Living Tribunal", 2), ("One-Above-All", 1)])
15
16 # get the keys as a list
17 >>> list(D.keys())
18 ["Captain Marvel", "Living Tribunal", "One-Above-All"]

```

3.5 Tuples

What is a tuple?

Tuples are sequences of immutable Python objects. They are similar to lists, but they are immutable. Tuples are created by enclosing a comma-separated list of values in parentheses.

Are Tuples mutable?

Tuples are immutable, which means that once they are created, they cannot be changed!!

Why should I use tuples?

Tuples are useful for storing data that is not to be changed, such as the coordinates of a point in a two-dimensional space. In general, we use tuples any time information integrity is a concern — in other words when we want to ensure the information included in an object will not change because of another reference in our program.

How do I create a tuple?

Python objects, separated by a comma, must be included between parentheses (see Snippet 4.19, line 2).

How do I access the information in a tuple?

By positional offsets, like lists (see Snippet 4.19, lines 5 and 9).

Snippet 4.19 — creating and accessing a tuple

```

1 # the tuple
2 >>> T = ("Captain Marvel", 3)
3
4 # access a tuple element
5 >>> T[0]
6 "Captain Marvel"
7
8 # access a tuple element
9 >>> T[1]
10 3

```

Can I convert a tuple into a list?

Yes, you can. To do that, you must pass the tuple as the argument of `list` (see Snippet 4.20).

Snippet 4.20 — tuple conversion

```

1 >>> T = ("Captain Marvel", 3)
2
3 # from a tuple to a list
4 >>> L = list(T)
5 >>> print(L)
6 ["Captain Marvel", 3]
7
8 # amend L's items
9 >>> L[1] = 4
10
11 # get back to a tuple
12 >>> T = tuple(L)
13 >>> print(T)
14 ("Captain Marvel", 4)

```

Can I create an advanced data container based on a tuple?

`collections` is a module that is shipped with Python and provides data containers that are alternative to Python's general purpose built-in containers, i.e., `dict`, `list`, `set`, and `tuple`. One of these containers can be created with the function `namedtuple` (see Snippet 4.21), which allows annotating the tuple items with names. In line 2, we import the function `namedtuple` from the `collections` module. In line 5, we create an ad hoc class that best represents the structure of our sample data concerning Marvel characters' names and the year they first appeared in the comic series. The first argument taken by the function is customary and regards the name of the class we are about to create. The second argument is a list with the names of the attributes included in our data structure. In line 8, we use the newly created class `Rec` to create a tuple, which is eventually printed as per line 11.

Snippet 4.21 — creating an annotated tuple with the collection module

```

1 # import the named tuple function from the module collection
2 >>> from collections import namedtuple
3
4 # create an ad hoc class object 'Rec' that fits our data structure
5 >>> Rec = namedtuple("Rec", ["character", "first_appearance"])
6
7 # use the generated class "Rec"
8 >>> IRONMAN = Rec("Iron Man", 1963)
9
10 # A named-tuple record
11 >>> IRONMAN
12 Rec(character="Iron Man", first_appearance=1963)
```

3.6 Sets

What is a set?

A `set` is an *unordered* collection of *unique* and *immutable* objects.

What does it mean that sets are unordered collections?

By design, `set` is a data structure with *undefined element ordering* (see Snippet 4.22 — the outcome included in line 6 does not follow any particular order).

What does it mean that sets have unique items?

By definition, an item appears only once in a set, no matter how many times it is added (see Snippet 4.22, line 2 Vs. line 7).

Snippet 4.22 — creating a set

```

1 # create a list
2 >>> L = ["a", "a", "b", "c", "c"]
3
4 # get a set from L
5 >>> S = set(L)
6 >>> print(S)
7 {"b", "a", "c"}

```

Why should I use sets?

Sets made this way support common mathematical set operations (see Snippet 4.23). Hence, they have a variety of applications, especially in numeric and database-focused work.

Snippet 4.23 — set operations

```

1 # create two sets
2 >>> X = set(["a", "b", "c"])
3 >>> Y = set(["c", "d", "e"])
4
5 # set difference
6 >>> X - Y
7 set()
8 >>> Y - X
9 {"a", "b"}
10
11 # union
12 >>> X | Y
13 {"a", "b", "c", "d", "e"}
14
15 # intersection
16 >>> X & Y
17 {"c"}
18
19 # superset
20 >>> X > Y
21 False
22
23 # subset
24 >>> X < Y
25 False

```

3.7 Files

How do the files in my operating system relate with Python?

How do I open a file?

How do I source the data stored in a file?

Your Python program may involve input and/or output operations. In other words, you may want to read data from a file stored in your machine and/or write the outcome of your analysis to a file. The built-in function `open` creates a Python file object, which serves as a link to a file residing on your machine. As Lutz notes:

“Compared to the types you’ve seen so far, file objects are somewhat unusual. They are considered a core type because a built-in function creates them, but they’re not numbers, sequences, or mappings, and they don’t respond to expression operators; they export only methods for common file-processing tasks” (page 282)

You open a pipe to a file using the built-in function `open`. The output of the function is a `file` object.

You open a pipe to a file using the built-in function `open`. The output of the function is a `file` object. Snippet 4.24 illustrates how to use `open` for data sourcing. In the first part of the snippet, we create a `file` object to read the data included in the existing file `my_file.txt`.¹¹ At least, we have to pass one argument to `open`: the path pointing to the file. A second optional argument is `mode`, which specifies the mode in which the file is opened to the source. It defaults to `r`, which means open for reading in text mode. Other common values are `w` for writing,¹² `x` for exclusive creation, and `a` for appending.¹³ To read a file’s contents, we use the `.read()` method (see line 14), returning a string object (see line 15).

Snippet 4.24 — data input with open

```

1 # a file to read. NOTE Snippet 4.25 is what normally creates it; these two
2 # lines are here so this snippet stands on its own
3 >>> with open("my_file.txt", "w") as pipe:
4 ...     pipe.write("Hi there\n")
5
6 # create a pipe to a file
7 >>> file = open(file="my_file.txt", mode="r")
8
9 # calling "file" yields the attributes of the file object
10 >>> file
11 <_io.TextIOWrapper name='my_file.txt' mode='r' encoding='UTF-8'>
12
13 # let us source the data
14 >>> data = file.read()
15 >>> print(data)
16 Hi there
17
18 # close the pipe
19 >>> file.close()

```

How do I write the data in the current Python session to a file?

Snippet 4.25 illustrates how to use `open` for data writing. In the first part of the snippet, we create three strings — i.e., the information we are manipulating in the active Python session (see lines 2, 4, and 6). Then, we create a file object in ‘writing’ mode (see the value passed to `mode`, line 9). Finally, we manipulate the three strings (as a sample task, in line 12, we concatenate `FIRSTLAW`, `SECONDLAW`, `THIRDLAW`) and write the result to a file (line 16).

Snippet 4.25 — data output with open

```

1 # the strings (data) to save permanently to a file
2 >>> FIRSTLAW = "A robot may not injure a human being or, through inaction, "\
3               "allow a human being to come to harm."
4 >>> SECONDLAW = "A robot must obey the orders given it by human beings except "\
5               "where such orders would conflict with the First Law."
6 >>> THIRDLAW = "A robot must protect its own existence as long as such "\
7               "protection does not conflict with the First or Second Law."
8
9 # create a pipe to a file
10 >>> file = open(file="my_file.txt", mode="w")
11
12 # concatenate the strings
13 >>> TO_WRITE = "\n".join([FIRSTLAW, SECONDLAW, THIRDLAW])
14
15 # write the concatenated strings
16 >>> file.write(TO_WRITE)
17
18 # close the pipe

```

```
19 >>> file.close()
```

How about reading a single line from a file?

Hold on: what is a line? A string whose last character is `\n`. We can read a single line from a file using the `.readline()` method (see Snippet 4.26). Such a method starts by reading the first line included in the file (see line 11); then, it reads any subsequent lines included in the file (see line 15); when it reaches the end of the file (EOF), it returns the empty string `""` (see line 19).

Snippet 4.26 — reading one line at a time with `.readline()`

```

1 # the strings (data) to save permanently to a file
2 >>> DATA = "The first line\nThe second line"
3
4 # create a pipe to a file and write DATA
5 >>> file = open(file="my_file.txt", mode="w")
6 >>> file.write(DATA)
7 >>> file.close()
8
9 # read one line from the file
10 >>> file = open(file="my_file.txt", mode="r")
11 >>> file.readline()
12 "The first line\n"
13
14 # calling file.readline() again reads the subsequent line
15 >>> file.readline()
16 "The second line"
17
18 # ... and so on until the end of the file is reached
19 >>> file.readline()
20 ""

```

How about reading multiple lines at a time?

The `.readlines()` method reads the lines from a file and returns them as a list (see Snippet 4.27).

Snippet 4.27 — reading multiple lines at a time with `.readlines()`

```

1 # the strings (data) to save permanently to a file
2 >>> DATA = "A\nB\nC\nD"
3
4 # create a pipe to a file and write DATA
5 >>> file = open(file="my_file.txt", mode="w")
6 >>> file.write(DATA)
7 >>> file.close()
8
9 # read multiple lines
10 >>> file = open(file="my_file.txt", mode="r")
11 >>> file.readlines()
12 ['A\n', 'B\n', 'C\n', 'D']
13

```

What are the most common file methods?

Table 3.9 illustrates some key file methods' names and their corresponding synopsis.

TABLE 3.9
Popular File Methods

Method	Description
<code>file.close()</code>	Closes the file
<code>file.detach()</code>	Returns the separated raw stream from the buffer
<code>file.fileno()</code>	Returns a number that represents the stream as per the OS' perspective
<code>file.flush()</code>	Flushes the internal buffer
<code>file.isatty()</code>	Returns whether the file stream is interactive or not
<code>file.read()</code>	Returns the file content
<code>file.readable()</code>	Returns whether the file stream can be read or not
<code>file.readline()</code>	Returns one line from the file
<code>file.readlines()</code>	Returns a list of lines from the file
<code>file.seek()</code>	Change the file position
<code>file.seekable()</code>	Returns whether the file allows us to change the file position
<code>file.tell()</code>	Returns the current file position
<code>file.truncate()</code>	Resizes the file to a specified size
<code>file.writable()</code>	Returns whether the file can be written to or not
<code>file.write()</code>	Writes the specified string to the file
<code>file.writelines()</code>	Writes a list of strings to the file

Notes: `file` is a fictionary object used to illustrate the usage of the file methods.

3.8 Python Statements and Syntax

What is a Python statement?

In his popular book ‘Learning Python,’ Lutz provides a concise and effective description of what a Python statement is:

In simple terms, statements are the things you write to tell Python what your programs should do. If, as suggested [omitted], programs “do things with stuff,” then statements are the way you specify what sort of things a program does. Less informally, Python is a procedural, statement-based language; by combining statements, you specify a procedure Python performs to satisfy a program’s goals.

What are the most common Python statements?

Table 3.10 illustrates common Python statements, their role, and application examples. Some of these statements were used in the examples considered so far. Other statements — the majority — will be faced in the next sections of the current chapter and/or the subsequent chapters.

TABLE 3.10
Python Statements

Statement	Role	Example
<code>import</code>	Module access	<code>import math</code>
<code>from</code>	Attribute access	<code>from math import sqrt</code>
<code>class</code>	Building ad hoc objects	<code>class Subclass(Superclass):</code>
<code>del</code>	Deleting references	<code>def method(self):</code>
Assignment	Creating references	<code>pass</code>
Calls and other expressions	Running functions	<code>del a</code>
<code>print</code>	Printing objects	<code>a = "before b"</code>
<code>if/elif/else</code>	Selecting actions	<code>file.write("Hello")</code>
<code>for/else</code>	Iteration	<code>print("Hello")</code>
<code>while/else</code>	General loops	<code>if "abc" in text: print(text)</code>
<code>pass</code>	Empty placeholder	<code>for x in mylist: print(x)</code>
<code>break</code>	Loop exit	<code>while X > Y: print("Hello")</code>
<code>continue</code>	Loop continue	<code>while True: pass</code>
<code>def</code>	Functions and methods	<code>while True: if exit test(): break</code>
<code>return</code>	Functions results	<code>while True: if skip test(): continue</code>
<code>yield</code>	Generator functions	<code>def f(a, b, c=1, *d): print(a+b+c+d[0])</code>
		<code>def f(a, b, c=1, *d): return a+b+c+d[0]</code>
		<code>def gen(n): for i in n: yield i*2</code>

3.9 Control Flow (or If-Then Statements)

What is control flow in Python?

Many Python statements we write are compound statements: there is one statement nested inside another. The outer statement is called the ‘if’ statement, and the inner statement is called the ‘then’ statement. The ‘if’ statement determines whether to execute the ‘then’ statement. Specifically, the ‘then’ statement is executed insofar as the ‘if’ statement evaluates to ‘True.’ Snippet 4.28 illustrates a control flow case, a simple rule-based product recommender that suggests products based on users’ purchasing patterns. If product `x` belongs to a user’s set of past purchases, then a certain item is recommended; otherwise, no recommendation is offered (see lines 7 and 8, containing the `else` statement).

Snippet 4.28 — an example of control flow

```
1 # a set with a customer's past purchases
2 >>> S = set(["a", "x", "u"])
3
4 # a rule-based product recommender
5 >>> if "x" in S:
6 ...     print("Customers who bought x also bought Air Jordan 7 Retro Miro")
7 ... else:
8 ...     pass
9 Customers who bought x also bought Air Jordan 7 Retro Miro
```

Does ‘end of the line’ mean ‘end of the statement’?

Any Python statements are contained in the same line — the end of the line equates to the statement end. In the interest of redundancy, Python statements do not traverse multiple lines. In Snippet 4.28, the if statement is in line 5; the then statement is in line 6.

How do I create a nested statement?

The colon character is required to separate the if statement from the ‘then’ statement (see Snippet 4.28 line 5).

Does indentation have substantive meaning in Python?

Yes, it does. ‘Then’ statements are indented (with a tab or four consecutive spaces). Do not creatively use indents to embellish your code — that is not consistent with Python’s rules and design principles (see Snippet 4.28 line 6).

Does ‘end of indentation’ mean ‘end of nested statements’?

How do I concatenate multiple 'then' statements?

'Then' statements are indented (with a tab or four consecutive spaces). In Snippet 4.28, the indentation in line 6 makes lines 5 and 6 to be evaluated together.

Exmple 4.29 shows how to use `elif` to concatenate multiple 'if-then' statements in the same control flow. Like in Snippet 4.28, the 'else' statement defines the residual behavior of the control flow; that is, what Python does when both the 'if' and 'elif' statements evaluate to 'False.'

Snippet 4.29 — an example of control flow with multiple 'if-then' statements

```

1 # a list with a customer's past purchases
2 >>> S = set(["a", "w", "u"])
3
4 # a rule-based product recommender
5 >>> if "x" in S:
6 ...     print("Customers who bought x also bought Air Jordan 7 Retro Miro")
7 ... elif "w" in S:
8 ...     print("How about Converse Chuck Taylor All Star?")
9 ... else:
10 ...     print("Falling short of suggestions --- I'm a dull recommender!")
11 How about Converse Chuck Taylor All Star?
```

How do I create nested if-then statements?

In Python, it is possible to nest an if-then statement into another. In Snippet 4.30, the 'if' statements in lines 6 and 8 are nested inside the 'if' statement in line 5. It is worth noticing that lines 7 and 9 — regarding 'then' statements — are indented twice because they terminate distinct if-then statements nested in the broader if-then statement commencing on line 5.

Snippet 4.30 — an example of nested control flow

```

1 # a list with a customer's past purchases
2 >>> S = set(["a", "x", "b"])
3
4 # a rule-based product recommender
5 >>> if "x" in S:
6 ...     if "a" in S:
7 ...         print("Customers who bought x & a also bought Air Force")
8 ...     elif "u" in S:
9 ...         print("Customers who bought x & u also bought Air Max 95")
10 ... else:
11 ...     print("Falling short of suggestions --- I'm a dull recommender!")
12 Customers who bought x & a also bought Air Force
```


3.10 While and For Loops

Loops?!

Oftentimes, we write Python statements that repeat the same task — i.e. they loop a certain number of times or over multiple items.

How do I write loops in Python?

Using `for` and `while` statements.

What is the difference between `for` and `while` statements?

The `while` statement provides a way to code general loops. The `for` statement is designed for stepping through the items in a sequence or other iterable object and running a block of code for each.

Snippet 4.31 — while loop examples

```

1  # loop until reaching a numeric threshold
2  >>> i = 0
3  >>> while i <= 3:
4  ...     print(i)
5  ...     i = i + 1
6  0
7  1
8  2
9  3
10
11 # loop until an empty string is returned
12 >>> x = "Indiana Jones"
13 >>> while x != "":
14 ...     print(x)
15 ...     x = x[1:]
16 ...
17 Indiana Jones
18 ndiana Jones
19 diana Jones
20 iana Jones
21 ana Jones
22 na Jones
23 a Jones
24 Jones
25 Jones
26 ones
27 nes
28 es
29 s

```

Can you give an example of a `while` loop?

Can you give an example of a `for` loop?

`while` statements run a code block insofar as a test evaluates to `True`. In the upper section of Example 4.31, we assign a `i` to a number. Then we create a `for` loop with the following elements: the first one is a statement testing whether `i` is smaller or equal to 3 (see line 3); the second element is the loop body (indented), which is repeated as long as the test evaluates to `True`. It is worth noticing that every iteration of the loop body produces a unitary increase in `i` — the program leaves the loop after four iterations. In the lower section of Snippet 4.31, we assign the variable `x` to a string (line 13), which we eventually print (line 14) and slice (line 15) until we get an empty string (line 13x).

`for` steps through a sequence of items and carries out a task. In the upper section of Snippet 4.32, we print the result of a mathematical operation deployed over the items of a `list` (an example of a Python iterable object). The code included in line 2 assigns the variable `item` to an element of iterable temporarily. Then, the code block (indented) is executed over the temporary object. In the lower section of Snippet 4.32, the execution of the loop operates a mathematical expression over the items of a first list and appends the outcome to a second list (line 10).

Snippet 4.32 — a `for` loop example

```

1 # print the result of a mathematical operation carried out over a list of items
2 >>> for item in [0, -99, 13, 6.54]:
3     ...     print(item ** 0.5)
4 0.0
5 (6.092540900222253e-16+99498743710662j)
6 3605551275463989
7 2.5573423705088842
8 # run a mathematical operation on a list of items and append the outcome
9 # to a second list
10 >>> input = [2, 8, 1]
11 >>> output = []
12 >>> for item in input:
13     ...     output.append(item + 1)
14 >>> print(output)
15 [3, 9, 2]
```

How do I use `for` loops with dictionaries?

Like lists, dictionaries are iterable objects. In the upper section of Snippet 4.33, we create a dictionary and iterate over its items printing a simple predicate. As we know from section 3.4, we access a dictionary's values by keys. Hence, in line 5, we retrieve the keys of `D`. Then, in line 12, we fetch the value of the temporary object `k`, namely, `D[k]`. Particularly, we print the temporary object `k`, the string object `IS`, and the value associated with `k`; that is, `D[k]`. In Snippet 34, we accomplish the same task of Snippet 33. However, the loop regards a dictionary's items — i.e., key-value pairs — instead of keys (that is self-evident from the comparison of Snippet 4.33's line 12 and Snippet 4.34's line 8).

Snippet 4.33 — looping on dictionary keys

```

1 # the dictionary
2 >>> D = {"Thor": "Asgardian", "Vision": "android", "Wanda Maximoff": "human"}
3
4 # get the keys of D
5 >>> keys = D.keys()
6 >>> print(keys)
7 dict_keys(['Thor', 'Vision', 'Wanda Maximoff'])
8
9 # iterate over the keys to fetch the dictionary values and do something
10 # with them
11 >>> for k in keys:
12 ...     print(k + " IS " + D[k])
13 Thor IS Asgardian
14 Vision IS android
15 Wanda Maximoff IS human

```

Snippet 4.34 — looping on dictionary items

```

1 # the dictionary
2 >>> D = {"Thor": "Asgardian", "Vision": "android", "Wanda Maximoff": "human"}
3
4 # get the items of D
5 >>> items = D.items()
6
7 # iterate over key-value pairs and do something with them
8 >>> for k, v in items:
9 ...     print(k + " IS " + v)
10 Thor IS Asgardian
11 Vision IS android
12 Wanda Maximoff IS human

```

Why are counter for loops so popular?

The built-in class `range` provides an immutable sequence that is particularly helpful for loops that repeat an action a certain number of times. Snippet 35 shows an example of a for loop with `range`.

Snippet 4.35 — a counter loop example

```

1 # show the outcome of range
2 >>> list(range(3))
3 [0, 1, 2]
4
5 # use range in a for loop
6 >>> for i in range(3):
7 ...     print(i, ":-)")
8 0 :-)
9 1 :-)
10 2 :-)

```

What is a nested for loop?

A Python statement that contains multiple `for` loops is a nested `for` loop. Mainly, a `for` loop allows to jointly carry out a task over the elements of two iterables. The outer loop considers the individual items of the first iterable (see Snippet 4.26, line 6); the inner loop (indented) considers the individual items of the second list (see line 7). Once we have created a pair of temporary objects, we can do something with it (see line 8).

Snippet 4.36 — a nested loop example

```

1 # the lists
2 >>> LETTERS = ["x", "y", "z"]
3 >>> COLORS = ["blue", "green", "red"]
4
5 # create all permutations of letters and colors and print them
6 >>> for i in LETTERS:
7 ...     for j in COLORS:
8 ...         print(i, " - ", j)
9 x <-> blue
10 x <-> green
11 x <-> red
12 y <-> blue
13 y <-> green
14 y <-> red
15 z <-> blue
16 z <-> green
17 z <-> red

```

Nested for loops Vs. `zip` for loops?

Contrarily to the nested `for` loops, which considers all permutations containing multiple iterables' items, the built-in `zip` steps through several iterables *in parallel*, producing tuples with an item from each one. As shown in Snippet 4.37, there is neither an inner nor an outer `for` loop in this case — instead, there is a single loop considering two temporary objects, `i` and `j`, that occupy the same position in the offset of the iterables at hand (see line 6; the first item from the first iterable goes with the first item from the second iterable, the second item from the first iterable goes with the second item from the second iterable, and so on).

Snippet 4.37 — looping over two iterables in parallel with `zip`

```

1 # the lists
2 >>> LETTERS = ["x", "y", "z"]
3 >>> COLORS = ["blue", "green", "red"]
4
5 # create one-to-one matches of items and do something with them
6 >>> for i, j in zip(LETTERS, COLORS):
7 ...     print(i, " <-> ", j)
8 x <-> blue
9 y <-> green
10 z <-> red

```

3.11 Iterations and Comprehensions

Is there any other Python iterator besides `while` and `for` loops?

As we know from the previous section, `while` and `for` loops can handle most repetitive tasks programs need to perform. However, Python provides additional tools to make loops *easier to write/read* and *more efficient*. One of the most prominent tools is [list comprehension](#).

Why do I use list comprehensions?

To create a list containing the outcome of an action repeated over an iterable's items (see Snippet 4.38, line 14).

How do I create list comprehensions?

We include a Python statement containing a `for` clause among brackets (see Snippet 4.38, line 12).

Snippet 4.38 — for loop Vs. list comprehension

```

1 # the for loop way
2 # --+ create an empty list
3 >>> L = []
4 # --+ create a for loop appending the square of some items
5 >>> for i in range(3):
6 ...     L.append(i ** 2)
7 # --+ print the list
8 >>> print(L)
9 [0, 1, 4]
10
11 # the list comprehension way
12 >>> L = [i ** 2 for i in range(3)]
13 >>> print(L)
14 [0, 1, 4]
15

```

How do I implement a nested for loop in list comprehensions?

As Snippet 4.39 shows, a nested for loop becomes a one-liner in a list comprehension. The first for clause in line 7 would correspond to the outer for loop reported in Snippet 4.36, whereas the second for clause in line 7 would correspond to the inner for loop reported in Snippet 4.36.

Snippet 4.39 — nested for loop with list comprehensions

```

1 # the lists
2 >>> LETTERS = ["x", "y", "z"]
3 >>> COLORS = ["blue", "green", "red"]
4
5 # implementing a nested for loop with a list comprehension
6 >>> LETTER2COLOR = ["{} <-> {}".format(i, j) for i in LETTERS for j in COLORS]
7 ['x <-> blue',
8  'x <-> green',
9  'x <-> red',
10 'y <-> blue',
11 'y <-> green',
12 'y <-> red',
13 'z <-> blue',
14 'z <-> green',
15 'z <-> red']

```

Can I use the zip generator within a list comprehension?

Yes, we can. To do that, the for clause must consider two iterables simultaneously (see Snippet 4.40, line 6).

Snippet 4.40 — looping over two iterables in parallel with zip and list comprehension

```

1 # the lists
2 >>> LETTERS = ["x", "y", "z"]
3 >>> COLORS = ["blue", "green", "red"]
4
5 # implementing a nested for loop with a list comprehension
6 >>> LETTER2COLOR = [{"{} <-> {}".format(i, j) for i, j in zip(LETTERS, COLORS)]
7 ['x <-> blue', 'y <-> green', 'z <-> red']

```

Can I embed control flow in a list comprehension?

Yes, we can. To do that, the for clause must be preceded by an if statement and, at least, an else statement (see for example Snippet 4.41's line 22).

Snippet 4.41 — control flow in list comprehensions

```

1 # import the function log from math
2 >>> from math import log
3
4 # the object to manipulate
5 >>> L1 = [0, 1, 2]
6
7 # the for loop way
8 # --+ the empty list
9 >>> L2 = []
10 # --+ the for loop appending the log of some items
11 >>> for i in L1:
12 ...     if i > 0:
13 ...         L2.append(log(i))
14 ...     else:
15 ...         L2.append(log(i + 0.001))
16 # --+ print the list
17 >>> print(L2)
18 [-6.907755278982137, 0.0, 0.6931471805599453]
19
20 # the list comprehension way
21 # --+ the list comprehension is a one-liner!
22 >>> L2 = [log(i) if i > 0 else log(i + 0.001) for i in L1]
23 # --+ print the list
24 >>> print(L2)
25 [-6.907755278982137, 0.0, 0.6931471805599453]

```

Notes

⁷Lutz, Mark. *Learning Python: Powerful object-oriented programming*. O'Reilly Media, Inc., 2013.

⁸Floating numbers are stored in binaries with an assigned level of precision typically equivalent to 15 or 16 decimals.

⁹As per the documentation of the Python programming language, `math` cannot be used with complex numbers.

¹⁰The official Python documentation has an extensive section on operator precedence rules in the section dedicated to [syntax of expressions](#)

¹¹For the sake of simplicity, we assume the target file is located in the same directory as the Python script.

¹²By default, `w` truncates the file if it already exists

¹³If encoding is not specified, the encoding used is platform-dependent. Specifically, `locale.getpreferredencoding(False)` is called to get the current locale encoding. Character encoding assigns

numbers to graphical characters, especially the written characters of human language, allowing them to be stored, transmitted, and transformed using digital computers.

Chapter 4

Technical & Scientific Computation with NumPy

At the end of the chapter, you will be able to:

- Create efficient data structures called `ndarrays`
 - Populate `ndarrays` with simulated or real-world data
 - Manipulated `ndarrays` with a variety of scalable routines
 - Carry out exploratory data analysis with NumPy
 - Read and write `ndarrays` in many formats
-

4.1 Installing NumPy

Does NumPy come with the official Python installation file?

No, it does not. You need to install it separately using the package manager `pip`.

I am an Anaconda user: do I need to install NumPy ‘separately’?

No, you do not. NumPy is included in ‘base,’ the default environment of Anaconda. However, if you create a new environment, you need to install NumPy — and all the other modules you need — with the package manager `conda`.

How do I install NumPy?

The easiest way is using the command line. Anaconda users run `$ conda install numpy scipy`, whereas Python official release users run `$ pip install numpy scipy`. Anaconda users can also install the modules they need from within Anaconda-Navigator.

4.2 NumPy ndarray

What is a NumPy ndarray?

Can ndarrays contain objects of different type?

How do I create an ndarray?

Put simply, an `ndarray` is a data container, like dictionaries and lists.

No, they cannot. An `ndarray` must contain homogeneous items; that is, items of the same type.

As shown in Snippet 5.1, we pass an object to `numpy.array`. If the object we pass is a scalar, a 0-dimensional array containing object is returned (line 5). Passing a list to `numpy.array` produces a one-dimensional array (line 9); passing a list of lists produces a two-dimensional array (line 13); finally, passing a list of lists of lists produces a three-dimensional array (line 18).

Snippet 5.1 — creating an ndarray

```

1  # import numpy with the socially accepted alias 'np'
2  >>> import numpy as np
3
4  # a 0-D array
5  >>> np.array(0)
6  array(0)
7
8  # a 1-D array
9  >>> np.array([1, 2, 3, 4])
10 array([1, 2, 3, 4])
11
12 # a 2-D array
13 >>> np.array([[1, 2], [3, 4]])
14 array([[1, 2],
15        [3, 4]])
16
17 # a 3-D array
18 >>> np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
19 array([[[1, 2],
20        [3, 4]],
21        [[5, 6],
22        [7, 8]]])
23
24
```

What are the distinctive features of `ndarrays`?

`ndarrays` have been designed — and tuned over time — with flexibility and efficiency in mind. For example, `ndarrays` allows to carry out computations on arrays with a syntax similar to scalar values. As shown in Snippet 5.2, we can multiply an array by a scalar (line 10) and add two vectors (14). That is not possible if we use pure Python code. Multiplying a list by a scalar N replicates the ordered collection of items N times (see line 23). Adding two lists yields concatenation (see line 30). At the same time, `ndarrays` support the analysis of large volumes of data¹⁴.

Snippet 5.2 — NumPy allows to manipulate vectors with an expressive syntax

```

1 # import numpy with the socially accepted alias 'np'
2 >>> import numpy as np
3
4 # generate some random
5 >>> DATA = np.random.randn(3)
6 >>> print(DATA)
7 [-0.44144029 -0.44451097  0.31997294]
8
9 # can we multiply a list by a scalar with NumPy? Of course!
10 >>> print(DATA * 3)
11 [-4.4144029 , -4.44510974,  3.19972941]
12
13 # can we sum two arrays with NumPy? Of course!
14 >>> print(DATA + DATA)
15 [-0.88288058, -0.88902195,  0.63994588]
16
17 # let us try to replicate the previous tasks in pure Python?
18 # --+ get the DATA as a list
19 >>> DATA = list(DATA)
20 >>> print(DATA)
21 [-0.4414402896845323, -0.4445109735283278, 0.31997294069261617]
22 # --+ is the NumPy syntax of line 10 still valid if we use a list? Nope
23 >>> print(DATA * 3)
24 [
25 -0.4414402896845323, -0.4445109735283278, 0.31997294069261617,
26 -0.4414402896845323, -0.4445109735283278, 0.31997294069261617,
27 -0.4414402896845323, -0.4445109735283278, 0.31997294069261617
28 ]
29 # --+ is the NumPy syntax of line 14 still valid if we use a list? Nope
30 >>> print(DATA + DATA)
31 [
32 -0.4414402896845323, -0.4445109735283278, 0.31997294069261617,
33 -0.4414402896845323, -0.4445109735283278, 0.31997294069261617
34 ]

```

How do I check an `array`'s number of dimensions?

In Snippet 5.3, we saw NumPy infers an `ndarrays`'s number of dimensions from the data. Snippet 5.3 shows how to access `.ndim`, the `ndarrays`'s attribute concerning the number of dimensions. In the example, `DATA` has two dimensions (e.g., coordinates).

How do I check an `array`'s shape?

The lower section of Snippet 5.3 shows how to access `.shape`, the `ndarrays`'s attribute concerning the shape. In the example, each dimension of the `DATA` has size 2.

Snippet 5.3 — an array's number of dimensions and shape

```

1 # import numpy with the socially accepted alias 'np'
2 >>> import numpy as np
3
4 # the data
5 >>> DATA = np.array([[1, 2], [3, 4]])
6 >>> print(DATA)
7 [[1 2]
8  [3 4]]
9
10 # get the number of dimensions
11 >>> DATA.ndim
12 2
13
14 # get the shape
15 >>> DATA.shape
16 (2, 2)

```

What are the attributes of an `array`?

Table 4.1 illustrates the common use attributes of `ndarrays`.

TABLE 4.1
Common Use Attributes of NumPy `array`

Attribute	Synopsis
<code>DATA.flags</code>	Information about the memory layout of the array
<code>DATA.shape</code>	Tuple of array dimensions
<code>DATA.strides</code>	Tuple of bytes to step in each dimension when traversing an array
<code>DATA.ndim</code>	Number of array dimensions
<code>DATA.data</code>	Python buffer object pointing to the start of the array's data
<code>DATA.size</code>	Number of elements in the array
<code>DATA.itemsize</code>	Length of one array element in bytes
<code>DATA.nbytes</code>	Total bytes consumed by the elements of the array
<code>DATA.dtype</code>	Data-type of the array's elements

Notes: `DATA` is a fictionary object used to illustrate the usage of the `array` attributes.

I know that a NumPy array must contain homogenous data — but which object types are allowed?

How do I specify the data type of an array?

Table 4.2 reports the NumPy data types. Python beginners are not supposed to appreciate the distinctive attributes of each type. Instead, they may want to get a clear understanding of the high-level types, namely, floating points, complex, integer, boolean, string, or general Python objects. When working on sophisticated projects requiring more control over the storage types, it is highly suggested to get a thorough knowledge of the types in Table 4.2. It is worth noticing that `dtypes` are a source of NumPy's flexibility for interacting with data coming from other systems. In most cases, they map directly onto an underlying disk or memory representation, making it easy to read and write binary data streams to disk and connect to code written in a low-level language like C or Fortran. The numerical `dtypes` are named the same way: a type name, like `float` or `int`, followed by a number indicating the number of bits per element. A standard double-precision floating-point value takes up 8 bytes or 64 bits. Thus, this type is known in NumPy as `float64`.

As Snippet 5.4 shows, `dtype` is an optional argument of `ndarray`. It is also possible to change `dtype` using the method `.astype()`.

Snippet 5.4 — specifying and changing dtype

```

1  # import numpy with the socially accepted alias 'np'
2  >>> import numpy as np
3
4  # accept the default type
5  >>> A = np.array([1, 2, 3, 4, 5])
6
7  # check the type
8  >>> A.dtype
9  dtype('int64')
10
11 # specify the type
12 >>> A = np.array([1, 2, 3, 4, 5], dtype=np.int32)
13
14 # check the type
15 >>> A.dtype
16 dtype('int32')
17
18 # type change
19 # --+ NOTE np.bytes_, not np.string_: the np.string_ alias was removed in
20 #     NumPy 2.0
21 >>> S = np.array(['1.25', '-9.6', '42'], dtype=np.bytes_)
22 >>> S
23 array([b'1.25', b'-9.6', b'42'], dtype='<S4')
```

```
24 >>> S = S.astype(float)
25 >>> S.dtype
26 dtype('float64')
```

TABLE 4.2
NumPy Data Types

Type	Type Code	Synopsis
int8, uint8	i1, u1	Signed and unsigned 8-bit (1 byte) integer types
int16, uint16	i2, u2	Signed and unsigned 16-bit integer types
int32, uint32	i4, u4	Signed and unsigned 32-bit integer types
int64, uint64	i8, u8	Signed and unsigned 64-bit integer types
float16	f2	Half-precision floating point float32 f4 or f Standard single-precision floating-point; compatible with C float
float64	f8 or d	Standard double-precision floating-point; compatible with C double and Python
float object	float128 f16 or g	Extended-precision floating point
complex64, complex128, complex256	c8, c16, c32	Complex numbers represented by two 32, 64, or 128 floats, respectively
bool	?	Boolean type storing True and False values
object	O	Python object type; a value can be any Python object
string_	S	Fixed-length ASCII string type (1 byte per character); for example, to create a string dtype with length 10, use 'S10'
unicode_	U	Fixed-length Unicode type (number of bytes platform specific); same specification semantics as string_ (e.g., 'U10')

4.3 Array Creation Routines

Does NumPy offer recipes for creating arrays?

Creating arrays from shape or value

Yes, it does. NumPy has seven families of array-creating routines:

- From shape or value
- From existing data
- Creating record arrays (`np.rec`)
- Numerical ranges
- Building matrices
- The Matrix class

This family of routines creates arrays with a certain number of dimensions, shapes, values, and attributes. Snippet 5.5. shows how to create:

- an array with a certain shape and a constant scalar (see lines 5 — `.zeros`, 12 — `.ones`, 19 — `full`), and
- an array containing an identity matrix (see lines 26 — `.eye` — and 33 — `.identity`)

Snippet 5.5 — creating arrays from shape or value

```
1 # import numpy with the socially accepted alias 'np'
2 >>> import numpy as np
3
4 # create an array with zeros only
5 >>> np.zeros([4,4])
6 array([[0., 0., 0., 0.],
7        [0., 0., 0., 0.],
8        [0., 0., 0., 0.],
9        [0., 0., 0., 0.]])
10
11 # create an array with ones only
12 >>> np.ones((4,4))
13 array([[1., 1., 1., 1.],
14        [1., 1., 1., 1.],
15        [1., 1., 1., 1.],
16        [1., 1., 1., 1.]])
17
18 # create a full matrix with a given scalar
19 >>> np.full((4,4), -99)
20 array([[ -99,  -99,  -99,  -99],
21        [ -99,  -99,  -99,  -99],
22        [ -99,  -99,  -99,  -99],
23        [ -99,  -99,  -99,  -99]])
24
25 # create an identity array of a given shape with .eye
26 >>> np.eye(4, 3)
27 array([[1., 0., 0.],
28        [0., 1., 0.],
29        [0., 0., 1.],
30        [0., 0., 0.]])
31
32 # create an identity array with .identity
33 >>> np.identity(4)
34 array([[1., 0., 0., 0.],
35        [0., 1., 0., 0.],
36        [0., 0., 1., 0.],
37        [0., 0., 0., 1.]])
```

TABLE 4.3
Routines for Creating Arrays from Shape or Value

Routine	Synopsis
<code>np.empty(shape, dtype, order, like)</code>	Return a new array of given shape and type, without initializing entries
<code>np.empty_like(prototype[, dtype, order, subok, ...])</code>	Return a new array with the same shape and type as a given array
<code>np.eye(N, M, k, dtype, order, like)</code>	Return a 2-D array with ones on the diagonal and zeros elsewhere
<code>np.identity(n[, dtype, like])</code>	Return the identity array
<code>np.ones(shape, dtype, order, like)</code>	Return a new array of given shape and type, filled with ones
<code>np.ones_like(a[, dtype, order, subok, ...])</code>	Return an array of ones with the same shape and type as a given array
<code>np.zeros(shape, dtype, order, like)</code>	Return a new array of given shape and type, filled with zeros
<code>np.full(shape, fill_value[, dtype, order, like])</code>	Return a new array of given shape and type, filled with fill value
<code>full_like(a, fill_value[, dtype, order, ...])</code>	Return a full array with the same shape and type as a given array

Notes: the statements included in the ‘Routine’ column assume NumPy is loaded with the `np` alias.

Creating arrays from existing data

In Snippets 5.1 – 5.4, we saw how to use `array` for passing data to a NumPy array. Snippet 5.6 shows other routines to create NumPy arrays from existing data, including for example:

- `.fromfunction`, creating an array by executing a function over each coordinate (line 8)
- `.fromfile`, creating an array from data in a text or binary file (line 19)
- `.loadtxt`, loading data from a text file (line 40). The example represents a real-world data set containing both numeric and text information. The first argument we pass `.loadtxt` is a file object. To correctly parse the data, we also pass the following discretionary arguments to `.loadtxt`: i) `comments="#"` indicates that any lines in the file commencing with `#` must be considered a comment, not a piece of data; ii) `delimiter=","` indicates that two items separated by the character `,` belong to different fields (i.e., ‘columns’ to use a spreadsheet-alike vocabulary); iii) `textttquotechar='"'` indicates that strings are enclosed between double quotes¹⁵.

Snippet 5.6 — creating arrays from existing data

```

1 # import numpy with the socially accepted alias 'np'
2 >>> import numpy as np
3
4 # get data from a function
5 # --+ create a function
6 >>> my_function = lambda x, y: x - 0.5 * y ** 2
7 # --+ create an array from my_function for given coordinates
8 >>> np.fromfunction(my_function, (3, 3), dtype=float)
9 array([[ 0. , -0.5, -2. ],
10        [ 1. ,  0.5, -1. ],
11        [ 2. ,  1.5,  0. ]])
12
13 # get data from a binary file
14 # --+ create an array from a list of numbers
15 >>> D = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9])
16 # --+ save the raw data to a binary file
17 >>> D.tofile("data.bin")
18 # --+ read the data back
19 >>> np.fromfile("data.bin", dtype=int)
20 array([1, 2, 3, 4, 5, 6, 7, 8, 9])
21
22 # get data from a text file
23 # --+ create a string with the data and some qualitative comments on them
24 >>> S = ""

```

```

25 ... # Below are some demographic data about Michael J. Jordan (basketball
26 ... # player) from Wikipedia.
27 ... #
28 ... # Data labels are:
29 ... #
30 ... # NAME, BORN, NBA CHAMPIONSHIPS, AVERAGE POINT PER GAME
31 ... "Jordan, Michael Jeffrey", "17-02-1963", 6, 30.1
32 ... """
33 # --+ write the data to a file. The `with' block closes the file for you --
34 #     there is no need to call .close() afterwards
35 >>> with open("my_data", "w") as pipe:
36 ...     pipe.write(S)
37 # --+ read the data and assign them to a NumPy array. The `with' block
38 #     still hands .loadtxt a FILE OBJECT, and closes it afterwards
39 >>> with open("my_data", "r") as pipe:
40 ...     A = np.loadtxt(
41 ...         pipe,
42 ...         dtype={
43 ...             "names": (
44 ...                 "NAME",
45 ...                 "BORN",
46 ...                 "NBA CHAMPIONSHIPS",
47 ...                 "AVERAGE POINT PER GAME"
48 ...             ),
49 ...             "formats": ("S30", "S10", "i1", "f2"),
50 ...         },
51 ...         comments="#",
52 ...         delimiter=",",
53 ...         quotechar='"'
54 ...     )
55 >>> A
56 array((b'Jordan, Michael Jeffrey', b'17-02-1963', 6, 30.1),
57        dtype=[('NAME', 'S30'), ('BORN', 'S10'), ('NBA CHAMPIONSHIPS', 'i1'), ('AVERAGE POINT

```

TABLE 4.4
Routines for Creating Arrays from Existing Data

Routine	Synopsis
<code>np.array(object[,dtype, copy, subok, ...])</code>	Create an array
<code>np.asarray(a[, dtype, order, like])</code>	Convert the input to an array
<code>np.asanyarray(a[, dtype, order, like])</code>	Convert the input to an ndarray, but pass ndarray subclasses through
<code>np.ascontiguousarray(a[, dtype, like])</code>	Return a contiguous array (ndim ≥ 1) in memory (C order)
<code>np.asmatrix(data[, dtype])</code>	Interpret the input as a matrix
<code>np.copy(a[, order, subok])</code>	Return an array copy of the given object
<code>np.frombuffer(buffer[, dtype, count, offset, like])</code>	Interpret a buffer as a 1-dimensional array
<code>np.fromfile(file[, dtype, count, sep, offset, like])</code>	Construct an array from data in a text or binary file
<code>np.fromfunction(function, shape, *[, dtype, like])</code>	Construct an array by executing a function over each coordinate
<code>np.fromiter(iter, dtype[, count, like])</code>	Create a new 1-dimensional array from an iterable object
<code>np.fromstring(string[, dtype, count, like])</code>	A new 1-D array initialized from text data in a string
<code>np.loadtxt(fname[, dtype, comments, delimiter, ...])</code>	Load data from a text file

Notes: the statements included in the ‘Routine’ column assume NumPy is loaded with the `np` alias.

Record arrays

NumPy arrays do not contain any information about the attributes of the data. For example, a NumPy array cannot accommodate any meta-data, such as the fields' names in the data. Here is where `.rec` kick in (see Table 4.5). For example, `.core.records.array` allows to flexibly specify a field's type and name (see Snippet 5.7, line 11). Once a 'recarray' is created, it is possible to fetch its data by field name (see Snippet 5.7, line 14).

Snippet 5.7 — creating record arrays

```

1 # import the record-array constructor with an alias that does not conflict
2 # with 'standard' NumPy arrays. NOTE np.rec: the numpy.core path used to
3 # work but became private (numpy._core) in NumPy 2.0, and importing it now
4 # raises a DeprecationWarning
5 >>> import numpy as np
6 >>> from numpy.rec import array as recarray
7
8 # the data
9 >>> LOCS = [("51.5072° N", "0.1276° W"), ("35.6762° N", "139.6503° E")]
10
11 # create a recarray
12 >>> D = recarray(LOCS, formats=["U12", "U12"], names=["Latitude", "Longitude"])
13
14 # fetch the data by field name
15 >>> D.Latitude
16 array(['51.5072° N', '35.6762° N'], dtype='<U12')
```

TABLE 4.5
Routines for Creating Record Arrays

Routine	Synopsis
<code>np.core.records.array(obj[, dtype, shape, ...])</code>	Construct a record array from a wide variety of objects
<code>np.core.records.fromarrays(arrayList[, dtype, ...])</code>	Create a record array from a (flat) list of arrays
<code>np.core.records.fromrecords(records[, dtype, ...])</code>	Create a record array from a list of records in text form
<code>np.core.records.fromstring(datastring[, dtype, ...])</code>	Create a record array from binary data
<code>np.core.records.fromfile(fd[, dtype, shape, ...])</code>	Create an array from binary file data

Notes: the statements included in the ‘Routine’ column assume NumPy is loaded with the `np` alias.

Creating numerical ranges

One may want to create a numerical range for different reasons, including running functional analysis or computer simulation. NumPy has a bunch of array-creating routines for numerical ranges (see Table 4.6), some of which are quite popular in technical and scientific computation as well as data science. For example, `np.arange` and `np.linspace` frequently appear in Python programs when it comes to create evenly spaced values in a certain interval and evenly spaced samples respectively (see Snippet 5.8, lines XX and XX). Another popular routine is `.meshgrid`, returning coordinate matrices from coordinate vectors.

Snippet 5.8 — creating numerical ranges

```

1 # import numpy with the socially accepted alias 'np'
2 >>> import numpy as np
3
4 # import a data viz module -- the last section of this snippet plots a
5 # contour of the coordinate matrices
6 >>> import matplotlib.pyplot as plt
7
8 # two ranges of evenly spaced values
9 # --+ evenly spaced values between 0 and 10
10 >>> np.arange(0, 10, 1)
11 array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
12 # --+ ... equivalent to
13 >>> np.arange(10)
14 array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
15 # --+ evenly spaced values between 0 and 10 divided by a 2-unit step
16 >>> np.arange(0, 10, 2)
17 array([0, 2, 4, 6, 8])
18
19 # 50 evenly spaced values between 0 and 1
20 >>> np.linspace(0, 1, 10)
21 array([0.          , 0.11111111, 0.22222222, 0.33333333, 0.44444444,
22        0.55555556, 0.66666667, 0.77777778, 0.88888889, 1.          ])
23
24 # get coordinate matrices from coordinate vectors
25 # --+ the 'x-' and 'y-axis' vectors
26 >>> X = np.linspace(0, 1, 10)
27 >>> Y = np.linspace(0, 1, 5)
28 # --+ get 'x-axis' ('y-axis') coordinates for any value of vector Y (X)
29 >>> XX, YY = np.meshgrid(X, Y)
30 >>> XX
31 array([[0.          , 0.11111111, 0.22222222, 0.33333333, 0.44444444,
32        0.55555556, 0.66666667, 0.77777778, 0.88888889, 1.          ],
33       [0.          , 0.11111111, 0.22222222, 0.33333333, 0.44444444,
34        0.55555556, 0.66666667, 0.77777778, 0.88888889, 1.          ],
35       [0.          , 0.11111111, 0.22222222, 0.33333333, 0.44444444,
36        0.55555556, 0.66666667, 0.77777778, 0.88888889, 1.          ],
37       [0.          , 0.11111111, 0.22222222, 0.33333333, 0.44444444,
```



```

38     0.55555556, 0.66666667, 0.77777778, 0.88888889, 1.        ],
39     [0.        , 0.11111111, 0.22222222, 0.33333333, 0.44444444,
40     0.55555556, 0.66666667, 0.77777778, 0.88888889, 1.        ]])
41 >>> YY
42 array([[0.   , 0.   , 0.   , 0.   , 0.   , 0.   , 0.   , 0.   , 0.   , 0.   ],
43        [0.25, 0.25, 0.25, 0.25, 0.25, 0.25, 0.25, 0.25, 0.25, 0.25],
44        [0.5  , 0.5  , 0.5  , 0.5  , 0.5  , 0.5  , 0.5  , 0.5  , 0.5  , 0.5  ],
45        [0.75, 0.75, 0.75, 0.75, 0.75, 0.75, 0.75, 0.75, 0.75, 0.75],
46        [1.   , 1.   , 1.   , 1.   , 1.   , 1.   , 1.   , 1.   , 1.   , 1.   ]])
47 # --+ create a matrix from X and Y
48 >>> ZZ = np.sqrt(XX**2 + YY**2)
49 # --+ check the dimensions of the newly created objects
50 >>> print(XX.shape, YY.shape, ZZ.shape)
51 >>> ((101, 101), (101, 101), (101, 101))
52 # --+ make a contour plot showing the associations among X, Y, and Z
53 # (see Figure 5.1)
54 >>> fig = plt.figure()
55 >>> ax = fig.add_subplot(111)
56 >>> ax = plt.contourf(X, Y, ZZ)
57 >>> plt.axis('scaled')
58 >>> plt.colorbar()
59 >>> plt.show()

```

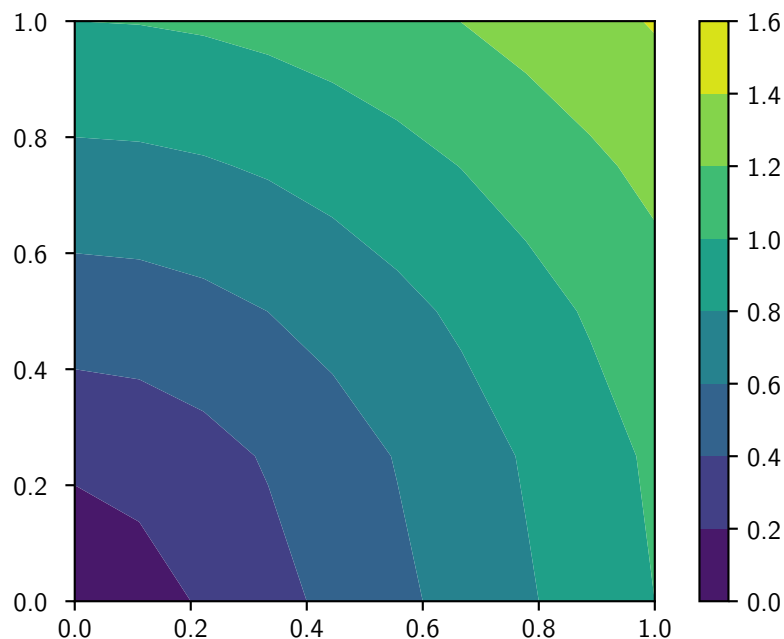


Figure 4.1: A contour plot showing the associations among X, Y, and Z
Notes. — the data behind this plot come from Snippet 5.8, lines 22 - 44.

TABLE 4.6
Routines for Numerical Ranges

Routine	Synopsis
<code>np.arange([start,] stop[, step[, L[, dtype, like]])</code>	Return evenly spaced values within a given interval
<code>np.linspace(start, stop[, num, endpoint, ...])</code>	Return evenly spaced numbers over a specified interval
<code>np.logspace(start, stop[, num, endpoint, base, ...])</code>	Return numbers spaced evenly on a log scale
<code>np.geomspace(start, stop[, num, endpoint, ...])</code>	Return numbers spaced evenly on a log scale (a geometric progression)
<code>np.meshgrid(*xi[, copy, sparse, indexing])</code>	Return coordinate matrices from coordinate vectors
<code>np.mgrid</code>	<code>nd_grid</code> instance which returns a dense multi-dimensional “meshgrid”
<code>np.ogrid</code>	<code>nd_grid</code> instance which returns an open multi-dimensional “meshgrid”

Notes: the statements included in the ‘Routine’ column assume NumPy is loaded with the `np` alias.

Building matrices

NumPy has routines to create arrays from an existing matrix as well as build matrices with certain properties (see Table 4.7) As Snippet 5.9 shows, `.diag` creates an array by fetching a matrix's diagonal (see line 10), while `.tri` creates a triangular matrix (see line 17).

Snippet 5.9 — creating arrays from existing data

```

1  # import numpy with the socially accepted alias 'np'
2  >>> import numpy as np
3
4  # create an array by fetching a matrix diagonal
5  # --+ the matrix
6  >>> M = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
7  >>> M.shape
8  (3, 3)
9  # --+ the new array
10 >>> A = np.diag(M)
11 >>> print(A)
12 [1 5 9]
13 >>> A.shape
14 (3,)
15
16 # create a triangular matrix
17 >>> np.tri(10, 10)
18 array([[1., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
19        [1., 1., 0., 0., 0., 0., 0., 0., 0., 0.],
20        [1., 1., 1., 0., 0., 0., 0., 0., 0., 0.],
21        [1., 1., 1., 1., 0., 0., 0., 0., 0., 0.],
22        [1., 1., 1., 1., 1., 0., 0., 0., 0., 0.],
23        [1., 1., 1., 1., 1., 1., 0., 0., 0., 0.],
24        [1., 1., 1., 1., 1., 1., 1., 0., 0., 0.],
25        [1., 1., 1., 1., 1., 1., 1., 1., 0., 0.],
26        [1., 1., 1., 1., 1., 1., 1., 1., 1., 0.],
27        [1., 1., 1., 1., 1., 1., 1., 1., 1., 1.]])

```

TABLE 4.7
Routines for Building Matrices

Routine	Synopsis
<code>np.diag(v[, k])</code>	Extract a diagonal or construct a diagonal array
<code>np.diagflat(v[, k])</code>	Create a two-dimensional array with the flattened input as a diagonal
<code>np.tri(N[, M, k, dtype, like])</code>	An array with ones at and below the given diagonal and zeros elsewhere
<code>np.tril(m[, k])</code>	Lower triangle of an array
<code>np.triu(m[, k])</code>	Upper triangle of an array
<code>np.vander(x[, N, increasing])</code>	Generate a Vandermonde matrix

Notes: the statements included in the ‘Routine’ column assume NumPy is loaded with the `np` alias.

The Matrix Class

What is the advantage of using a matrix class object?

In Snippet 5.9, line 19, I claim to create a matrix. The outcome displayed in line 20 is consistent with the concept of ‘matrix’ we have been taught in a typical linear algebra class: what a human being sees is a set of numbers arranged in rows and columns. However, in NumPy terms, the object displayed in line 20 is an array with two dimensions. If we want to create a NumPy Matrix Class object¹⁶, we have to call `np.matrix`, a subclass of `np.ndarray`. Table 4.8 illustrates the two matrix-creating routines of NumPy.

As we will see later on in this chapter, the advantage of using a matrix class object is that we can use the matrix class object to perform matrix operations with a simple and intuitive syntax. For example, we can use the matrix class object to perform matrix manipulation multiplication, addition, and subtraction. In the lower section of Snippet 5.10, we use `.linalg.inv` to compute the inverse of a matrix.

Snippet 5.10 — creating a NumPy Matrix Class object

```

1 # import numpy with the socially accepted alias 'np'
2 >>> import numpy as np
3
4 # create a matrix class object. NOTE np.matrix now raises a
5 # PendingDeprecationWarning on construction: NumPy's own advice is to use a
6 # 2-D array and the @ operator for matrix multiplication instead
7 # --+ the arrays to populate a matrix class object
8 >>> A = np.array([0, 1, 2])
9 >>> B = np.array([-99, 203, 1009])
10 >>> C = np.array([-1000, -1001, -1002])
11 # --+ the matrix class object
12 >>> M = np.matrix([A, B, C])
13 matrix([[ 0, 1, 2],
14         [-99, 203, 1009],
15         [-1000, -1001, -1002]])
16
17 # get the inverse of M
18 >>> np.linalg.inv(M)
19 matrix([[-1.60040278e+00, 1.98412698e-03, -1.19642857e-03],
20         [ 2.19880556e+00, -3.96825397e-03, 3.92857143e-04],
21         [-5.99402778e-01, 1.98412698e-03, -1.96428571e-04]])

```

TABLE 4.8
Routines for the Matrix Class

Routine	Synopsis
<code>np.mat(data[, dtype])</code>	Interpret the input as a matrix
<code>bmat(obj[, ldict, gdict])</code>	Build a matrix object from a string, nested sequence, or array

Notes: the statements included in the ‘Routine’ column assume NumPy is loaded with the `np` alias.

4.4 Array Manipulation Routines

I have an array. What can I do with it?

NumPy offers many array manipulation routines (see Table A.7), which can be grouped around the following families:

- Basic operations
- Changing array shape
- Transpose-like operations
- Changing number of dimensions
- Changing kind of array
- Joining arrays
- Splitting arrays
- Tiling arrays
- Adding and removing elements
- Rearranging elements

Presenting every NumPy array manipulation routine would require writing a dedicated book (and certainly falls beyond the remit of an introductory module on Python). That said, let me whet the reader's appetite by illustrating a sample of miscellaneous routines (see Snippet 5.11). Here is the sequence of tasks we accomplish: first, we create an array of shape (6,) (see line 5); then, we change the shape of the array (line 10) and transpose its rows and columns (line 16); in the lower section of the snippet we join two arrays using several routines: `.concatenate` joins two arrays along the desired axis (the default axis is the first one, meaning the second arrays are concatenated row-wise); `.vstack` requires to pass a tuple of arrays to stack row-wise; `.hstack` requires to pass a tuple of arrays to stack column-wise. It is worth noticing that the rules for vector/matrix manipulation we learned at school apply to the routines that join arrays. In other words, two NumPy arrays can be joined if and only have the same length on the joining axis. For example, it is possible to stack vertically two arrays with the shape (3, 3) and (2, 3) because they have the same number of columns.

Snippet 5.11 — miscellaneous array-manipulating routines

```
1 # import numpy with the socially accepted alias 'np'
2 >>> import numpy as np
3
4 # the array
5 >>> A = np.arange(6)
6 >>> A
7 array([0, 1, 2, 3, 4, 5])
8
9 # reshape the array into a 2x3 array
10 >>> A = A.reshape(2, 3)
11 >>> A
12 array([[0, 1, 2],
13        [3, 4, 5]])
14
15 # transpose the array
16 >>> A.T
17 array([[0, 3],
18        [1, 4],
19        [2, 5]])
20
21 # get back to a 'flat' array with shape (6,)
22 >>> A.ravel()
23 array([0, 1, 2, 3, 4, 5])
24
25 # reshape the array into a 3x2 array
26 >>> A.reshape(3, 2)
27 array([[0, 1],
28        [2, 3],
29        [4, 5]])
30
31 # join two arrays --- note the dimensions for the concatenation must match
32 >>> np.concatenate((A.reshape(3, 2), A.T))
33 array([[0, 1],
34        [2, 3],
35        [4, 5],
36        [0, 3],
37        [1, 4],
38        [2, 5]])
39
40 # stack two arrays vertically (ROW-WISE)
41 >>> np.vstack((A, np.array([6, 7, 8])))
42
43 # stack two arrays horizontally (COLUMN-WISE)
44 >>> np.hstack((A, np.array([6, 7]).reshape(2, 1)))
45 array([[0, 1, 2, 6],
46        [3, 4, 5, 7]])
47
```


4.5 Universal Functions in NumPy

What is a universal function?

A universal function, or **ufunc**, is a function that performs element-wise operations on **ndarrays**. You can think of them as fast vectorized wrappers for simple functions that take one or more scalar values and produce one or more scalar results¹⁷.

What is the rationale for using **ufuncs**?

Per the previous point, using a **ufunc** offer substantial performance advantages *vis a' vis* non-vectorized code — i.e., code using built-in Python iterators.

What are the **ufunc** options available in NumPy?

There are circa sixty universal functions implemented in NumPy. For the sake of convenience, the full list of **ufunc** options is reported in Tables A.3 - A.6. The following sections of the current chapter will illustrate how to use some of the popular central **ufuncs** in NumPy.

4.6 Mathematical Functions

What are the mathematical functions available in NumPy?

There are many **mathematical functions available in NumPy**, some of which are implemented as universal functions (for a complete list of universal functions, see Tables A.3). The available routines can be grouped into:

- Trigonometric functions
- Hyperbolic functions
- Rounding functions
- Sums, products, and differences
- Exponential and logarithmic functions
- Extrema finding

What are the mathematical functions available in NumPy?

Similar to the case of array-manipulating routines, a detailed discussion of every NumPy mathematical function exceeds the scope of these notes. However, every function can be accessed using the same procedure; we need a set of values to pass to the function's argument- that is it! Snippet 5.12 shows the procedure to use a NumPy mathematical functions by means of two trigonometric functions, namely, `.sin` and `.cos`. In line 9, we create a range of values to assign to the variable `X`; then, in line 11, we create two further arrays assigned to the outcome of `.sin` and `.cos` respectively; finally, we use the Matplotlib module to visualize the two functions (if you do not get the Matplotlib code logic, do not worry at all — you will familiarize yourself with it in the Fall Term module 'SMM635, Data Visualization.')

Snippet 5.12 — NumPy trigonometric functions in action

```

1 # import numpy with the socially accepted alias 'np'
2 >>> import numpy as np
3
4 # import a data viz module
5 >>> import matplotlib.pyplot as plt
6
7 # trigonometric functions
8 # --+ x-values
9 >>> X = np.arange(0, 2 * np.pi, 0.1)
10 # --+ y-values
11 >>> SI, CS = np.sin(X), np.cos(X)
12
13 # plot the functions
14 # --+ plot SI
15 >>> fig = plt.figure(figsize=(2.5, 2.5))
16 >>> ax = fig.add_subplot(111)
17 >>> ax.axhline(y=0, color="k", linewidth=0.5)
18 <matplotlib.lines.Line2D object at 0x10c7338c0>
19 >>> ax.set_xticks([0, 0.5 * np.pi, np.pi, 1.5 * np.pi, 2 * np.pi])
20 [<matplotlib.axis.XTick object at 0x10c733620>, <matplotlib.axis.XTick object at 0x10c65bed0>]
21 >>> ax.set_xticklabels(
22 ...     ["0", r"$\frac{1}{2} \pi$", r"$\pi$", r"$\frac{3}{2} \pi$", r"$2 \pi$"]
23 ... )
24 [Text(0.0, 0, '0'), Text(1.5707963267948966, 0, '$\frac{1}{2} \pi$'), Text(3.141592653589793, 0, '$\pi$'),
25 >>> plt.xlabel("$X$")
26 Text(0.5, 0, '$X$')
27 >>> plt.ylabel("$sine(X)$")
28 Text(0, 0.5, '$sine(X)$')
29 >>> ax.grid(True)
30 >>> ax.plot(X, SI, color="Blue")
31 [<matplotlib.lines.Line2D object at 0x10c7a1e80>]
32 >>> plt.title("A")
33 Text(0.5, 1.0, 'A')

```

```

34 >>> plt.show()
35 # --+ plot CS
36 >>> fig = plt.figure(figsize=(2.5, 2.5))
37 >>> ax = fig.add_subplot(111)
38 >>> ax.axhline(y=0, color="k", linewidth=0.5)
39 <matplotlib.lines.Line2D object at 0x10c7a2cf0>
40 >>> ax.set_xticks([0, 0.5 * np.pi, np.pi, 1.5 * np.pi, 2 * np.pi])
41 [<matplotlib.axis.XTick object at 0x10c7d2710>, <matplotlib.axis.XTick object at 0x10c7d1e5
42 >>> ax.set_xticklabels(
43 ...     ["0", r"$\frac{1}{2} \pi$", r"$\pi$", r"$\frac{3}{2} \pi$", r"$2 \pi$"]
44 ... )
45 [Text(0.0, 0, '0'), Text(1.5707963267948966, 0, '$\frac{1}{2} \pi$'), Text(3.141592653589
46 >>> plt.xlabel("$X$")
47 Text(0.5, 0, '$X$')
48 >>> plt.ylabel("$\cosine(X)$")
49 Text(0, 0.5, '$\cosine(X)$')
50 >>> ax.grid(True)
51 >>> ax.plot(X, CS, color="Blue")
52 [<matplotlib.lines.Line2D object at 0x10c9181a0>]
53 >>> plt.title("B")
54 Text(0.5, 1.0, 'B')
55 >>> plt.show()

```

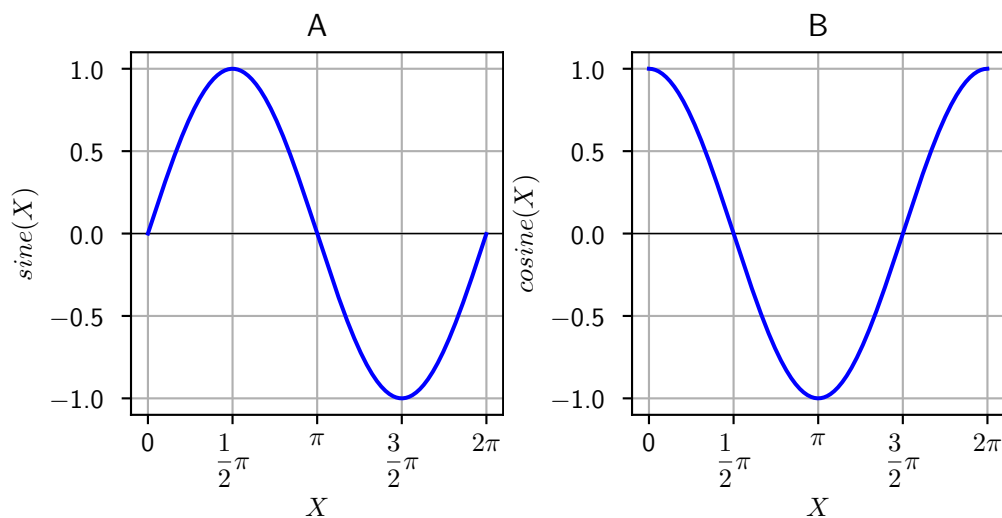


Figure 4.2: Visual representation of NumPy `.sin` and `.cos` functions as per Snippet 5.12.

4.7 Statistics

What are the statistical functions available in NumPy?

NumPy offers essential statistical functions sufficient to implement an Exploratory Data Analysis/descriptive statistics. Specifically, there are four families of statistical routines (see Tables 4.9, 4.10, 4.11, and 4.12):

- Order statistics (e.g., quantiles)
- Average and variances
- Correlations
- Histograms

Can I run multivariate statistical analysis with NumPy

Short answer: no. There are dedicated Python modules to run multivariate analyses, though. For example [linearmodels](#) and [statsmodels](#) are two popular modules to carry out econometric models in Python; [scikit-learn](#) is the acclaimed module for machine learning in Python.

TABLE 4.9
NumPy Statistical Routines: Order Statistics

Routine	Synopsis
<code>np.ptp(a[, axis, out, keepdims])</code>	Range of values (maximum-minimum) along an axis
<code>np.percentile(a, q[, axis, out, ...])</code>	Compute the q-th percentile of the data along the specified axis
<code>np.nanpercentile(a, q[, axis, out, ...])</code>	Compute the qth percentile of the data along the specified axis, while ignoring nan values
<code>np.quantile(a, q[, axis, out, overwrite_input, ...])</code>	Compute the q-th quantile of the data along the specified axis
<code>np.nanquantile(a, q[, axis, out, ...])</code>	Compute the qth quantile of the data along the specified axis, while ignoring nan values

Notes: the statements included in the ‘Routine’ column assume NumPy is loaded with the `np` alias.

TABLE 4.10
numpy Statistical Routines: Average and Variances

Routine	Synopsis
<code>np.median(a[, axis, out, overwrite_input, keepdims])</code>	Compute the median along the specified axis
<code>np.average(a[, axis, weights, returned, keepdims])</code>	Compute the weighted average along the specified axis
<code>np.mean(a[, axis, dtype, out, keepdims, where])</code>	Compute the arithmetic mean along the specified axis
<code>np.std(a[, axis, dtype, out, ddof, keepdims, where])</code>	Compute the standard deviation along the specified axis
<code>np.var(a[, axis, dtype, out, ddof, keepdims, where])</code>	Compute the variance along the specified axis
<code>np.nanmedian(a[, axis, out, overwrite_input, ..])</code>	Compute the median along the specified axis, while ignoring NaNs
<code>np.nanmean(a[, axis, dtype, out, keepdims, where])</code>	Compute the arithmetic mean along the specified axis, ignoring NaNs
<code>np.nanstd(a[, axis, dtype, out, ddof, ..])</code>	Compute the standard deviation along the specified axis, while ignoring NaNs
<code>np.nanvar(a[, axis, dtype, out, ddof, ..])</code>	Compute the variance along the specified axis, while ignoring NaNs

Notes: the statements included in the ‘Routine’ column assume NumPy is loaded with the `np` alias.

TABLE 4.11
NumPy Statistical Routines: Correlating

Routine	Synopsis
<code>np.corrcoef(x[, y, rowvar, bias, ddof, dtype])</code>	Return Pearson product-moment correlation coefficients
<code>np.correlate(a, v[, mode])</code>	Cross-correlation of two 1-dimensional sequences
<code>np.cov(m[, y, rowvar, bias, ddof, fweights, ...])</code>	Estimate a covariance matrix, given data and weights

Notes: the statements included in the ‘Routine’ column assume NumPy is loaded with the `np` alias.

TABLE 4.12
NumPy Statistical Routines: Histograms

Routine	Synopsis
<code>np.histogram(a[, bins, range, normed, weights, ...])</code>	Compute the histogram of a dataset
<code>np.histogram2d(x, y[, bins, range, normed, ...])</code>	Compute the bi-dimensional histogram of two data samples
<code>np.histogramdd(sample[, bins, range, normed, ...])</code>	Compute the multidimensional histogram of some data
<code>np.bincount(x, /[, weights, minlength])</code>	Count the number of occurrences of each value in an array of non-negative ints
<code>np.histogram_bin_edges(a[, bins, range, weights])</code>	Function to calculate only the edges of the bins used by the histogram function
<code>np.digitize(x, bins[, right])</code>	Return the indices of the bins to which each value in the input array belongs

Notes: the statements included in the ‘Routine’ column assume NumPy is loaded with the `np` alias.

So, how do I produce a set of summary stats in NumPy?

Snippet 5.13 shows how to create a typical set of summary stats, including an array's mean, standard deviation, minimum, maximum, and some percentiles of interest. Mainly, the snippet has three steps. First, we create an array (line 12). Second, we assign a couple of variables to NumPy statistical functions such as `.mean`, `.std`, `.min`, `np.max`, and `.percentile` (see lines 15-21). It is worth noting that `min` and `max` are keywords reserved for Python built-in functions. To avoid any name conflict and potential sources of confusion, in lines 17 and 18, we use the names `min_` and `max_`. Finally, we create a table displaying the variables created in the previous step (see lines 25, 29, and 33).

Snippet 5.13 — a set of summary statistics

```

1 # import numpy with the socially accepted alias 'np'
2 >>> import numpy as np
3
4 # import a module for arranging numbers in a tabular format. NOTE the form
5 # of the import: `import tabulate` brings in the MODULE, and calling a
6 # module raises TypeError -- it is the FUNCTION inside it that we want.
7 # tabulate is not part of the course environment; install it with
8 # `conda install tabulate` (or pip) if you want to run this snippet
9 >>> from tabulate import tabulate
10
11 # the array
12 >>> X = np.array([0, 0, -3, 12, 7, 2, -4, 6, 9, -1, 5, 3, -1, 3, 10, 9])
13
14 # get the descriptive stats
15 >>> mean = np.mean(X)
16 >>> std = np.std(X)
17 >>> min_ = np.min(X)
18 >>> max_ = np.max(X)
19 >>> pp25 = np.percentile(X, 25)
20 >>> pp50 = np.percentile(X, 50)
21 >>> pp75 = np.percentile(X, 75)
22
23 # arrange the stats in a tabular format
24 # --+ create the table header
25 >>> headers = [
26 ...     "Mean", "St. Dev.", "Min", "Max", "25th pp", "50th pp", "75th pp"
27 ... ]
28 # --+ round the numbers to three decimal places and get a string
29 >>> stats = [
30 ...     str(np.round(i, 3)) for i in [mean, std, min_, max_, pp25, pp50, pp75]
31 ... ]
32 # --+ print the table
33 >>> print(tabulate([stats], headers=headers, tablefmt="grid"))
34 +-----+-----+-----+-----+-----+-----+-----+
35 | Mean | St. Dev. | Min | Max | 25th pp | 50th pp | 75th pp |
36 +=====+=====+=====+=====+=====+=====+=====+

```

```

37 | 3.562 | 4.756 | -4 | 12 | -0.25 | 3 | 7.5 |
38 +-----+-----+-----+-----+-----+-----+-----+

```

Can I calculate Pearson's correlation coefficients in NumPy?

Yes, you can. Snippet 5.14 shows how to do so by using `.corrcoef`, one of the functions included in Table 4.11.

Snippet 5.14 — a matrix with Pearson's correlation coefficients

```

1 # import numpy with the socially accepted alias 'np'
2 >>> import numpy as np
3
4 # the arrays
5 >>> X = np.array([0, 0, -3, 12, 7, 2, -4, 6, 9, -1, 5, 3, -1, 3, 10, 9])
6 >>> Y = np.array([12, 12, 4, 3, 9, 2, -6, 15, 0, -12, 15, -3, -1, 0, 0, 1])
7
8 # get Pearson's correlation coefficients
9 >>> np.corrcoef(X, Y)
10 array([[1.          , 0.19763628],
11        [0.19763628, 1.          ]])

```

4.8 Linear Algebra

Is NumPy a good choice for linear algebra?

Yes, it is. NumPy offers a rich set of routines comparable to Matalab's one.¹⁸ Currently, there are five families of routines concerning the field of linear algebra at large:

- products
- decomposition
- eigenvalues
- norms
- equations and inversions

For an overview of numPy's linear algebra routines, see Tables 4.13, 4.14, 4.15, 4.16, and 4.17.

TABLE 4.13
NumPy Linear Algebra Routines: Matrix and Vector Products

Routine	Synopsis
<code>np.dot(a, b[, out])</code>	Dot product of two arrays
<code>np.linalg.multi_dot(arrays, *[, out])</code>	Compute the dot product of two or more arrays in a single function call, while automatically selecting the fastest evaluation order
<code>np.vdot(a, b, /)</code>	Return the dot product of two vectors
<code>np.inner(a, b, /)</code>	Inner product of two arrays
<code>np.outer(a, b[, out])</code>	Compute the outer product of two vectors
<code>np.matmul(x1, x2, /[, out, casting, order, ...])</code>	Matrix product of two arrays
<code>np.tensordot(a, b[, axes])</code>	Compute tensor dot product along specified axes
<code>np.einsum(subscripts, *operands[, out, dtype, ...])</code>	Evaluates the Einstein summation convention on the operands
<code>np.einsum_path(subscripts, *operands[, optimize])</code>	Evaluates the lowest cost contraction order for an einsum expression by considering the creation of intermediate arrays
<code>np.linalg.matrix_power(a, n)</code>	Raise a square matrix to the (integer) to the power n
<code>np.kron(a, b)</code>	Kronecker product of two arrays

Notes: the statements included in the ‘Routine’ column assume NumPy is loaded with the `np` alias.

TABLE 4.14
NumPy Linear Algebra Routines: Decompositions

Routine	Synopsis
<code>np.linalg.cholesky(a)</code>	Cholesky decomposition
<code>np.linalg.qr(a[, mode])</code>	Compute the qr factorization of a matrix
<code>np.linalg.svd(a[, full_matrices, compute_uv, ...])</code>	Singular Value Decomposition

Notes: the statements included in the ‘Routine’ column assume NumPy is loaded with the `np` alias.

TABLE 4.15
NumPy Linear Algebra Routines: Matrix Eigenvalues

Routine	Synopsis
<code>np.linalg.eig(a)</code>	Compute the eigenvalues and right eigenvectors of a square array
<code>np.linalg.eigh(a[, uplo])</code>	Return the eigenvalues and eigenvectors of a complex Hermitian (conjugate symmetric) or a real symmetric matrix
<code>np.linalg.eigvals(a)</code>	Compute the eigenvalues of a general matrix
<code>np.linalg.eigvalsh(a[, uplo])</code>	Compute the eigenvalues of a complex Hermitian or real symmetric matrix

Notes: the statements included in the ‘Routine’ column assume NumPy is loaded with the `np` alias.

TABLE 4.16
NumPy Linear Algebra Routines: Norms and Other Number

Routine	Synopsis
<code>np.linalg.norm(x[, ord, axis, keepdims])</code>	Matrix or vector norm
<code>np.linalg.cond(x[, p])</code>	Compute the condition number of a matrix
<code>np.linalg.det(a)</code>	Compute the determinant of an array
<code>np.linalg.matrix_rank(A[, tol, hermitian])</code>	Return matrix rank of array using SVD method
<code>np.linalg.slogdet(a)</code>	Compute the sign and (natural) logarithm of the determinant of an array
<code>np.trace(a[, offset, axis1, axis2, dtype, out])</code>	Return the sum along diagonals of the array

Notes: the statements included in the ‘Routine’ column assume NumPy is loaded with the `np` alias.

TABLE 4.17
NumPy Linear Algebra Routines: Solving Equations and Inverting Matrices

Routine	Synopsis
<code>np.linalg.solve(a, b)</code>	Solve a linear matrix equation, or system of linear scalar equations
<code>np.linalg.tensorsolve(a, b[, axes])</code>	Solve the tensor equation $a x = b$ for x
<code>np.linalg.lstsq(a, b[, rcond])</code>	Return the least-squares solution to a linear matrix equation
<code>np.linalg.inv(a)</code>	Compute the (multiplicative) inverse of a matrix
<code>np.linalg.pinv(a[, rcond, hermitian])</code>	Compute the (Moore-Penrose) pseudo-inverse of a matrix
<code>np.linalg.tensorinv(a[, ind])</code>	Compute the 'inverse' of an N-dimensional array

Notes: the statements included in the 'Routine' column assume NumPy is loaded with the `np` alias.

Matrix and vector products

Snippet 5.15 shows how to use NumPy to get dot, inner, and outer products — whose definitions are reported in equations 4.1, 4.2, and 4.3 respectively:

$$x \cdot y = \sum_{i=1}^n x_i y_i \quad (4.1)$$

Where x and y are vectors of length n .

$$\langle x, y \rangle = x^T y \quad (4.2)$$

Where x^T is the transpose of x .

$$x \otimes y = x y^T \quad (4.3)$$

Where y^T is the transpose of y .

Snippet 5.15 — matrix and vector products

```

1 # import numpy with the socially accepted alias 'np'
2 >>> import numpy as np
3
4 # the arrays
5 >>> X = np.arange(1, 11, 1)
6 >>> X
7 array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10])
8
9 >>> Y = np.arange(10, 0, -1)
10 >>> Y
11 array([10,  9,  8,  7,  6,  5,  4,  3,  2,  1])
12
13 # dot product
14 >>> np.dot(X, Y)
15 220
16
17 # inner product
18 # --+ let's reshape the arrays
19 >>> X = X.reshape(5, 2)
20 >>> X
21 array([[ 1,  2],
22        [ 3,  4],
23        [ 5,  6],
24        [ 7,  8],
25        [ 9, 10]])
26 >>> Y = Y.reshape(5, 2)
27 >>> Y
28 array([[10,  9],
29        [ 8,  7],
30        [ 6,  5],
31        [ 4,  3],
32        [ 2,  1]])
33 >>> np.inner(X, Y)
34 array([[ 28,  22,  16,  10,   4],
35        [ 66,  52,  38,  24,  10],
```



```

36         [104, 82, 60, 38, 16],
37         [142, 112, 82, 52, 22],
38         [180, 142, 104, 66, 28]])
39
40 # outer product
41 >>> np.outer([0, 1, 2], [4, 5, 6, 7])
42 array([[ 0,  0,  0,  0],
43        [ 4,  5,  6,  7],
44        [ 8, 10, 12, 14]])

```

...

Solving a system of equations

Snippet 5.16 shows how to solve the following system of equations:

$$\begin{cases} 4x + 2y = 2000 \\ 7x + 13y = 3000 \end{cases}$$

Snippet 5.16 — solving a system of equations such as

```

1 # import numpy with the socially accepted alias 'np'
2 >>> import numpy as np
3
4 # the system of equations
5 # --+ left-hand side
6 >>> a = np.array([[4, 2], [7, 13]])
7 # --+ right-hand side
8 >>> b = np.array([2000, 3000])
9
10 # solve the system of equations
11 >>> np.linalg.solve(a, b)
12 array([526.31578947, -52.63157895])
13

```

How do I carry out a least-square estimation in NumPy?

The least-square estimator is a popular choice for estimating the relationship between two variables, one of the most common tasks in statistics and econometrics.¹⁹ Specifically, the least-square estimator is defined as follows:

$$\hat{\beta} = (X^T X)^{-1} X^T y \quad (4.4)$$

Where $\hat{\beta}$ is the estimate of the regression coefficients, X is the matrix of regressors, y is the outcome vector, and $X^T X$.

In Snippet 5.17, we create two arrays, $\mathbf{x}$ — playing the role of the regressor — and $\mathbf{y}$ — which we pretend to be the outcome variable. Then, we arrange the data in an array containing $\mathbf{x}$ and a vector of ones. In this way, we add an intercept to the model, namely a scalar that does not change across the observations in the data (see line 12). Finally, we estimate the two regression coefficients of interest:

- The regression coefficient for $\mathbf{x}$, capturing the expected change in $\mathbf{y}$ for a unitary increase in $\mathbf{x}$
- The regression coefficient for the intercept, namely, the expected value of $\mathbf{y}$ when the association between $\mathbf{x}$ and $\mathbf{y}$ is partialled-out

Let me stress that `.linalg.lstsq` returns four pieces of information:

- An array with the estimated regression coefficients
- The sums of squared residuals (i.e., the differences between the observed $\mathbf{y}$ and the values predicted by the model)
- The rank of the matrix containing the regressors
- The singular values of the matrix containing the regressors

Thus, in line 30, we fetch the first element of the returned array, i.e., the regression coefficients.

Figure 4.3 provides a visual representation of the least-square estimator.

Snippet 5.17 — computing the least-square solutions to a linear matrix equation

```

1 # import numpy with the socially accepted alias 'np'
2 >>> import numpy as np
3
4 # the arrays. NOTE they must have the SAME length -- lstsq matches the rows
5 # of A against the entries of y
6 >>> x = np.arange(0, 15, 1)
7 >>> y = [3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25, 27, 29, 31]
8
9 # data preparation / adding a constant to the model
10 >>> A = np.vstack([x, np.ones(len(x))]).T
11 >>> A
12 array([[ 0.,  1.],
13        [ 1.,  1.],
14        [ 2.,  1.],
15        [ 3.,  1.],
16        [ 4.,  1.],
17        [ 5.,  1.],
18        [ 6.,  1.],
19        [ 7.,  1.],
20        [ 8.,  1.],
21        [ 9.,  1.],
22        [10.,  1.],
23        [11.,  1.],
24        [12.,  1.],
25        [13.,  1.],
26        [14.,  1.]])
27
28 # estimate the regression coefficients (a.k.a., the regression slopes) of
29 # the linear model. NOTE rcond=None: leaving it out used to warn, and the
30 # silent default changed in NumPy, so it is worth being explicit
31 >>> b = np.linalg.lstsq(A, y, rcond=None)[0]
32 # these data sit exactly on the line  $y = 2x + 3$ , so the estimator recovers
33 # the generating slope and intercept with no residual
34 >>> b
35 array([2., 3.])

```

4.9 Pseudorandom Number Generation

Why do I need pseudorandom number generators?

What is the gamut of pseudorandom numbers available in NumPy?

Pseudorandom number generators play a central role in a computer simulation, a flexible and powerful tool that can be used in different ways, e.g., to get a better understanding of real-world data, to appreciate the functioning of complex systems, or for scenario analysis.²⁰

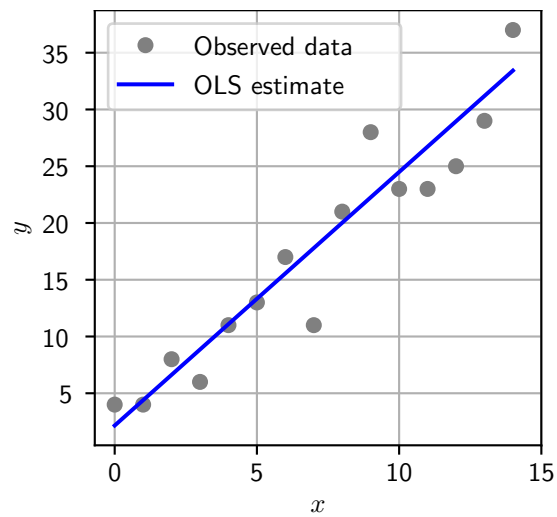


Figure 4.3: A visual illustration of the least-square estimation carried out in Snippet 5.17

NumPy has three families of [pseudorandom number generators](#):

- Simple random data
- Permutations
- Distributions

Tables [4.18](#), [4.19](#), and [4.20](#) provide a summary of the available generators.

TABLE 4.18
NumPy Pseudorandom Generators: Simple Random Data

Routine	Synopsis
<code>integers(low[, high, size, dtype, endpoint])</code>	Return random integers from low (inclusive) to high (exclusive), or if <code>endpoint=True</code> , low (inclusive) to high (inclusive)
<code>random([size, dtype, out])</code>	Return random floats in the half-open interval <code>[0.0, 1.0)</code>
<code>choice(a[, size, replace, p, axis, shuffle])</code>	Generates a random sample from a given array
<code>bytes(length)</code>	Return random bytes

Notes: the statements included in the ‘Routine’ column assume `numpy.random.Generator` has been imported.

TABLE 4.19
NumPy Pseudorandom Generators: Permutations

Routine	Synopsis
<code>shuffle(xL, axis)</code>	Modify an array or sequence in place by shuffling its contents
<code>permutation(xL, axis)</code>	Randomly permute a sequence, or return a permuted range
<code>permuted(xL, axis, out)</code>	Randomly permute x along axis

Notes: the statements included in the ‘Routine’ column assume `numpy.random.Generator` has been imported.

TABLE 4.20
NumPy Pseudorandom Generators: Distributions

Routine	Synopsis
<code>beta(a, b[, size])</code>	Draw samples from a Beta distribution
<code>binomial(n, p[, size])</code>	Draw samples from a binomial distribution
<code>chisquare(df[, size])</code>	Draw samples from a chi-square distribution
<code>dirichlet(alpha[, size])</code>	Draw samples from the Dirichlet distribution
<code>exponential([scale, size])</code>	Draw samples from an exponential distribution
<code>f(dfnum, dfden[, size])</code>	Draw samples from an F distribution
<code>gamma(shape[, scale, size])</code>	Draw samples from a Gamma distribution
<code>geometric(p[, size])</code>	Draw samples from the geometric distribution
<code>gumbel([loc, scale, size])</code>	Draw samples from a Gumbel distribution
<code>hypergeometric(ngood, nbad, nsample[, size])</code>	Draw samples from a Hypergeometric distribution
<code>laplace([loc, scale, size])</code>	Draw samples from the Laplace or double exponential distribution with specified location (or mean) and scale (decay)
<code>logistic([loc, scale, size])</code>	Draw samples from a logistic distribution
<code>lognormal([mean, sigma, size])</code>	Draw samples from a log-normal distribution
<code>logseries(p[, size])</code>	Draw samples from a logarithmic series distribution
<code>multinomial(n, pvals[, size])</code>	Draw samples from a multinomial distribution
<code>multivariate_hypergeometric(colors, nsample)</code>	Generate variates from a multivariate hypergeometric distribution
<code>multivariate_normal(mean, cov[, size, ..])</code>	Draw random samples from a multivariate normal distribution

TABLE 5.20 (cont'd)

Routine	Synopsis
<code>negative_binomial(n, p[, size])</code>	Draw samples from a negative binomial distribution
<code>noncentral_chisquare(df, nonc[, size])</code>	Draw samples from a noncentral chi-square distribution
<code>noncentral_f(dfnum, dfden, nonc[, size])</code>	Draw samples from the noncentral F distribution
<code>normal(loc, scale, size)</code>	Draw random samples from a normal (Gaussian) distribution
<code>pareto(a[, size])</code>	Draw samples from a Pareto II or Lomax distribution with a specified shape
<code>poisson(lam, size)</code>	Draw samples from a Poisson distribution
<code>power(a[, size])</code>	Draws samples in [0, 1] from a power distribution with positive exponent $a - 1$
<code>rayleigh(scale, size)</code>	Draw samples from a Rayleigh distribution
<code>standard_cauchy([size])</code>	Draw samples from a standard Cauchy distribution with mode = 0
<code>standard_exponential([size, dtype, method, out])</code>	Draw samples from the standard exponential distribution
<code>standard_gamma(shape[, size, dtype, out])</code>	Draw samples from a standard Gamma distribution
<code>standard_normal([size, dtype, out])</code>	Draw samples from a standard Normal distribution (mean=0, stdev=1)
<code>standard_t(df[, size])</code>	Draw samples from a standard Student's t distribution with df degrees of freedom
<code>triangular(left, mode, right[, size])</code>	Draw samples from the triangular distribution over the interval [left, right]
<code>uniform([low, high, size])</code>	Draw samples from a uniform distribution
<code>vonmises(mu, kappa[, size])</code>	Draw samples from a von Mises distribution
<code>wald(mean, scale[, size])</code>	Draw samples from a Wald, or inverse Gaussian, distribution
<code>weibull(a[, size])</code>	Draw samples from a Weibull distribution
<code>zipf(a[, size])</code>	Draw samples from a Zipf distribution

Notes: the statements included in the 'Routine' column assume `numpy.random.Generator` has been imported.

Permutation in NumPy

Permuting means changing the order of the elements in an array. Array permutations can be done ‘in-place’ (see Snippet 5.17, line 13), so the original array is modified, or a copy is created (see line 17). Note the outcome of the `np.random` functions are not reproducible. Simply, running the same NumPy generator n times might return n different outcomes. To ensure the reproducibility of NumPy code, we must initialize a random generator instance, as shown in the lower section of Snippet 5.17. Specifically, in line 23, we assign the object `rgn` to the outcome of `.default_rng`. The parameter passed to the generator, known as ‘seed,’ is an arbitrary number object. If you reproduce lines 23–25, you will get the outcome I got, displayed in line 26.

Snippet 5.18 — `.shuffle` Vs. `.permutation`

```

1 # import numpy with the socially accepted alias 'np'
2 >>> import numpy as np
3
4 # shuffling/permuting an existing array
5 # --+ the array
6 >>> A = np.arange(10)
7 >>> A
8 array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
9 # --+ .shuffle produces in-place changes
10 >>> np.random.shuffle(A)
11 >>> A
12 array([6, 9, 3, 8, 7, 2, 4, 1, 0, 5])
13 # --+ .permutation creates a copy
14 >>> np.random.permutation(A)
15 array([7, 6, 5, 2, 1, 8, 0, 4, 9, 3])
16 >>> A
17 array([6, 9, 3, 8, 7, 2, 4, 1, 0, 5])
18
19 # shuffling an existing array and ensuring reproducibility
20 # --+ the array
21 >>> B = [0, 1, 2, 3]
22 # --+ initialize a random generator instance
23 >>> rng = np.random.default_rng(12345)
24 >>> rng.shuffle(B)
25 >>> B
26 [2, 0, 3, 1]
```

Sampling what?

Random sampling is the process of selecting a random subset of elements from an array. NumPy allows us:

- to sample the elements belonging to an existing array
- to sample the elements included in a certain interval
- to sample from a theoretical distribution

Lines 19 and 22 included in Snippet 5.19 show how to sample a given number of elements (see parameter `size`) from an existing array. The code in line 22 differs from the code in line 19 because of the optional parameter `replace` that is set to `False` (default is `True`). In so doing, an element can be sampled once and once only.

In lines 32 and 37, we create two arrays of shape (10000,) from the random normal (see `.random.normal`) and Poisson (see `.random.poisson`) distribution respectively. In the interesting of redundancy, `.random.normal` takes three mandatory arguments: the mean of the distribution (`loc`), the standard deviation of the distribution (`scale`), and the size of the array (`size`); (see `.random.poisson`) takes two mandatory arguments: the expected value/variance of the distribution (`lam`) and the size of the distribution (`size`). In lines 42 - 60, we use Matplotlib to visualize the distribution of the two arrays (see Figure 4.4)

Snippet 5.19 — `.integers` Vs. `.random` Vs. `.choice`

```

1 # import numpy with the socially accepted alias 'np'
2 >>> import numpy as np
3
4 # import a data viz module
5 >>> import matplotlib.pyplot as plt
6
7 # NumPy's modern random API is a Generator object, created with
8 # .default_rng. Seeding it makes the draws below reproducible: run this
9 # snippet yourself and you will get exactly these numbers. The older
10 # np.random.seed / np.random.randint interface still works, but it drives
11 # one global state that any library in the session can disturb
12 >>> rng = np.random.default_rng(42)
13
14 # the array
15 >>> A = np.arange(10)
16
17 # random sampling from an existing array
18 # --+ sampling with replacement (items can be drawn multiple times)

```

```

19 >>> rng.choice(A, size=7)
20 array([0, 7, 6, 4, 4, 8, 0])
21 # --+ sampling without replacement (items can be drawn only once)
22 >>> rng.choice(A, size=7, replace=False)
23 array([1, 3, 9, 2, 0, 7, 6])
24
25 # sampling from a range
26 >>> rng.integers(0, 10, size=4)
27 array([5, 3, 1, 9])
28
29 # sampling from a theoretical distribution
30 # --+ 10,000 items from a normal distribution with a mean of 10 and
31 # standard deviation of 10
32 >>> N = rng.normal(loc=10, scale=10, size=10000)
33 >>> N
34 array([10.66030698, 21.27241207, 14.67509342, ..., 17.69686517,
35        18.65807215,  8.06749993], shape=(10000,))
36 # --+ 10,000 items from the Poisson distribution with lambda=10
37 >>> P = rng.poisson(lam=10, size=10000)
38 >>> P
39 array([15, 12, 15, ..., 10, 10, 14], shape=(10000,))
40
41 # visualize the two arrays
42 # --+ Normal distribution
43 >>> fig = plt.figure(figsize=(3, 3))
44 >>> ax = fig.add_subplot(111)
45 >>> ax.hist(N, color='blue', bins=50)
46 (array([ 1.,  0.,  0.,  0.,  1.,  2.,  1.,  9.,  8., 23., 25.,
47         32., 47., 67., 98., 161., 192., 247., 289., 387., 394., 486.,
48         553., 603., 671., 701., 666., 633., 648., 579., 493., 409., 360.,
49         293., 255., 186., 152., 104., 78., 42., 38., 23., 18.,  8.,
50         9.,  5.,  2.,  0.,  0.,  1.]), array([-33.89114585, -32.20815812, -30.5251704
51         -27.15919495, -25.47620722, -23.7932195 , -22.11023177,
52         -20.42724404, -18.74425632, -17.06126859, -15.37828087,
53         -13.69529314, -12.01230542, -10.32931769,  -8.64632997,
54         -6.96334224,  -5.28035451,  -3.59736679,  -1.91437906,
55         -0.23139134,   1.45159639,   3.13458411,   4.81757184,
56         6.50055956,   8.18354729,   9.86653501,  11.54952274,
57         13.23251047,  14.91549819,  16.59848592,  18.28147364,
58         19.96446137,  21.64744909,  23.33043682,  25.01342454,
59         26.69641227,  28.3794      ,  30.06238772,  31.74537545,
60         33.42836317,  35.1113509 ,  36.79433862,  38.47732635,
61         40.16031407,  41.8433018 ,  43.52628953,  45.20927725,
62         46.89226498,  48.5752527 ,  50.25824043]), <BarContainer object of 50 artists>)
63 >>> ax.set_ylabel('Count')
64 Text(0, 0.5, 'Count')
65 >>> ax.set_xlabel('Value')
66 Text(0.5, 0, 'Value')
67 >>> plt.title('Normal distribution data')
68 Text(0.5, 1.0, 'Normal distribution data')
69 >>> plt.grid(True, ls="--")
70 >>> plt.show()
71 # --+ Poisson distribution. NOTE list(...) around the dictionary views:

```

```

72 # Matplotlib needs a sequence it can index, not a view object
73 >>> from collections import Counter
74 >>> P_FR = Counter(P)
75 >>> fig = plt.figure(figsize=(3, 3))
76 >>> ax = fig.add_subplot(111)
77 >>> ax.scatter(list(P_FR.keys()), list(P_FR.values()), color='blue')
78 <matplotlib.collections.PathCollection object at 0x10b0c1be0>
79 >>> ax.set_ylabel('Count')
80 Text(0, 0.5, 'Count')
81 >>> ax.set_xlabel('Value')
82 Text(0.5, 0, 'Value')
83 >>> plt.title('Poisson distribution data')
84 Text(0.5, 1.0, 'Poisson distribution data')
85 >>> plt.grid(True, ls="--")
86 >>> plt.show()

```

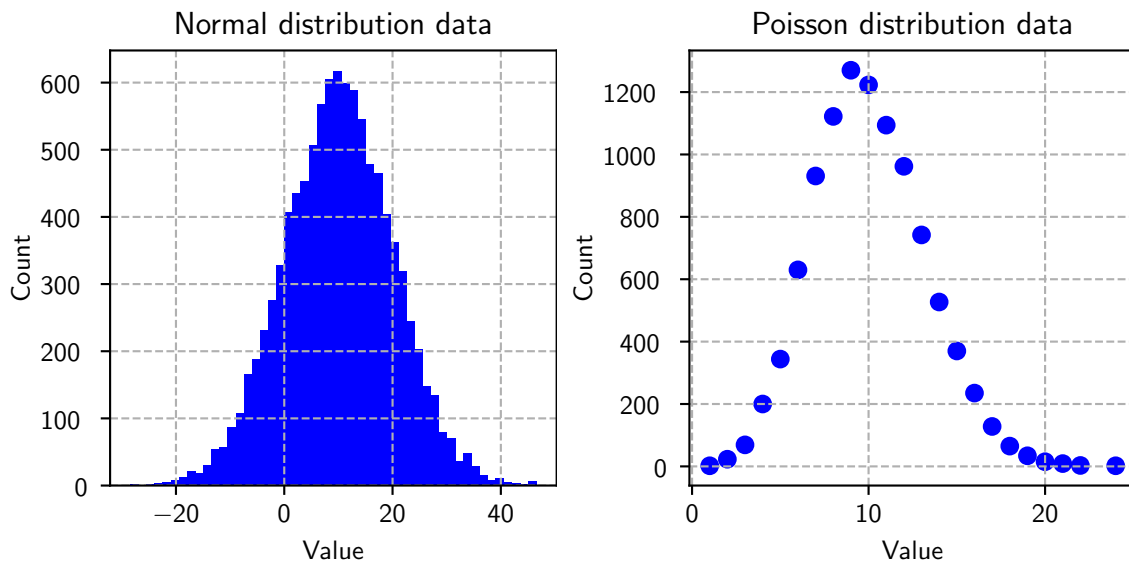


Figure 4.4: A visual illustration of the Normal and Poisson data generated in Snippet 5.19

4.10 File Input and Output (IO) with ndarrays

NumPy binary files

What are the advantages of IO with the .npy format?

In Section 4.3, we saw how to create an array from the data included in a file stored locally. In this section, we will focus on how to input and output NumPy arrays as NumPy binary files

What are the IO routines for the `.npy` format?

Can you show me some IO examples?

There are several advantages:

- Efficiency! Efficiency!! Efficiency!!! IO with NumPy arrays is easy to code and fast to operate
- We do not have to care about character encoding aspects
- It is possible to wrap multiple arrays up in the same file

There are four routines, summarized in Table 4.21.

Of course! Snippet 5.20 deals with the following cases:

- Single array case: lines 18 and 20 illustrate how to write and read an `.npy` file (accepting one and one array only!)
- Multiple array case: lines 35 - 60 illustrate how to write and read an `.npz` file (accepting multiple arrays)

Regarding the ‘multiple array case,’ it is worth noticing that it is possible to write the arrays to a file preserving the variable names assigned to the arrays (e.g., `A`, `B`, `C`, see line 55). Such an option is beneficial when the variable names are meaningful and, perhaps, we expect to load the data back sometime in the future, when we may or may not remember ‘what is what.’

TABLE 4.21
File Input and Output with NumPy Arrays

Routine	Synopsis
<code>np.load(file[, mmap_mode, allow_pickle, ...])</code>	Load arrays or pickled objects from <code>.npy</code> , <code>.npz</code> or pickled files
<code>np.save(file, arr[, allow_pickle, fix_imports])]</code>	Save an array to a binary file in NumPy <code>.npy</code> format
<code>np.savez(file, *args, **kwargs)</code>	Save several arrays into a single file in uncompressed <code>.npz</code> format
<code>np.savez-compressed(file, *args, **kwargs)</code>	Save several arrays into a single file in compressed <code>.npz</code> format

Notes: the statements included in the ‘Routine’ column assume NumPy is loaded with the `np` alias.

Snippet 5.20 — IO with the .npz (.npz) format

```

1 # import numpy with the socially accepted alias 'np'
2 >>> import numpy as np
3
4 # the array
5 >>> A = np.reshape(np.arange(100), (10, 10))
6 >>> A
7 array([[ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9],
8        [10, 11, 12, 13, 14, 15, 16, 17, 18, 19],
9        [20, 21, 22, 23, 24, 25, 26, 27, 28, 29],
10       [30, 31, 32, 33, 34, 35, 36, 37, 38, 39],
11       [40, 41, 42, 43, 44, 45, 46, 47, 48, 49],
12       [50, 51, 52, 53, 54, 55, 56, 57, 58, 59],
13       [60, 61, 62, 63, 64, 65, 66, 67, 68, 69],
14       [70, 71, 72, 73, 74, 75, 76, 77, 78, 79],
15       [80, 81, 82, 83, 84, 85, 86, 87, 88, 89],
16       [90, 91, 92, 93, 94, 95, 96, 97, 98, 99]])
17
18 # save A, delete it, and load the data back in
19 >>> np.save('A.npy', A)
20 >>> del A
21 >>> A = np.load('A.npy')
22 >>> A
23 array([[ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9],
24        [10, 11, 12, 13, 14, 15, 16, 17, 18, 19],
25        [20, 21, 22, 23, 24, 25, 26, 27, 28, 29],
26        [30, 31, 32, 33, 34, 35, 36, 37, 38, 39],
27        [40, 41, 42, 43, 44, 45, 46, 47, 48, 49],
28        [50, 51, 52, 53, 54, 55, 56, 57, 58, 59],
29        [60, 61, 62, 63, 64, 65, 66, 67, 68, 69],
30        [70, 71, 72, 73, 74, 75, 76, 77, 78, 79],
31        [80, 81, 82, 83, 84, 85, 86, 87, 88, 89],
32        [90, 91, 92, 93, 94, 95, 96, 97, 98, 99]])
33
34 # working w/multiple arrays
35 # --+ save A and its transpose into a single file in uncompressed format
36 >>> np.savez('AAT.npz', A, A.T)
37 # --+ load the arrays back
38 # ----+ create a NpzFile object
39 >>> my_arrays = np.load('AAT.npz')
40 # ----+ check the arrays with the file attribute files
41 >>> my_arrays.files
42 ['arr_0', 'arr_1']
43 # ----+ fetch the data on the second item
44 >>> my_arrays['arr_1']
45 array([[ 0, 10, 20, 30, 40, 50, 60, 70, 80, 90],
46        [ 1, 11, 21, 31, 41, 51, 61, 71, 81, 91],
47        [ 2, 12, 22, 32, 42, 52, 62, 72, 82, 92],
48        [ 3, 13, 23, 33, 43, 53, 63, 73, 83, 93],
49        [ 4, 14, 24, 34, 44, 54, 64, 74, 84, 94],
50        [ 5, 15, 25, 35, 45, 55, 65, 75, 85, 95],
51        [ 6, 16, 26, 36, 46, 56, 66, 76, 86, 96],
52        [ 7, 17, 27, 37, 47, 57, 67, 77, 87, 97],

```

```

53         [ 8, 18, 28, 38, 48, 58, 68, 78, 88, 98],
54         [ 9, 19, 29, 39, 49, 59, 69, 79, 89, 99]])
55 # --+ it is also possible to save the arrays with their original names
56 >>> np.savez('AAT.npz', A=A, AT=A.T)
57 >>> my_arrays = np.load('AAT.npz')
58 >>> my_arrays.files
59 ['A', 'AT']
60 >>> AT = my_arrays['AT']
61 >>> AT
62 array([[ 0, 10, 20, 30, 40, 50, 60, 70, 80, 90],
63        [ 1, 11, 21, 31, 41, 51, 61, 71, 81, 91],
64        [ 2, 12, 22, 32, 42, 52, 62, 72, 82, 92],
65        [ 3, 13, 23, 33, 43, 53, 63, 73, 83, 93],
66        [ 4, 14, 24, 34, 44, 54, 64, 74, 84, 94],
67        [ 5, 15, 25, 35, 45, 55, 65, 75, 85, 95],
68        [ 6, 16, 26, 36, 46, 56, 66, 76, 86, 96],
69        [ 7, 17, 27, 37, 47, 57, 67, 77, 87, 97],
70        [ 8, 18, 28, 38, 48, 58, 68, 78, 88, 98],
71        [ 9, 19, 29, 39, 49, 59, 69, 79, 89, 99]])
72

```

Notes

¹⁴The Internet has many blog posts showing the performance of NumPy in linear algebra tasks is comparable to compiled languages, such as C.

¹⁵This feature was introduced with NumPy 1.23.

¹⁶The documentation of NumPy 1.23 states that *It is no longer recommended to use this class, even for linear algebra. Instead, use regular arrays. The class may be removed in the future.*

¹⁷Here is an interesting passage from the official NumPy documentation: *NumPy hands off array processing to C, where looping and computation are much faster than in Python. To exploit this, programmers using NumPy eliminate Python loops in favor of array-to-array operations. vectorization can refer both to the C offloading and to structuring NumPy code to leverage it.*

¹⁸Matlab is a numeric computing environment that is particularly popular in academia and industry.

¹⁹Students who want to familiarize with the field of econometrics are warmly encouraged to read Angrist, Joshua D., and Jörn-Steffen Pischke. *Mostly harmless econometrics: An empiricist's companion*. Princeton university press, 2009.

²⁰The students who want to familiarize themselves with the role of computer simulation in understanding economics and social formations may want to refer to i) Axelrod, R. *The complexity of cooperation*, Princeton University Press, 1997; ii) Schelling, T. C. *Micromotives and macrobehavior*. WW Norton & Company, 1978; iii) Epstein, J. M. & Axtell, R. *Growing Artificial Societies—Social Science from the Bottom Up*. Artif Life 3, 237–242, 1997.

Chapter 5

Data Management with Pandas

At the end of the chapter, you will be able to:

- Create a DataFrame
 - Manipulate the individual columns of a DataFrame
 - Arrange a DataFrame in the way that best fits your data analysis needs
 - Group, the cases of DataFrame, to perform aggregation and transformation tasks
 - Work with multiple DataFrames
 - Read and write data in a variety of formats
-

5.1 Pandas 101

What is special about Pandas?

Pandas is the *de facto* standard when it comes to managing data (in general, not only in Python!). So, what is special about Pandas?

- Compared to competing frameworks for data management,²¹ Pandas offers a unique combination of features, including ease of use, flexibility, and performance
- Pandas has an edge in the area of time series²² and panel data²³— which is not surprising since Pandas was created for quantitative financial analysts, and it is still prevalent in the field of finance at large²⁴
- Pandas builds on NumPy. That means access to a large variety of vectorized functions; that is, users/developers do not have to write boring and inefficient loops to perform operations on data
- The quality of the documentation covering Pandas is off-the-chart.²⁵

Is Pandas a must-have skill and why?

Got it: Pandas is a must-have skill. But what can I achieve with it?

Yes, it is. The reason is straightforward: data are clean and easy to work with in an ideal world. In the real world, data are messy and difficult to work with. So, it is important to have a tool that can help you work with data. Also, junior analysts are — often — required to carry out a substantial amount of data management. So you may want to get prepared before you land in the industry ...

Data scientists and quantitative analysts use Pandas to carry out tasks falling into the following families:

- *Data preparation*, consisting of maximizing the quality of the data at hand; that is, getting information offering the most accurate representation of the business, economic, or financial process of interest. An example is cleaning the outcome of a web-crawling project targeting an online community of, say, beer enthusiasts
- *Data augmentation*, consisting of expanding on the raw information to create the variables that best capture the process we want to analyze. An example is creating a measure of customer satisfaction based on the data acquired using web-crawling
- *Data transformation*, consisting of arranging the data in the way that best supports a data visualization or analysis task. An example is converting a long data table, wherein observations are nested in one or multiple grouping variables, to a wide structure (hold your horses, we will see a concrete example later on in this chapter)

5.2 The Pandas DataFrame

What is a Pandas DataFrame?

Per the [Pandas API](#), a DataFrame is a:

“two-dimensional, size-mutable, potentially heterogeneous tabular data”

Put simply (i.e., from a user standpoint), a DataFrame is a tabular data structure with rows and columns. Typically, the rows are the cases (e.g., individuals, groups, firms, countries, etc.), and the columns are the so-called fields, variables, or features (e.g., wage, job satisfaction, stock-market value, etc.).²⁶ Figure 5.1 shows a stylized representation of a case-by-variable data structure, very common in Pandas.

Case	Var a	Var b	...	Var i	...	Var k
1	a_1	b_1	...	i_1	...	k_1
2	a_2	b_2	...	i_2	...	k_2
j	a_j	b_j	...	i_j	...	k_j
n	a_n	b_n	...	i_n	...	k_n

Figure 5.1: A stylized representation of a case-by-variable data structure

How do I create a DataFrame?

Mainly, there are two alternatives:

- Option 1: passing tables loaded onto the current Python session²⁷ to the `.DataFrame` class
- Option 2: sourcing external data, available in a local file or on a server,

How do I pass an iterable to the DataFrame class?

Table 5.1 shows three functions that can be used to create a DataFrame from existing iterables, namely:

- `.DataFrame.from_dict`, which creates a DataFrame from a dictionary
- `.DataFrame.from_records`, which creates a DataFrame from a list of tuples
- `.DataFrame.sparse.from_spmatrix`, which creates a DataFrame from a SciPy sparse matrix (a convenient option when it comes to manipulate network data, which, oftentimes, exhibit sharp sparseness)

TABLE 5.1
Creating a DataFrame from Data Loaded in the Python Session

Routine	Synopsis
<code>pd.DataFrame.from_dict</code>	Construct DataFrame from dict of array-like or dicts
<code>pd.DataFrame.from_records</code>	Convert structured or record ndarray to DataFrame
<code>pd.DataFrame.sparse.from_spmatrix</code>	Create a new DataFrame from a SciPy sparse matrix (helpful for network data)

Notes: the statements included in the ‘Routine’ column assume Pandas is loaded with the `pd` alias.

Can you show me
`.DataFrame.from_dict` in
 action?

Snippet 6.1 shows how to create a `DataFrame` from one or more iterables loaded in the current Python session. In line 5, we create a dictionary whose keys are associated with variables — i.e., the columns of the `DataFrame`. In line 10, we assign the variable `df` to the output of `.from_dict` with the dictionary `my_data` as input. In line 11, `df` is printed to the screen. You may have noticed that the mathematical progression reported on the left-right of the tabular data is the so-called Pandas ‘index,’ a concept we will see in Section 5.3.

By default, `.from_dict` parses the values of the input dictionary as columns. However, it is also possible to parse a dictionary’s values as cases. We achieve that in line 30, where we populate the discretionary parameter `orient` with the value `index`, meaning Python must consider the dictionary keys as cases rather than columns. In line 38, we make a further adjustment by passing a list with the column names to the discretionary parameter `columns`. It is not necessary to do that, but it helps to interpret the columns — which, when we set `orient="index"`, are named with a mathematical progression by default.

Snippet 6.1 — creating a `DataFrame` from a dictionary

```

1 # import pandas with the socially accepted alias 'pd'
2 >>> import pandas as pd
3
4 # create a dictionary of arrays
5 >>> my_data = {
6 ...     "var_1": [1, 2, 3, 4, 5],
7 ...     "var_2": ["ABC", "Hello world", "Bazinga!", "cheers", "ciao"],
8 ... }
9 # get a DataFrame from the dictionary and display it
10 >>> df = pd.DataFrame.from_dict(my_data)
11 >>> df
12      var_1      var_2
13 0         1        ABC
14 1         2  Hello world
15 2         3   Bazinga!
16 3         4    cheers
17 4         5        ciao
18
19 # create a dictionary whose keys are cases
20 >>> my_data = {"case_1": ["Pluto", "dog"], "case_2": ["Goofy", "dog"]}
21
22
23 # get a DataFrame from the dictionary and display it
24 >>> df = pd.DataFrame.from_dict(my_data)
25 >>> df
26      case_1 case_2

```

```

27 0   Pluto   Goofy
28 1     dog    dog
29
30 # ... something wrong here - let us adjust the optional param 'orient'
31 >>> df = pd.DataFrame.from_dict(my_data, orient="index")
32 >>> df
33
34      0    1
35 case_1  Pluto  dog
36 case_2  Goofy  dog
37
38 # ... still something wrong here - where are the column names?
39 >>> df = pd.DataFrame.from_dict(
40 ...     my_data, orient="index",
41 ...     columns=["name", "species"]
42 ... )
43 >>> df
44      name species
45 case_1  Pluto   dog
46 case_2  Goofy   dog

```

How do I create a DataFrame from 'external' data?

As we will see in Section 5.13, there are plenty of Pandas IO utilities that target alternative file formats/extensions (e.g., `.json`, `.csv`, `.xlsx`). In this section, I focus on one specific case: creating a DataFrame from a CSV file. In the first part of the snippet, up until line 17, we create fake data and write them to a local `.csv` file. Creating a DataFrame from a CSV file is a simple matter of passing the name of the file (or a file path) to the `.read_csv`. Such a function has a substantial number of discretionary parameters — I warmly encourage you to go through the documentation and familiarize with the various options.

Snippet 6.2 — creating a DataFrame from a CSV file

```

1  # import pandas with the socially accepted alias 'pd'
2  >>> import pandas as pd
3
4  # create fake data and write them to a .csv
5  # --+ column names
6  >>> columns = ["x", "y"]
7  # --+ column values
8  >>> x = [0, 1, 2]
9  >>> y = ["A", "B", "C"]
10 # --+ write the data to a file. The `with` block closes the file on the way
11 #     out, so there is no f.close() to remember
12 >>> with open("my_data.csv", "w") as f:
13 ...     # write the column names first
14 ...     f.write(",".join(columns) + "\n")
15 ...     # then, write the data case-by-case

```

```

16 ...     for i, j in zip(x, y):
17 ...         f.write("{}{}".format(i, j) + "\n")
18
19 # create a DataFrame from the .csv file and display it
20 >>> df = pd.read_csv("my_data.csv")
21 >>> df
22      x  y
23 0  0  A
24 1  1  B
25 2  2  C

```

5.3 Checking a DataFrame's Attributes

What are the key attributes of a DataFrame?

Pandas DataFrame can be characterized along several attributes, such as:

- Shape
- Size
- Number of dimensions
- Set of column names
- Set of case indices

A convenient way to access multiple DataFrame's attributes in a row is by using the `.info` method. Per Snippet 6.3 (see lines 19-27), we know that `df` has three entries and as many indices, two columns (an object, `country`, and a float `gdp_pc`), and occupies circa 176 bytes in memory.

The bottom section of Snippet 6.3 shows how to access the individual attributes of a DataFrame using `.shape`, `.size`, `.ndim`, and `.memory_usage`.

Snippet 6.3 — accessing a DataFrame's attributes with `.info`

```

1 # import pandas with the socially accepted alias 'pd'
2 >>> import pandas as pd
3
4 # create a DataFrame from a dictionary
5 >>> gdp_data = {
6 ...     "country": ["Belgium", "France", "Germany"],
7 ...     "gdp_pc": [51767.8, 43518.5, 50801.8]
8 ... }
9 >>> df = pd.DataFrame.from_dict(gdp_data)
10 >>> df
11      country  gdp_pc
12 0  Belgium  51767.8
13 1   France  43518.5
14 2  Germany  50801.8
15

```



```

16 # get DataFrame 'info'
17 >>> df.info()
18 <class 'pandas.DataFrame'>
19 RangeIndex: 3 entries, 0 to 2
20 Data columns (total 2 columns):
21 #   Column   Non-Null Count  Dtype
22 ---  ---
23 0    country   3 non-null      str
24 1    gdp_pc     3 non-null      float64
25 dtypes: float64(1), str(1)
26 memory usage: 180.0 bytes
27
28 # let us get df's attributes one-by-one
29 # --+ shape (cases by columns)
30 >>> df.shape
31 (3, 2)
32 # --+ size (cases X columns)
33 >>> df.size
34 6
35 # --+ number of dimensions (columns)
36 >>> df.ndim
37 2
38 # --+ check memory usage column-by-column
39 >>> df.memory_usage()
40 Index      132
41 country     24
42 gdp_pc      24
43 dtype: int64

```

5.4 The Anatomy of a DataFrame

What are the components of a DataFrame?

Mainly, DataFrames have two components:

- An [index](#)
- A set of columns, each of which is a [Pandas Series](#)

What is the index of a DataFrame?

Put simple, it is a Python object associated with a case or cases in a DataFrame. The index is the primary key for a DataFrame.

How does a DataFrame index matter?

The index is the primary key for a DataFrame and makes data querying easier and more efficient (we will see this in Section [5.5](#)).

Is it mandatory to pass an index when I create a DataFrame?

How do I access a DataFrame index?

Can I edit a DataFrame index?

In general, it is not necessary. When we do not pass an index, Pandas will create a mathematical progression and assign it to the index (see Snippet 6.3).

`index` is a DataFrame's attribute (in a Pythonic sense!). Snippet 6.4 shows how to access the index (line 13). Also, the snippet shows that a `RangeIndex` object is an iterable object (line 17).

Yes, you can. Snippet 6.4 illustrates assigning a DataFrame to an iterable object (line 24). To change the index, it is also possible to use the function `.set_index` (see Section 5.6.)

Snippet 6.4 — accessing and amending a DataFrame's index

```

1 # import pandas with the socially accepted alias 'pd'
2 >>> import pandas as pd
3
4 # create a DataFrame from a dictionary
5 >>> df = pd.DataFrame.from_dict({"S":["s1", "s2", "s3"], "X":[-99, 8, 0]})
6 >>> df
7      S    X
8 0  s1 -99
9 1  s2   8
10 2  s3   0
11
12 # access the index
13 >>> df.index
14 RangeIndex(start=0, stop=3, step=1)
15
16 # iterate over the index
17 >>> for item in df.index:
18 ...     print(item)
19 0
20 1
21 2
22
23 # change the index
24 >>> df.index = ["case_1", "case_2", "case_3"]
25 >>> df
26      S    X
27 case_1 s1 -99
28 case_2 s2   8
29 case_3 s3   0

```

What is a Pandas Series?

A **Pandas Series** is a one-dimensional object. The columns of a DataFrame are a collection of Series objects.

How do I create a Series?

It is possible to create a Series using the `.series` class (see Snippet 6.5, line 5).

How do I access a Series included in DataFrame?

A Series object can be accessed as a DataFrame attribute (see Snippet 6.5, line 14). It is self-evident that a Series borrows the index of its parent DataFrame (see lines 17 and 26).

Snippet 6.5 — accessing a Series included in a DataFrame

```

1  # import pandas with the socially accepted alias 'pd'
2  >>> import pandas as pd
3
4  # create a series
5  >>> S = pd.Series(['s1', 's2', 's3'])
6  >>> print(S)
7  0    s1
8  1    s2
9  2    s3
10 dtype: str
11
12 # accessing a DataFrame column as a Series
13 # --+ the data
14 >>> df = pd.DataFrame.from_dict({"S":["s1", "s2", "s3"], "X":[-99, 8, 0]})
15 # --+ assign S to the fetched column and print S
16 >>> S = df.S
17 >>> print(S)
18 0    s1
19 1    s2
20 2    s3
21 Name: S, dtype: str
22 # --+ amend the index
23 >>> df.index = ["case_1", "case_2", "case_3"]
24 # --+ assign S to the fetched column and print S
25 >>> S = df.S
26 >>> print(S)
27 case_1    s1
28 case_2    s2
29 case_3    s3
30 Name: S, dtype: str

```

5.5 Querying DataFrames

What is data querying?

How do I query data in Pandas?

Data queries can take two non-mutually exclusive forms. One may want to:

- Select a subset of cases
- Select a subset of columns

Pandas DataFrame objects have a property called `.loc` that allows accessing a group of rows and columns by label(s) or a boolean array. To use `.loc`, one has to pass two inputs among brackets:

- The set of cases to select (the left-hand side element among brackets)
- The set of columns to select (the right-hand side element among brackets)

Snippet 6.6 shows a couple of use cases for the `.loc` property:

- Line 34 selects the first two cases of the DataFrame
- Line 41 selects the column “price” first two rows of the DataFrame
- Line 48 selects the columns “price” and “color” for all cases included in the DataFrame
- Line 55 selects the cases for which the column “price” is less than or equal to 8.00
- Line 62 selects the cases for which the column “price” is less than 9.00 **and** the column “color” is equal to “green”
- Line 68 expands on line 62 by selecting the column “price” when “price” is less than 9.00 **and** “color” is equal to “green”

Please refer to sections 3.8 and 3.9 to consolidate your knowledge on the Python syntax of statements and control flow.

Snippet 6.6 — querying data in Pandas with `.loc`

```

1 # import pandas with the socially accepted alias 'pd'
2 >>> import pandas as pd
3
4 # create a DataFrame from a dictionary
5 >>> df = pd.DataFrame.from_dict(

```

```

6 ...     {
7 ...         "product": ["a", "b", "c"],
8 ...         "price": [9.87, 8.63, 6.45],
9 ...         "color": ["green", "green", "blue"],
10 ...     }
11 ... )
12
13 # data preview
14 >>> df
15   product  price  color
16 0        a   9.87  green
17 1        b   8.63  green
18 2        c   6.45   blue
19
20 # info
21 >>> df.info()
22 <class 'pandas.DataFrame'>
23 RangeIndex: 3 entries, 0 to 2
24 Data columns (total 3 columns):
25  #   Column      Non-Null Count  Dtype
26  ---  ---
27 0   product      3 non-null      str
28 1   price        3 non-null      float64
29 2   color        3 non-null      str
30 dtypes: float64(1), str(2)
31 memory usage: 204.0 bytes
32
33 # select the first the two cases using a range of indices
34 >>> df.loc[0:2]
35   product  price  color
36 0        a   9.87  green
37 1        b   8.63  green
38 2        c   6.45   blue
39
40 # select the column "price" for the first two cases using a boolean array
41 >>> df.loc[0:2, "price"]
42 0    9.87
43 1    8.63
44 2    6.45
45 Name: price, dtype: float64
46
47 # select the columns "price" and "color" for all cases
48 >>> df.loc[:, ["price", "color"]]
49   price  color
50 0   9.87  green
51 1   8.63  green
52 2   6.45   blue
53
54 # select all cases for which the column "price" is greater than or equal to 8.00
55 >>> df.loc[df["price"] >= 8.00, ]
56   product  price  color
57 0        a   9.87  green
58 1        b   8.63  green

```

```

59
60 # select all cases for which the column "price" is less than 9.00 and
61 # "color" is equal to "green"
62 >>> df.loc[(df["price"] < 9.00) & (df["color"] == "green" ), ]
63    product  price  color
64 1         b   8.63  green
65
66 # select all cases for which the column "price" is less than to 9.00 and
67 # "color" is equal to "green"; also, keep the column "price" only
68 >>> df.loc[(df["price"] < 9.00) & (df["color"] == "green" ), "price"]
69 1      8.63
70 Name: price, dtype: float64

```

5.6 Manipulating DataFrame Columns

What are the main operations on a DataFrame's columns?

Mainly, one may want to:

- Rename a column
- Drop a column
- Create a new column
- Amend an existing column's values

How do I rename or drop a column in Pandas?

As shown in Snippet 6.7, Pandas has two methods, `.rename()` and `.drop()`, to rename and drop columns respectively.

Line 36 shows how to rename a column. A first argument is a dictionary mapping the new name onto the old name (the dictionary's key). The second argument is a boolean flag that indicates whether to make the change effective (if "True") or not (if "False").

Lines 46 and 52 show how to drop a column. The first argument is an array with the name(s) of the column(s) to delete. The second argument is a boolean flag that indicates whether to make the change effective (if "True") or not (if "False").

Snippet 6.7 — renaming and dropping DataFrame columns

```

1 # import pandas with the socially accepted alias 'pd'
2 >>> import pandas as pd
3
4 # the df
5 >>> df = pd.DataFrame(
6     ...     {
7     ...         "laptop": ["MacBook Pro 13inch", "Thinkpad T14", "Dell XPS 13"],
8     ...         "ram": ["16 GB", "48 GB", "8 GB"],
9     ...         "os": ["macOS Monterey", "Debian 11", "Windows 11"],

```

```

10 ...             "chip": ["M1", "Ryzen", "Intel Core i7"]
11 ...         }
12 ...     )
13
14 # data view
15 >>> df
16           laptop    ram    os    chip
17 0  MacBook Pro 13inch  16 GB  macOS Monterey    M1
18 1      Thinkpad T14  48 GB    Debian 11    Ryzen
19 2      Dell XPS 13   8 GB    Windows 11 Intel Core i7
20
21 # info
22 >>> df.info()
23 <class 'pandas.DataFrame'>
24 RangeIndex: 3 entries, 0 to 2
25 Data columns (total 4 columns):
26 #   Column  Non-Null Count  Dtype
27 ---  -
28 0   laptop  3 non-null      str
29 1   ram     3 non-null      str
30 2   os      3 non-null      str
31 3   chip    3 non-null      str
32 dtypes: str(4)
33 memory usage: 228.0 bytes
34
35 # rename the column "ram" to "memory"
36 >>> df.rename(columns={"ram": "memory"}, inplace=True)
37 >>> df
38           laptop memory    os    chip
39 0  MacBook Pro 13inch  16 GB  macOS Monterey    M1
40 1      Thinkpad T14  48 GB    Debian 11    Ryzen
41 2      Dell XPS 13   8 GB    Windows 11 Intel Core i7
42
43 # drop the column "chip"
44 # --+ when inplace is set to False, the outcom of .drop is sent to the
45 # interactive sessions, but the data in memory are not affected
46 >>> df.drop(columns=["chip"], inplace=False)
47           laptop memory    os
48 0  MacBook Pro 13inch  16 GB  macOS Monterey
49 1      Thinkpad T14  48 GB    Debian 11
50 2      Dell XPS 13   8 GB    Windows 11
51 # --+ when inplace is set to True, the data in memory are affected
52 >>> df.drop(columns=["chip"], inplace=True)
53 >>> df
54           laptop memory    os
55 0  MacBook Pro 13inch  16 GB  macOS Monterey
56 1      Thinkpad T14  48 GB    Debian 11
57 2      Dell XPS 13   8 GB    Windows 11

```

How do I create a new column or amend an existing Pandas column?

The `.loc` method is the most appropriate way to create a new column or alter an existing column's values. Snippet 6.8 illustrates a couple of common tasks. In line 16, we assign a list of string objects to a new column called `manufacturer` for all cases included in `df`.

In lines 27, 47, and 63 we amend the value of an existing column. First, we manipulate the strings included under `RAM` to drop every character that is not a digit, turning "16 GB" into "16" (line 27). Note the `regex=True` there: since Pandas 2.0, `.str.replace` treats its first argument as a literal string unless told otherwise, so a pattern passed without it silently matches nothing. Second, we change the type of the column from `string` to `integer` using the Pandas' `.astype()` method (line 47). Then, we take the log of `ram` and assign it to the new column `log_ram` (line 63).

In line 80, we use `.loc` to create a new variable for a subset of cases only. Specifically, we populate the column `gpu` provided the value of `laptop` is "Thinkpad T14."

Snippet 6.8 — creating and amending DataFrame columns

```

1 # import pandas and numpy with the socially accepted aliases
2 >>> import numpy as np
3 >>> import pandas as pd
4
5 # the data
6 >>> df = pd.DataFrame(
7 ...     {
8 ...         "laptop": ["MacBook Pro 13inch", "Thinkpad T14", "Dell XPS 13"],
9 ...         "ram": ["16 GB", "48 GB", "8 GB"],
10 ...         "os": ["macOS Monterey", "Debian 11", "Windows 11"],
11 ...         "chip": ["M1", "Ryzen", "Intel Core i7"],
12 ...     }
13 ... )
14
15 # create a new column from a list
16 >>> df.loc[:, "manufacturer"] = ["Apple", "Lenovo", "Dell"]
17 >>> df
18
19      laptop    ram    os    chip manufacturer
20 0  MacBook Pro 13inch  16 GB  macOS Monterey    M1      Apple
21 1    Thinkpad T14  48 GB    Debian 11    Ryzen    Lenovo
22 2    Dell XPS 13    8 GB    Windows 11 Intel Core i7    Dell
23
24 # clean an existing column
25 # --+ drop every character that is not a digit. NOTE the regex=True: since
26 #     Pandas 2.0 .str.replace treats its argument as a LITERAL string by
27 #     default, so a pattern without regex=True silently does nothing
28 >>> df.loc[:, "ram"] = df["ram"].str.replace(r"[^0-9]", "", regex=True)

```



```

28 >>> df.info()
29 <class 'pandas.DataFrame'>
30 RangeIndex: 3 entries, 0 to 2
31 Data columns (total 5 columns):
32 #   Column          Non-Null Count  Dtype
33 ---  ---
34 0   laptop          3 non-null     str
35 1   ram              3 non-null     str
36 2   os               3 non-null     str
37 3   chip             3 non-null     str
38 4   manufacturer    3 non-null     str
39 dtypes: str(5)
40 memory usage: 252.0 bytes
41
42 # change a column's data type
43 # --+ NOTE the plain [] assignment. .loc writes INTO the existing column, so
44 #     it must respect that column's dtype -- and on Pandas 3 writing integers
45 #     into a str column raises TypeError. Plain [] REPLACES the column, which
46 #     is what a cast needs, and works on every Pandas version
47 >>> df["ram"] = df["ram"].astype(int)
48 >>> df.info()
49 <class 'pandas.DataFrame'>
50 RangeIndex: 3 entries, 0 to 2
51 Data columns (total 5 columns):
52 #   Column          Non-Null Count  Dtype
53 ---  ---
54 0   laptop          3 non-null     str
55 1   ram              3 non-null     int64
56 2   os               3 non-null     str
57 3   chip             3 non-null     str
58 4   manufacturer    3 non-null     str
59 dtypes: int64(1), str(4)
60 memory usage: 252.0 bytes
61
62 # create a new column out of an existing one
63 >>> df["log_ram"] = np.log(df["ram"])
64 >>> df.info()
65 <class 'pandas.DataFrame'>
66 RangeIndex: 3 entries, 0 to 2
67 Data columns (total 6 columns):
68 #   Column          Non-Null Count  Dtype
69 ---  ---
70 0   laptop          3 non-null     str
71 1   ram              3 non-null     int64
72 2   os               3 non-null     str
73 3   chip             3 non-null     str
74 4   manufacturer    3 non-null     str
75 5   log_ram          3 non-null     float64
76 dtypes: float64(1), int64(1), str(4)
77 memory usage: 276.0 bytes
78
79 # create a new column conditional on another column's value
80 >>> df.loc[df["laptop"] == "Thinkpad T14", "gpu"] = True

```

```

81 >>> df.info()
82 <class 'pandas.DataFrame'>
83 RangeIndex: 3 entries, 0 to 2
84 Data columns (total 7 columns):
85  #   Column                Non-Null Count  Dtype
86  ---  ---
87  0   laptop                3 non-null     str
88  1   ram                   3 non-null     int64
89  2   os                    3 non-null     str
90  3   chip                  3 non-null     str
91  4   manufacturer          3 non-null     str
92  5   log_ram               3 non-null     float64
93  6   gpu                   1 non-null     object
94 dtypes: float64(1), int64(1), object(1), str(4)
95 memory usage: 300.0+ bytes
96 # --+ preview
97 >>> df
98               laptop  ram      os      chip manufacturer  log_ram  gpu
99 0  MacBook Pro 13inch  16  macOS Monterey      M1      Apple  2.772589  NaN
100 1      Thinkpad T14   48   Debian 11      Ryzen     Lenovo  3.871201  True
101 2      Dell XPS 13    8    Windows 11  Intel Core i7     Dell  2.079442  NaN

```

What is a missing value?

A missing value is a datapoint for which no information is available in the dataset. Missing values can arise for many reasons. The most popular reasons are:

- the case does not present a value for the variable. For example, it is not possible to record the stock market of a company before the IPO
- the case does present a value for the variable. However, the value was not recorded because of the limitations of the data gathering process
- the recorded value is not accurate/valid — hence, it was removed from the dataset

How does Pandas represent missing values?

Pandas denote missing values using NumPy's floating representation `numpy.NaN`

How do I handle missing values in Pandas?

There are several approaches to cope with missing values:

- deleting all cases where a missing value is present
- replacing missing values with a fixed value
- replacing missing values with the mean of the column
- replacing missing values with the estimate of a statical model

The latter approach is the most sophisticated one and falls beyond the remit of these notes. In Snippet 6.9, I show how to implement the first three approaches in Pandas. Line 36 uses the `.dropna()` method to delete all cases presenting at least one missing value. Line 43 draws on the `.fillna()` method to replace the missing values with a fixed value (e.g., a value that the analyst considers ‘plausible’ based on her/his contextual knowledge). Like line 43, line 52 relies the `.fillna()` method. However, it does not replace the missing values with a fixed value. Instead, it uses the mean value of `price`.

Snippet 6.9 — handling missing values

```

1  .
2  # import numpy with the socially accepted alias "np"
3  >>> import numpy as np
4  # import pandas with the socially acceptable alias "pd"
5  >>> import pandas as pd
6
7  # the dataframe
8  >>> df = pd.DataFrame(
9  ...     {
10 ...         "item": ["a", "b", "c", "d", "e"],
11 ...         "price": [16.32, 16.78, np.nan, np.nan, 16.41],
12 ...     }
13 ... )
14
15 # data view
16 >>> df
17    item  price
18 0     a  16.32
19 1     b  16.78
20 2     c    NaN
21 3     d    NaN
22 4     e  16.41
23
24 # info
25 >>> df.info()
```

```

26 <class 'pandas.DataFrame'>
27 RangeIndex: 5 entries, 0 to 4
28 Data columns (total 2 columns):
29 #   Column  Non-Null Count  Dtype
30 ---  ---
31 0    item      5 non-null      str
32 1    price     3 non-null      float64
33 dtypes: float64(1), str(1)
34 memory usage: 212.0 bytes
35
36 # approach 1 --- delete all cases where a missing value is present
37 >>> df.dropna()
38    item  price
39 0     a  16.32
40 1     b  16.78
41 4     e  16.41
42
43 # approach 2 --- replace missing values with a fixed value
44 >>> df.fillna(value=16.99, inplace=False)
45    item  price
46 0     a  16.32
47 1     b  16.78
48 2     c  16.99
49 3     d  16.99
50 4     e  16.41
51
52 # approach 3 --- replace missing values with the mean of the columns
53 >>> df.fillna(value=np.mean(df["price"]), inplace=False)
54    item      price
55 0     a  16.320000
56 1     b  16.780000
57 2     c  16.503333
58 3     d  16.503333
59 4     e  16.410000

```

5.7 Data Types and Pandas

What are the data types admitted in a Pandas object?

Can a Series contain multiple data types?

Mainly, Pandas uses NumPy arrays as the concrete objects contained with an Index, Series, or DataFrame. To recall the various NumPy dtypes, please refer to Sections 4.2.

Yes, a Series can contain multiple data types (hence, a DataFrame can). In this sense, Pandas offers a more flexible data structure than NumPy arrays, which must contain objects of the same type. Snippet 6.10 shows that a list with mixed types is downgraded to a NumPy array with string objects. Instead, a Series preserves the type of each object (see line 23).

Snippet 6.10 — multiple data types in a Series

```

1 # import numpy and pandas with the socially accepted alias 'np' and 'pd'
2 >>> import numpy as np
3 >>> import pandas as pd
4
5 # a list with a string, a float, and an integer
6 >>> L = ["xyz", -17.64, 0]
7
8 # create a NumPy array from the list
9 >>> A = np.array(L)
10 >>> print(A)
11 ['xyz' '-17.64' '0']
12
13 # create a Series from the list
14 >>> S = pd.Series(L)
15 >>> print(S)
16 0      xyz
17 1    -17.64
18 2         0
19 dtype: object
20
21 # proof that Series preserves the type of the individual object
22 # --+ fetch the second item of the list and carry out a mathematical operation
23 >>> S[1]/2
24 -8.82

```

5.8 Handling ‘Time’ Data in Pandas

How good is Pandas at handling ‘time’ data?

How many types of ‘time’ data are supported in Pandas?

Pandas excels at handling time data, a key component of time series and panels.

Pandas has three kinds of objects that can be used to store and represent information on time-related quantities:

- `.Timestamp`, the Pandas replacement for Python’s `Datetime` object, represent a specific point in time
- `.Timedelta` represents two dates or time
- `.Period` represents a period of time

Can you give me an example of `.Timestamp` and `.Timedelta` objects?

Snippet 6.11 illustrates how to initialize and use `.Timestamp` and `.Timedelta` objects. The output of line 30 shows that the column `timestamp` has been parsed as a string. At this point, we cannot perform any operation on it (e.g., getting the time elapsed since an event). Hence, in line 46 we pass the values under the column `timestamp` to `.Timestamp`. To apply the transformation column-wise, we draw on the `.apply()` method, which takes a function as an argument. Specifically, we build our function using a `lambda expression` getting the `.Timestamp` of an element `s` (that is, the individual values included in `timestamp`). The output of line 49 indicates that, after the transformation, the column `timestamp` contains `time quantities`.

The remainder of the snippet illustrates how to extract the individual attributes of a `.Timestamp` object (see lines 64 - 74) and create a `.Timedelta` object (see line 93).

Snippet 6.11 — working with Timestamp and Timedelta objects

```

1 # import pandas with the socially accepted alias 'pd'
2 >>> import pandas as pd
3
4 # fake product review data
5 >>> df = pd.DataFrame.from_dict(
6 ...     {
7 ...         "timestamp": [
8 ...             "2017-10-08 12:03:05",
9 ...             "2020-09-07 08:09:45",
10 ...             "2021-04-11 10:12:13",
11 ...         ],
12 ...         "product": [
13 ...             "Darth Vader plush",
14 ...             "Obi-Wan Kenobi lightsaber",
15 ...             "Yoda pyjamas",
16 ...         ],
17 ...         "reviewer": ["Sheldon", "Sheldon", "Leonard"],
18 ...         "rating": [1, 2, 5],
19 ...     }
20 ... )
21
22 # data view
23 >>> df.head()
24          timestamp          product reviewer  rating
25 0  2017-10-08 12:03:05  Darth Vader plush  Sheldon      1
26 1  2020-09-07 08:09:45  Obi-Wan Kenobi lightsaber  Sheldon      2
27 2  2021-04-11 10:12:13        Yoda pyjamas  Leonard      5
28
29 # get basic info
30 >>> df.info()
```

```

31 <class 'pandas.DataFrame'>
32 RangeIndex: 3 entries, 0 to 2
33 Data columns (total 4 columns):
34 #   Column      Non-Null Count  Dtype
35 ---  ---
36 0   timestamp    3 non-null      str
37 1   product      3 non-null      str
38 2   reviewer     3 non-null      str
39 3   rating       3 non-null      int64
40 dtypes: int64(1), str(3)
41 memory usage: 228.0 bytes
42
43 # convert 'timestamp' from string to Timestamp type. NOTE the plain []
44 # assignment: .loc writes INTO the existing column and so must respect its
45 # dtype, and on Pandas 3 pushing timestamps into a str column raises
46 # TypeError. pd.to_datetime is also vectorised, so it beats .apply
47 >>> df["timestamp"] = pd.to_datetime(df["timestamp"])
48 # info shows that timestamp has datetime dtype
49 >>> df.info()
50 <class 'pandas.DataFrame'>
51 RangeIndex: 3 entries, 0 to 2
52 Data columns (total 4 columns):
53 #   Column      Non-Null Count  Dtype
54 ---  ---
55 0   timestamp    3 non-null      datetime64[us]
56 1   product      3 non-null      str
57 2   reviewer     3 non-null      str
58 3   rating       3 non-null      int64
59 dtypes: datetime64[us](1), int64(1), str(2)
60 memory usage: 228.0 bytes
61
62 # extract the timestamp components and assign them to new columns
63 # --+ year
64 >>> df["year"] = df["timestamp"].dt.year
65 # --+ month
66 >>> df["month"] = df["timestamp"].dt.month
67 # --+ day
68 >>> df["day"] = df["timestamp"].dt.day
69 # --+ hour
70 >>> df["hour"] = df["timestamp"].dt.hour
71 # --+ minute
72 >>> df["minute"] = df["timestamp"].dt.minute
73 # --+ second
74 >>> df["second"] = df["timestamp"].dt.second
75 # --+ data view
76 >>> df
77      timestamp      product reviewer  rating  year  month  day \
78 0 2017-10-08 12:03:05  Darth Vader plush  Sheldon      1  2017     10     8
79 1 2020-09-07 08:09:45  Obi-Wan Kenobi lightsaber  Sheldon      2  2020      9     7
80 2 2021-04-11 10:12:13      Yoda pyjamas  Leonard      5  2021      4    11
81
82      hour  minute  second
83 0       12        3        5

```

```

84 1      8      9      45
85 2     10     12     13
86
87 # calculate the time elapsed between the product launch and the review
88 # --+ fake product launch timestamps
89 >>> df["launch"] = pd.to_datetime(
90 ...     ["2011-11-01 08:45:19", "2012-02-07 13:07:07", "2011-05-10 13:04:05"]
91 ...     )
92 # --+ here is a Timedelta object
93 >>> df["deltat"] = df["timestamp"] - df["launch"]
94 # --+ data view
95 >>> df
96          timestamp          product reviewer  rating  year  month  day \
97 0 2017-10-08 12:03:05    Darth Vader plush  Sheldon      1  2017     10     8
98 1 2020-09-07 08:09:45  Obi-Wan Kenobi lightsaber  Sheldon      2  2020      9     7
99 2 2021-04-11 10:12:13          Yoda pyjamas  Leonard      5  2021      4    11
100
101      hour  minute  second          launch          deltat
102 0      12       3        5 2011-11-01 08:45:19 2168 days 03:17:46
103 1       8       9       45 2012-02-07 13:07:07 3134 days 19:02:38
104 2      10      12      13 2011-05-10 13:04:05 3623 days 21:08:08
105 # --+ data info
106 >>> df.info()
107 <class 'pandas.DataFrame'>
108 RangeIndex: 3 entries, 0 to 2
109 Data columns (total 12 columns):
110 #   Column      Non-Null Count  Dtype
111 ---  ---
112 0   timestamp    3 non-null      datetime64[us]
113 1   product      3 non-null      str
114 2   reviewer     3 non-null      str
115 3   rating       3 non-null      int64
116 4   year         3 non-null      int32
117 5   month        3 non-null      int32
118 6   day          3 non-null      int32
119 7   hour         3 non-null      int32
120 8   minute       3 non-null      int32
121 9   second       3 non-null      int32
122 10  launch       3 non-null      datetime64[us]
123 11  deltat       3 non-null      timedelta64[us]
124 dtypes: datetime64[us](2), int32(6), int64(1), str(2), timedelta64[us](1)
125 memory usage: 348.0 bytes

```


Can you give me an example of a `.Period` object?

`.Period` takes a string as input and returns an object of class `Period`, which has many attributes to establish the unit of time a timestamp falls on. For example, it is possible to assess a period's associated 'day' (see Snippet 6.12, line 12), 'day of the week' (line 16), 'month' (line 20), 'quarter' (line 24), or 'week of the year' (line 28).

Snippet 6.12 — working with Period objects

```

1 # import pandas with the socially acceptable alias pd
2 >>> import pandas as pd
3
4 # a timestamp as a string
5 >>> s = "2011-11-01 23:17:01"
6
7 # get a period object
8 >>> p = pd.Period(s, freq="s")
9
10 # extract sample information from p
11 # --+ calendar day
12 >>> p.day
13 1
14
15 # --+ day of the week (Monday is 0)
16 >>> p.dayofweek
17 1
18
19 # --+ month
20 >>> p.month
21 11
22
23 # --+ quarter
24 >>> p.quarter
25 4
26
27 # --+ week of the year (ranging from 0 to 59)
28 >>> p.weekofyear
29 44

```

5.9 Shaping and Reshaping DataFrames

What is 'panel data'?

Often, we work with panel data, wherein the same statistical unit (e.g., an employer, a company, an administrative city) is observed multiple times. An example is a security's price taken at different times of the day.

What are the alternative ways for representing panel data?

Mainly, there are two approaches to representing a panel dataset:

- The *wide* data structure consists of creating one row per unit and as many columns as measurement occasions. Figure 5.2 provides a pictorial representation of the wide data structure
- The *long* data structure consists of creating pairs of measurement occasions and values nested within the same unit. In other words, the same unit is repeated across as many rows as measurement occasions. Figure 5.3 provides a pictorial representation of the long data structure

Case	p_0	p_1	$\dots$	p_i	$\dots$	p_k
1	$p_{1,0}$	$p_{1,1}$	$\dots$	$p_{1,i}$	$\dots$	$p_{1,k}$
2	$p_{2,0}$	$p_{2,1}$	$\dots$	$p_{2,i}$	$\dots$	$p_{2,k}$
j	$p_{j,0}$	$p_{j,1}$	$\dots$	$p_{j,i}$	$\dots$	$p_{j,k}$
n	$p_{n,1}$	$p_{n,2}$	$\dots$	$p_{n,j}$	$\dots$	$p_{n,k}$

Figure 5.2: A stylized representation of a ‘wide’ data structure

What are the pros and cons of a wide data structure?

- Pros:
 - easier to work with for people coming from a spreadsheet environment
 - going through a row, one can make sense of the trend of a particular unit
- Cons:
 - the larger the number of measurement of occasions, the less readable the file, the larger the number of columns ($\rightarrow$ the more difficult to work with the data)
 - data manipulation activities must carry out column-by-column

What are the pros and cons of a wide data structure?

- Pros:
 - data manipulation activities are easier to carry out since there is only one column per case
 - it is easier to group and aggregate the data around cases
- Cons:
 - it is harder to inspect visually the data
 - this data structure is less familiar to people without information systems or programming background

Case	Time	Value
1	0	$p_{1,0}$
1	1	$p_{1,1}$
1	2	$p_{1,2}$
...	...	...
1	i	$p_{1,i}$
...	...	...
1	k	$p_{1,k}$
2	0	$p_{2,0}$
2	1	$p_{2,1}$
2	2	$p_{2,2}$
...	...	...
2	i	$p_{2,i}$
...	...	...
2	k	$p_{2,k}$
j	0	$p_{j,0}$
j	1	$p_{j,1}$
j	2	$p_{j,2}$
...	...	...
j	i	$p_{j,i}$
...	...	...
j	k	$p_{j,k}$
n	0	$p_{n,0}$
n	1	$p_{n,1}$
n	2	$p_{n,2}$
...	...	...
n	i	$p_{n,i}$
...	...	...
n	k	$p_{n,k}$

Figure 5.3: A stylized representation of a ‘long’ data structure

How can I move from a wide to a long data structure or the other way around?

To get a long data structure, one may want to use the Pandas function `pd.melt()`, and, to achieve a wide structure, the Pandas function `pd.pivot()`.

Per Snippet 6.13, typically we pass four inputs to `pd.melt()` (see line 16):

- **frame:** the source DataFrame to re-arrange
- **id_vars:** the column(s) that will be used to identify the cases
- **var_name:** the name of the target DataFrame column that will contain the information regarding the measurement occasions (i.e., the time)
- **value_name:** the name of the source DataFrame column containing the values to re-arrange

Snippet 6.13 also passes from a long to a wide data structure. In line 53, we pass the following inputs to `pd.pivot()`:

- **data:** the source DataFrame to re-arrange
- **index:** the column(s) that will be used to identify the cases
- **columns:** the name of the source DataFrame column(s) whose categories will be used to create the columns of the target DataFrame
- **values:** the name of the source DataFrame column containing the values to re-arrange

Snippet 6.13 — from wide data to long and back

```

1 # import pandas with the socially accepted alias 'pd'
2 >>> import pandas as pd
3
4 # create a wide data frame
5 >>> df = pd.DataFrame.from_dict(
6 ...     {
7 ...         "product": ["a", "b", "c"],
8 ...         "t0sales": [10, 5, 0],
9 ...         "t1sales": [7, 6, 5],
10 ...        "t2sales": [4, 6, 9],
11 ...     }
12 ... )
13
14 # get a long data frame
15 # --+ let us use the melt method
16 >>> molten = pd.melt(

```

```

17 ...     frame=df, id_vars=["product"], var_name="time", value_name="sales"
18 ...     )
19 # --+ the result is a long data frame
20 >>> molten
21   product    time  sales
22 0        a  t0sales    10
23 1        b  t0sales     5
24 2        c  t0sales     0
25 3        a  t1sales     7
26 4        b  t1sales     6
27 5        c  t1sales     5
28 6        a  t2sales     4
29 7        b  t2sales     6
30 8        c  t2sales     9
31 # --+ the time column is not clean, though. Let us clean it
32 # ----+ get rid of non-numeric characters. NOTE regex=True: since Pandas 2.0
33 #       .str.replace treats its argument as a literal string by default, so
34 #       the alternation "t|sales" would otherwise match nothing at all
35 >>> molten["time"] = molten["time"].str.replace("t|sales", "", regex=True)
36 # ----+ convert the time column to an integer. NOTE the plain [] assignment
37 #       rather than .loc -- see Snippet 6.8
38 >>> molten["time"] = molten["time"].astype(int)
39 # ----+ get info
40 >>> molten.info()
41 <class 'pandas.DataFrame'>
42 RangeIndex: 9 entries, 0 to 8
43 Data columns (total 3 columns):
44 #   Column   Non-Null Count  Dtype
45 ---  -
46 0   product  9 non-null      str
47 1   time     9 non-null      int64
48 2   sales    9 non-null      int64
49 dtypes: int64(2), str(1)
50 memory usage: 348.0 bytes
51 # ----+ data view
52 >>> molten
53   product  time  sales
54 0        a     0     10
55 1        b     0      5
56 2        c     0      0
57 3        a     1      7
58 4        b     1      6
59 5        c     1      5
60 6        a     2      4
61 7        b     2      6
62 8        c     2      9
63
64 # from long data to wide
65 # NOTE pd.pivot_table AGGREGATES, and its default aggfunc is "mean", so the
66 # result is float even when every cell holds a single integer. Use pd.pivot
67 # for a pure reshape that keeps the integers
68 >>> wide = pd.pivot_table(
69 ...     data=molten, index="product", columns="time", values="sales"

```

```
70 ... )
71 >>> wide
72 time          0    1    2
73 product
74 a          10.0  7.0  4.0
75 b           5.0  6.0  6.0
76 c           0.0  5.0  9.0
77 >>> pd.pivot(molten, index="product", columns="time", values="sales")
78 time          0  1  2
79 product
80 a           10  7  4
81 b            5  6  6
82 c            0  5  9
```

5.10 Group By Part I: Data Aggregation

Long data re-crossing the road?

So what?

Got it. How can I do this in Pandas?

As we saw in the previous section (5.9), long data present a ‘nested structure’ since values cluster around cases.

The nested structure of the data plays a key role in aggregating the data. For example, one may want to aggregate daily sales data into monthly or quarterly sales data. In so doing, one trades off the granularity of the data (e.g., daily sales records) for the possibility to best appreciate a general trend in the data (e.g., comparing the sales of two contiguous quarters).

Pandas has a class to help with this task, namely, `.groupby`. Mainly, this class captures the nested structure of the data and allows to aggregate or transform the data (see next section 5.11) according to the nested structure.

Snippet 6.14 shows how to use `.groupby` for aggregation purposes. The sample DataFrame contains monthly sales data for three products, each observed for three months (line 14). First, we must tell Pandas what grouping data to consider. We do that in line 27, wherein we pass the column name used to group the data. In this example, we use the `product` column. That means we will pass from product-time data points (i.e., monthly data) to product data points (e.g., the total sales for each product over the three months). Since the grouping structure is defined by one column only, the input is a string with the column’s name. An array of strings must be passed if the grouping structure is based on multiple columns.

As shown in line 27, calling the object `gr` does not affect the data. Instead, the signature of the object is displayed. To aggregate the data, we must deploy `gr` as shown in line 34. The string between brackets is the target column whose values we want to consider; `.aggregate()` defines the operation to carry out based on the grouping; the array of NumPy functions is the set of mathematical operations to perform as the data are aggregated. In our example, we ask for the total sales for each product and the average monthly sales over the three months (see line 36).

Snippet 6.14 — data aggregation with groupby

```

1 # import pandas with the socially accepted alias 'pd'
2 >>> import pandas as pd
3
4 # create a long data frame
5 >>> df = pd.DataFrame.from_dict(
6 ...     {
7 ...         "product": ['a', 'b', 'c', 'a', 'b', 'c', 'a', 'b', 'c'],
8 ...         "month": [0, 0, 0, 1, 1, 1, 2, 2, 2],
9 ...         "sales": [10, 5, 0, 7, 6, 5, 4, 6, 9]
10 ...     }
11 ... )
12
13 # data view
14 >>> df
15   product  month  sales
16 0        a      0     10
17 1        b      0      5
18 2        c      0      0
19 3        a      1      7
20 4        b      1      6
21 5        c      1      5
22 6        a      2      4
23 7        b      2      6
24 8        c      2      9
25
26 # group by
27 >>> gr = df.groupby("product")
28 # --+ what do we get if we call gr?
29 >>> gr
30 <pandas.api.typing.DataFrameGroupBy object at 0x10ea28ec0>
31
32 # aggregate sales data around the grouping structure. NOTE the aggregation
33 # functions are named as STRINGS: passing the NumPy callables np.sum and
34 # np.mean was deprecated in Pandas 2.0
35 >>> gr["sales"].aggregate(["sum", "mean"])
36           sum      mean
37 product
38 a           21  7.000000
39 b           17  5.666667
40 c           14  4.666667

```


5.11 Group By Part II: Data Transformation

Aggregate Vs. transform

Once we have created a grouping structure, we can use it for aggregating the data (i.e., passing from monthly to yearly data points, see the previous section 5.10) as well as for expanding the source data. This procedure is called transformation and does not affect the number of rows in the DataFrame.

In Snippet 6.15, we get the cumulative sum of a product's sales for each period (the output of line 27 shows the structure of the data). Having defined the grouping structure (line 41), we use the `.transform()` method to apply NumPy's `cumsum` function to the values of `sales` that regard the same product. As shown in line 46, the outcome of the transformation is assigned to the new column `cumulative_size`.

Snippet 6.15 — data transformation with groupby

```

1 # import the pandas library with the socially accepted alias 'pd'
2 >>> import pandas as pd
3
4 # the data frame
5 >>> df = pd.DataFrame.from_dict(
6 ...     {
7 ...         "product": ['a', 'b', 'c', 'a', 'b', 'c', 'a', 'b', 'c'],
8 ...         "month": [0, 0, 0, 1, 1, 1, 2, 2, 2],
9 ...         "sales": [10, 5, 0, 7, 6, 5, 4, 6, 9]
10 ...     }
11 ... )
12
13 # data view
14 >>> df
15    product  month  sales
16 0        a      0     10
17 1        b      0      5
18 2        c      0      0
19 3        a      1      7
20 4        b      1      6
21 5        c      1      5
22 6        a      2      4
23 7        b      2      6
24 8        c      2      9
25
26 # sort the data by product and month values
27 >>> df.sort_values(by=["product", "month"], inplace=True)
28 >>> df
29    product  month  sales
30 0        a      0     10
31 3        a      1      7
32 6        a      2      4
33 1        b      0      5

```

```

34 4      b      1      6
35 7      b      2      6
36 2      c      0      0
37 5      c      1      5
38 8      c      2      9
39
40 # group by
41 >>> gr = df.groupby("product")
42
43 # transform the sales data: unlike aggregate, transform returns a result
44 # the same length as the input, so it can go straight back into the frame.
45 # NOTE the string "cumsum" rather than the NumPy callable np.cumsum, which
46 # was deprecated in Pandas 2.0
47 >>> df["cumulative_sales"] = gr["sales"].transform("cumsum")
48 >>> df
49   product  month  sales  cumulative_sales
50 0      a      0     10                10
51 3      a      1      7                17
52 6      a      2      4                21
53 1      b      0      5                 5
54 4      b      1      6                11
55 7      b      2      6                17
56 2      c      0      0                 0
57 5      c      1      5                 5
58 8      c      2      9                14

```

5.12 Working with Multiple DataFrames

My data cut across data tables:
how can I combine multiple
them into a single DataFrame?

Often, the information we want to analyze is spread across different data tables. In this case, we must combine multiple DataFrame objects. Mainly, there are two approaches to data combination:

- Concatenate consists of stacking two or more data frames (either on the first axis or on the second axis)
- Merge consists of joining two or more data frames on a common set of keys

How do I concatenate two DataFrame objects in Pandas?

The `.concat()` method is used to concatenate two or more data frames. The method takes an array of DataFrames as input and returns a new data frame with the rows or columns from all the input data frames.

In Snippet 6.15, we carry out the following:

- We concatenate two DataFrame objects along the first axis (column-wise)
- We concatenate two DataFrame objects along the second axis (row-wise)

concatenate two data frames. The first data frame contains sales data for the year's first quarter; the second data frame includes the information for the second quarter.

Snippet 6.16 — concatenating two DataFrame objects

```

1 # import pandas with the socially accepted alias pd
2 >>> import pandas as pd
3
4 # concatenate along the first axis (axis = 0)
5 # --+ the data frames
6 >>> df0 = pd.DataFrame.from_dict(
7 ...     {"quarter": ["first", "second"], "sales": [10, 30]}
8 ...     )
9 >>> df1 = pd.DataFrame.from_dict(
10 ...     {"quarter": ["third", "fourth"], "sales": [10, 40]}
11 ...     )
12 # --+ set the indices to match the data frames
13 >>> df0.set_index("quarter", inplace=True)
14 >>> df1.set_index("quarter", inplace=True)
15 # --+ concatenate the data frames
16 >>> pd.concat([df0, df1], axis=0)
17
18     sales
19 quarter
19 first      10
20 second     30
21 third      10
22 fourth     40
23
24 # concatenate along the second axis (axis = 1)
25 # --+ the data frames
26 >>> df0 = pd.DataFrame.from_dict(
27 ...     {"product": [1, 2, 3], "sales": [10, 30, 20]}
28 ...     )
29 >>> df1 = pd.DataFrame.from_dict(
30 ...     {"product": [1, 2, 3], "units_sold": [3, 6, 2]}
31 ...     )
32 # --+ set the indices to match the data frames
33 >>> df0.set_index("product", inplace=True)

```

```

34 >>> df1.set_index("product", inplace=True)
35 # --+ the concatenated data frame
36 >>> pd.concat([df0, df1], axis=1)
37      sales  units_sold
38 product
39 1         10          3
40 2         30          6
41 3         20          2

```

How do I merge two DataFrame objects in Pandas?

The `.merge()` method is used to merge two or more data frames. The method takes the following inputs:

- `left` and `right` are the DataFrame objects to be merged
- `on` is the column/list of columns on which the merge is to be performed
- `how` is the type of merge to be performed
 - `how="left"` — all cases from the left DataFrame are preserved irrespective of the fact they are present in the right DataFrame
 - `how="right"` — all cases from the right DataFrame are preserved irrespective of the fact they are present in the left DataFrame
 - `how = "inner"` — only the cases that are present in both the left and right DataFrame are preserved
 - `how = "outer"` — all cases from the left and right DataFrame are preserved irrespective of the fact they are present in the left or right DataFrame
- Optionally, one may want to set the parameters `left_index` and `right_index` to `True` in order to perform the merge on the indices of the DataFrames (default is `False`.)

Shaping and Reshaping DataFrames

```

1 # import pandas with the socially accepted alias 'pd'
2 >>> import pandas as pd
3
4 # the data
5 >>> df0 = pd.DataFrame.from_dict(
6 ...     {"product": [1, 2, 3], "sales": [10, 30, 20]}
7 ...     )
8 >>> df1 = pd.DataFrame.from_dict(
9 ...     {"product": [2, 3, 4], "margin": [0.13, 0.31, 0.21]}
10 ...     )
11
12 # merge on a shared column. NOTE the key is "product" -- a misspelt key
13 # raises KeyError rather than falling back to anything
14 >>> pd.merge(df0, df1, on="product", how="left")
15    product  sales  margin
16 0         1     10     NaN
17 1         2     30     0.13
18 2         3     20     0.31
19 >>> pd.merge(df0, df1, on="product", how="right")
20    product  sales  margin
21 0         2    30.0     0.13
22 1         3    20.0     0.31
23 2         4     NaN     0.21
24 >>> pd.merge(df0, df1, on="product", how="outer")
25    product  sales  margin
26 0         1    10.0     NaN
27 1         2    30.0     0.13
28 2         3    20.0     0.31
29 3         4     NaN     0.21
30 >>> pd.merge(df0, df1, on="product", how="inner")
31    product  sales  margin
32 0         2     30     0.13
33 1         3     20     0.31
34
35 # merge on the index instead
36 >>> df0.set_index("product", inplace=True)
37 >>> df1.set_index("product", inplace=True)
38 >>> pd.merge(df0, df1, left_index=True, right_index=True)
39    sales  margin
40 product
41 2         30     0.13
42 3         20     0.31

```

5.13 File Input and Output (IO) with Pandas

What type of data formats can I read/write with Pandas

There are plenty of options, including proprietary statistical languages such as SAS, SPSS, and Stata, Python's pickles, tabular data in CSV or TSV format, and data formats widely adopted online data sources, such as HTML, XML, and JSON. Table 5.2, 5.3, 5.4, 5.5 illustrate some of the most popular IO utilities.²⁸

TABLE 5.2
Routines for Reading and Writing Data with Pandas: Pickles

Routine	Synopsis
<code>pd.read_pickle(filepath_or_buffer[, ...])</code>	Load pickled pandas object (or any object) from file
<code>df.to_pickle(path[, compression, ...])</code>	Pickle (serialize) object to file

Notes: the statements included in the ‘Routine’ column assume Pandas is loaded with the `pd` alias and there is a `DataFrame` loaded with name `df`.

TABLE 5.3
Routines for Reading and Writing Data with Pandas: Excel Spreadsheets

Routine	Synopsis
<code>pd.read_excel(io[, sheet_name, header, names, ...])</code>	Write object to an Excel sheet
Read an Excel file into a pandas DataFrame. <code>df.to_excel(excel_writer[, ...])</code>	Parse specified sheet(s) into a DataFrame
<code>pd.ExcelFile.parse([sheet_name, header, names, ...])</code>	Write Styler to an Excel sheet
<code>df.Styler.to_excel(excel_writer[, sheet_name, ...])</code>	Class for writing DataFrame objects into excel sheets
<code>df.ExcelWriter(path[, engine, date_format, ...])</code>	

Notes: the statements included in the ‘Routine’ column assume Pandas is loaded with the `pd` alias and there is a DataFrame loaded with name `df`.

TABLE 5.4
Routines for Reading and Writing Data with Pandas: JSON Files

Routine	Synopsis
<code>pd.read_json([path_or_buf, orient, typ, dtype, ...])</code>	Convert a JSON string to pandas object
<code>pd.json_normalize(data[, record_path, meta, ...])</code>	Normalize semi-structured JSON data into a flat table
<code>df.to_json([path_or_buf, orient, ...])</code>	Convert the object to a JSON string
<code>df.build_table_schema(data[, index, ...])</code>	Create a Table schema from data

Notes: the statements included in the ‘Routine’ column assume Pandas is loaded with the `pd` alias and there is a DataFrame loaded with name `df`.

TABLE 5.5
Routines for Reading and Writing Data with Pandas: Flat Files

Routine	Synopsis
<code>pd.read_table(filepath_or_buffer[, sep, ...])</code>	Read general delimited file into DataFrame
<code>pd.read_csv(filepath_or_buffer[, sep, ...])</code>	Read a comma-separated values (csv) file into DataFrame
<code>df.to_csv([path_or_buf, sep, na_rep, ...])</code>	Write object to a comma-separated values (csv) file
<code>pd.read_fwf(filepath_or_buffer[, colspecs, ...])</code>	Read a table of fixed-width formatted lines into DataFrame

Notes: the statements included in the ‘Routine’ column assume Pandas is loaded with the `pd` alias and there is a DataFrame loaded with name `df`.

Notes

²¹The set of competing frameworks include Python’s [PyTables](#) and [Dask](#), as well as R’s [data.tables](#) and Julia’s [DataFrames](#).

²²A time series is a set of points arranged in chronological order.

²³Panel data, widely adopted in economics and finance, are multidimensional datasets wherein the same statistical units are repeatedly observed over time.

²⁴Students interested in the origins of Pandas may want to read article on/interview with Wes McKinney “[Meet the man behind the most important tool in data science.](#)”

²⁵Pandas’ [API](#) is well-documented and well-tested. Also, the [user guide](#) has plenty of examples showing Panda’s features in action.

²⁶In field of information systems and computer science, columns are often called fields; statisticians, economists, and analysts in general use the term variable to refer to a column; the term feature is common in the Machine Learning field.

²⁷throughout the book, I refer to Python sessions without loss of generality. The points I make are valid also for IPython or Jupyter sessions.

²⁸For the complete list of Pandas IO routines, please refer to <https://pandas.pydata.org/docs/reference/io.html>.

Chapter 6

Coda

In terms of *scope*, these notes emphasized the application of the Python Programming Language to ‘data science’ tasks. Mainly, we went through the basics of the language and considered two core modules: NumPy — focusing on technical and scientific computation — and Pandas — the *de facto* industry standard for data management.

In terms of *process*, learning the Python programming language takes months, if not years. However, it is possible to acquire a minimum understanding of Python in a few weeks. To do that, you should resist the temptation to search the web for the ultimate Python tutorial or off-the-shelf snippets that address your coding issues. Instead, you may want to spend time reading and digesting these notes offline. Why? Two arguments. First, learning a programming language requires scrutinizing and reasoning with coding snippets. Borrowing others’ solutions helps to solve a problem but does not facilitate learning. Second, efficiency: the snippets reported in these notes have been selected to address the most common coding problems encountered in four previous Bayes Business Analytics MSc cohorts. So, you may not want to waste time on materials not relevant to the various Python-based modules offered at Bayes.

Finally, if you are a programming newbie, there will be times when you will find yourself asking questions such as: “What is the best way to do this?” or “How can I do this in Python?” Do not get frustrated — that is an integral part of the learning process!! Instead, keep calm and reach out to the lecturer and fellow students to share your thoughts and clarify your doubts.

Appendices

Appendix A

Cheat Sheets

A.1 Escapes

TABLE A.1
Helpful Escapes

Escape	Meaning
\\	Backslash (stores one \)
\'	Single quotes escape (stores ')
\"	Double quotes escape (stores ")
\a	Bell
\b	Backspace
\f	Formfeed
\n	Newline
\r	Carriage return
\t	Horizontal tab
\v	Vertical tab

A.2 String Methods

TABLE A.2
Comprehensive List of String Methods

<i>Cases I</i>	
<code>s.capitalize()</code>	Capitalize s # 'hello' => 'Hello'
<code>s.lower()</code>	Lowercase s # 'HELLO' => 'hello'
<code>s.swapcase()</code>	Swap cases of all characters in s # 'Hello' => 'hELLO'
<code>s.title()</code>	Titlecase s # 'hello world' => 'Hello World'
<code>s.upper()</code>	Uppercase s # 'hello' => 'HELLO'
<i>Sequence Operations I</i>	
<code>s2 in s</code>	Return true if s contains s2
<code>s + s2</code>	Concat s and s2
<code>len(s)</code>	Length of s
<code>min(s)</code>	Smallest character of s
<code>max(s)</code>	Largest character of s
<i>Sequence Operations II</i>	
<code>s2 not in s</code>	Return true if s does not contain s2
<code>s * integer</code>	Return integer copies of s concatenated # 'hello' => 'hellohellohello'
<code>s[index]</code>	Character at index of s
<code>s[i:j:k]</code>	Slice of s from i to j with step k
<code>s.count(s2)</code>	Count of s2 in s
<i>Whitespace I</i>	
<code>s.center(width)</code>	Center s with blank padding of width # 'hi' => ' hi '
<code>s.isspace()</code>	Return true if s only contains whitespace characters
<code>s.ljust(width)</code>	Left justify s with total size of width # 'hello' => 'hello '
<code>s.rjust(width)</code>	Right justify s with total size of width # 'hello' => ' hello'
<code>s.strip()</code>	Remove leading and trailing whitespace from s # ' hello ' => 'hello'
<i>Find / Replace I</i>	
<code>s.index(s2, i, j)</code>	Index of first occurrence of s2 in s after index i and before index j
<code>s.find(s2)</code>	Find and return lowest index of s2 in s
<code>s.index(s2)</code>	Return lowest index of s2 in s (but raise ValueError if not found)
<code>s.replace(s2, s3)</code>	Replace s2 with s3 in s
<code>s.replace(s2, s3, count)</code>	Replace s2 with s3 in s at most count times
<code>s.rfind(s2)</code>	Return highest index of s2 in s
<code>s.rindex(s2)</code>	Return highest index of s2 in s (raise ValueError if not found)
<i>Cases II</i>	
<code>s.casefold()</code>	Casefold s (aggressive lowercasing for caseless matching) # 'ßorat' => 'ssorat'
<code>s.islower()</code>	Return true if s is lowercase
<code>s.istitle()</code>	Return true if s is titlecased # 'Hello World' => true
<code>s.isupper()</code>	Return true if s is uppercase

TABLE A.2
(Cont'ed)

<i>Inspection I</i>	
<code>s.endswith(s2)</code>	Return true if s ends with s2
<code>s.isalnum()</code>	Return true if s is alphanumeric
<code>s.isalpha()</code>	Return true if s is alphabetic
<code>s.isdecimal()</code>	Return true if s is decimal
<code>s.isnumeric()</code>	Return true if s is numeric
<code>s.startswith(s2)</code>	Return true is s starts with s2
<i>Splitting I</i>	
<code>s.join('123')</code>	Return s joined by iterable '123' # 'hello' => '1hello2hello3'
<code>s.partition(sep)</code>	Partition string at sep and return 3-tuple with part before, the sep itself, and part after # 'hello' => ('he', 'l', 'lo')
<code>s.rpartition(sep)</code>	Partition string at last occurrence of sep, return 3-tuple with part before, the sep, and part after # 'hello' => ('hel', 'l', 'o')
<code>s.rsplit(sep, maxsplit)</code>	Return list of s split by sep with rightmost maxsplits performed
<code>s.split(sep, maxsplit)</code>	Return list of s split by sep with leftmost maxsplits performed
<code>s.splitlines()</code>	Return a list of lines in s # 'hello\nworld' => ['hello', 'world']
<i>Inspection II</i>	
<code>s[i:j]</code>	Slice of s from i to j
<code>s.endswith((s1, s2, s3))</code>	Return true if s ends with any of string tuple s1, s2, and s3
<code>s.isdigit()</code>	Return true if s is digit
<code>s.isidentifier()</code>	Return true if s is a valid identifier
<code>s.isprintable()</code>	Return true is s is printable
<i>Whitespace II</i>	
<code>s.center(width, pad)</code>	Center s with padding pad of width # 'hi' => 'padpadhipadpad'
<code>s.expandtabs(integer)</code>	Replace all tabs with spaces of tabsize integer # 'hello\tworld' => 'hello world'
<code>s.lstrip()</code>	Remove leading whitespace from s # ' hello ' => 'hello '
<code>s.rstrip()</code>	Remove trailing whitespace from s # ' hello ' => 'hello'
<code>s.zfill(width)</code>	Left fill s with ASCII '0' digits with total length width # '42' => '00042'

A.3 NumPy Array Manipulation Routines

TABLE A.3
NumPy Universal Functions: Mathematical Operations

Universal Function	Synopsis
<code>add(x1, x2, /[, out, where, casting, order, ...])</code>	Add arguments element-wise
<code>subtract(x1, x2, /[, out, where, casting, ...])</code>	Subtract arguments, element-wise
<code>multiply(x1, x2, /[, out, where, casting, ...])</code>	Multiply arguments element-wise
<code>matmul(x1, x2, /[, out, casting, order, ...])</code>	Matrix product of two arrays
<code>divide(x1, x2, /[, out, where, casting, ...])</code>	Divide arguments element-wise
<code>logaddexp(x1, x2, /[, out, where, casting, ...])</code>	Logarithm of the sum of exponentiations of the inputs
<code>logaddexp2(x1, x2, /[, out, where, casting, ...])</code>	Logarithm of the sum of exponentiations of the inputs in base-2
<code>true_divide(x1, x2, /[, out, where, ...])</code>	Divide arguments element-wise
<code>floor_divide(x1, x2, /[, out, where, ...])</code>	Return the largest integer smaller or equal to the division of the inputs
<code>negative(x, /[, out, where, casting, order, ...])</code>	Numerical negative, element-wise
<code>positive(x, /[, out, where, casting, order, ...])</code>	Numerical positive, element-wise
<code>power(x1, x2, /[, out, where, casting, ...])</code>	First array elements raised to powers from second array, element-wise
<code>float.power(x1, x2, /[, out, where, ...])</code>	First array elements raised to powers from second array, element-wise
<code>remainder(x1, x2, /[, out, where, casting, ...])</code>	Returns the element-wise remainder of division
<code>mod(x1, x2, /[, out, where, casting, order, ...])</code>	Returns the element-wise remainder of division
<code>fmod(x1, x2, /[, out, where, casting, ...])</code>	Returns the element-wise remainder of division
<code>divmod(x1, x2, /[, out, where, casting, ...])</code>	Return element-wise quotient and remainder simultaneously
<code>absolute(x, /[, out, where, casting, order, ...])</code>	Calculate the absolute value element-wise
<code>fabs(x, /[, out, where, casting, order, ...])</code>	Compute the absolute values element-wise
<code>rint(x, /[, out, where, casting, order, ...])</code>	Round elements of the array to the nearest integer
<code>sign(x, /[, out, where, casting, order, ...])</code>	Returns an element-wise indication of the sign of a number
<code>heaviside(x1, x2, /[, out, where, casting, ...])</code>	Compute the Heaviside step function
<code>conj(x, /[, out, where, casting, order, ...])</code>	Return the complex conjugate, element-wise
<code>conjugate(x, /[, out, where, casting, ...])</code>	Return the complex conjugate, element-wise
<code>exp(x, /[, out, where, casting, order, ...])</code>	Calculate the exponential of all elements in the input array
<code>exp2(x, /[, out, where, casting, order, ...])</code>	Calculate 2^{**p} for all p in the input array
<code>log(x, /[, out, where, casting, order, ...])</code>	Natural logarithm, element-wise
<code>log2(x, /[, out, where, casting, order, ...])</code>	Base-2 logarithm of x
<code>log10(x, /[, out, where, casting, order, ...])</code>	Return the base 10 logarithm of the input array, element-wise
<code>expm1(x, /[, out, where, casting, order, ...])</code>	Calculate $\exp(x) - 1$ for all elements in the array
<code>sqrt(x, /[, out, where, casting, order, ...])</code>	Return the non-negative square-root of an array, element-wise

TABLE A.4
NumPy Universal Functions: Trigonometric Operations

Universal Function	Synopsis
<code>sin(x, /[, out, where, casting, order, ...])</code>	Trigonometric sine, element-wise
<code>cos(x, /[, out, where, casting, order, ...])</code>	Cosine element-wise
<code>tan(x, /[, out, where, casting, order, ...])</code>	Compute tangent element-wise
<code>arcsin(x, /[, out, where, casting, order, ...])</code>	Inverse sine, element-wise
<code>arccos(x, /[, out, where, casting, order, ...])</code>	Trigonometric inverse cosine, element-wise
<code>arctan(x, /[, out, where, casting, order, ...])</code>	Trigonometric inverse tangent, element-wise
<code>arctan2(x1, x2, /[, out, where, casting, ...])</code>	Element-wise arc tangent of x1/x2 choosing the quadrant correctly
<code>hypot(x1, x2, /[, out, where, casting, ...])</code>	Given the "legs" of a right triangle, return its hypotenuse
<code>sinh(x, /[, out, where, casting, order, ...])</code>	Hyperbolic sine, element-wise
<code>cosh(x, /[, out, where, casting, order, ...])</code>	Hyperbolic cosine, element-wise
<code>tanh(x, /[, out, where, casting, order, ...])</code>	Compute hyperbolic tangent element-wise
<code>arcsinh(x, /[, out, where, casting, order, ...])</code>	Inverse hyperbolic sine element-wise
<code>arccosh(x, /[, out, where, casting, order, ...])</code>	Inverse hyperbolic cosine, element-wise
<code>arctanh(x, /[, out, where, casting, order, ...])</code>	Inverse hyperbolic tangent element-wise
<code>degrees(x, /[, out, where, casting, order, ...])</code>	Convert angles from radians to degrees
<code>radians(x, /[, out, where, casting, order, ...])</code>	Convert angles from degrees to radians
<code>deg2rad(x, /[, out, where, casting, order, ...])</code>	Convert angles from degrees to radians
<code>rad2deg(x, /[, out, where, casting, order, ...])</code>	Convert angles from radians to degrees

TABLE A.5
NumPy Universal Functions: Floating Operations

Universal Function	Synopsis
<code>isfinite(x, /[, out, where, casting, order, ...])</code>	Test element-wise for finiteness (not infinity and not Not a Number)
<code>isinf(x, /[, out, where, casting, order, ...])</code>	Test element-wise for positive or negative infinity
<code>isnan(x, /[, out, where, casting, order, ...])</code>	Test element-wise for NaN and return the result as a boolean array
<code>isnat(x, /[, out, where, casting, order, ...])</code>	Test element-wise for NaT (not a time) and return the result as a boolean array
<code>fabs(x, /[, out, where, casting, order, ...])</code>	Compute the absolute values element-wise
<code>signbit(x, /[, out, where, casting, order, ...])</code>	Returns element-wise True where signbit is set (less than zero)
<code>copysign(x1, x2, /[, out, where, casting, ...])</code>	Change the sign of x1 to that of x2, element-wise
<code>nextafter(x1, x2, /[, out, where, casting, ...])</code>	Return the next floating-point value after x1 towards x2, element-wise
<code>spacing(x, /[, out, where, casting, order, ...])</code>	Return the distance between x and the nearest adjacent number
<code>modf(x[, out1, out2], /[, out, where, ...])</code>	Return the fractional and integral parts of an array, element-wise
<code>ldexp(x1, x2, /[, out, where, casting, ...])</code>	Returns $x1 * 2^{x2}$, element-wise
<code>frexp(x[, out1, out2], /[, out, where, ...])</code>	Decompose the elements of x into mantissa and twos exponent
<code>fmod(x1, x2, /[, out, where, casting, ...])</code>	Returns the element-wise remainder of division
<code>floor(x, /[, out, where, casting, order, ...])</code>	Return the floor of the input, element-wise
<code>ceil(x, /[, out, where, casting, order, ...])</code>	Return the ceiling of the input, element-wise
<code>trunc(x, /[, out, where, casting, order, ...])</code>	Return the truncated value of the input, element-wise

TABLE A.6
NumPy Universal Functions: Comparison Operations

Universal Function	Synopsis
<code>greater(x1, x2, /[, out, where, casting, ...])</code>	Return the truth value of (x1 $\geq$ x2) element-wise
<code>greater_equal(x1, x2, /[, out, where, ...])</code>	Return the truth value of (x1 $\geq$ x2) element-wise
<code>less(x1, x2, /[, out, where, casting, ...])</code>	Return the truth value of (x1 $<$ x2) element-wise
<code>less_equal(x1, x2, /[, out, where, casting, ...])</code>	Return the truth value of (x1 $\leq$ x2) element-wise
<code>not_equal(x1, x2, /[, out, where, casting, ...])</code>	Return (x1 $\neq$ x2) element-wise
<code>equal(x1, x2, /[, out, where, casting, ...])</code>	Return (x1 == x2) element-wise
<code>logical_and(x1, x2, /[, out, where, ...])</code>	Compute the truth value of x1 AND x2 element-wise
<code>logical_or(x1, x2, /[, out, where, casting, ...])</code>	Compute the truth value of x1 OR x2 element-wise
<code>logical_xor(x1, x2, /[, out, where, ...])</code>	Compute the truth value of x1 XOR x2, element-wise
<code>logical_not(x, /[, out, where, casting, ...])</code>	Compute the truth value of NOT x element-wise
<code>maximum(x1, x2, /[, out, where, casting, ...])</code>	Element-wise maximum of array elements
<code>minimum(x1, x2, /[, out, where, casting, ...])</code>	Element-wise minimum of array elements
<code>fmax(x1, x2, /[, out, where, casting, ...])</code>	Element-wise maximum of array elements
<code>fmin(x1, x2, /[, out, where, casting, ...])</code>	Element-wise minimum of array elements

TABLE A.7
NumPy Array Manipulation Routines

Routine	Synopsis
<i>Basic operations</i>	
<code>np.copyto(dst, src[, casting, where])</code> <code>np.shape(a)</code>	Copies values from one array to another, broadcasting as necessary Return the shape of an array
<i>Changing array shape</i>	
<code>np.reshape(a, newshape[, order])</code> <code>np.ravel(a[, order])</code> <code>np.ndarray.flat</code> <code>np.ndarray.flatten([order])</code>	Gives a new shape to an array without changing its data Return a contiguous flattened array A 1-D iterator over the array Return a copy of the array collapsed into one dimension
<i>Changing array shape</i>	
<code>np.moveaxis(a, source, destination)</code> <code>np.rollaxis(a, axis[, start])</code> <code>np.swapaxes(a, axis1, axis2)</code> <code>np.ndarray.T</code> <code>np.transpose(a[, axes])</code>	Move axes of an array to new position Roll the specified axis backwards, until it lies in a given position Interchange two axes of an array The transposed array Reverse or permute the axes of an array; returns the modified array
<i>Changing number of dimensions</i>	
<code>np.atleast_1d(*arys)</code> <code>np.atleast_2d(*arys)</code> <code>np.atleast_3d(*arys)</code> <code>np.broadcast</code> <code>np.broadcast_to(array, shape[, subok])</code> <code>np.broadcast_arrays(*args[, subok])</code> <code>np.expand_dims(a, axis)</code> <code>np.squeeze(a[, axis])</code>	Convert inputs to arrays with at least one dimension View inputs as arrays with at least two dimensions View inputs as arrays with at least three dimensions Produce an object that mimics broadcasting Broadcast an array to a new shape Broadcast any number of arrays against each other Expand the shape of an array Remove axes of length one from a

TABLE A.7 (cont'd)

<i>Changing kind of array</i>	
<code>np.asarray(a[, dtype, order, like])</code>	Convert the input to an array
<code>np.asanyarray(a[, dtype, order, like])</code>	Convert the input to an ndarray, but pass ndarray subclasses through
<code>np.asmatrix(data[, dtype])</code>	Interpret the input as a matrix
<code>np.asfarray(a[, dtype])</code>	Return an array converted to a float type
<code>np.asfortranarray(a[, dtype, like])</code>	Return an array (ndim ≥ 1) laid out in Fortran order in memory
<code>np.ascontiguousarray(a[, dtype, like])</code>	Return a contiguous array (ndim ≥ 1) in memory (C order)
<code>np.asarray_chkfinite(a[, dtype, order])</code>	Convert the input to an array, checking for NaNs or Infs
<code>np.require(a[, dtype, requirements, like])</code>	Return an ndarray of the provided type that satisfies requirements
<i>Joining arrays</i>	
<code>np.concatenate([axis, out, dtype, casting])</code>	Join a sequence of arrays along an existing axis
<code>np.stack(arrays[, axis, out])</code>	Join a sequence of arrays along a new axis
<code>np.block(arrays)</code>	Assemble an nd-array from nested lists of blocks
<code>np.vstack(tup)</code>	Stack arrays in sequence vertically (row wise)
<code>np.hstack(tup)</code>	Stack arrays in sequence horizontally (column wise)
<code>np.dstack(tup)</code>	Stack arrays in sequence depth wise (along the third axis)
<code>np.column_stack(tup)</code>	Stack 1-D arrays as columns into a 2-D array
<code>np.row_stack(tup)</code>	Stack arrays in sequence vertically (row wise)
<i>Splitting arrays</i>	
<code>np.split(ary, indices-or-sections[, axis])</code>	Split an array into multiple sub-arrays as views into array
<code>np.array_split(ary, indices-or-sections[, axis])</code>	Split an array into multiple sub-arrays
<code>np.dsplit(ary, indices-or-sections)</code>	Split array into multiple sub-arrays along the 3rd axis (depth)
<code>np.hsplit(ary, indices-or-sections)</code>	Split an array into multiple sub-arrays horizontally (column-wise)
<code>np.vsplit(ary, indices-or-sections)</code>	Split an array into multiple sub-arrays vertically (row-wise)

TABLE A.7 (cont'd)

<i>Tiling elements</i>	
<code>np.tile(A, reps)</code>	Construct an array by repeating A the number of times given by reps
<code>np.repeat(a, repeats[, axis])</code>	Repeat elements of an array
<i>Adding and removing elements</i>	
<code>np.delete(arr, obj[, axis])</code>	Return a new array with sub-arrays along an axis deleted
<code>np.insert(arr, obj, values[, axis])</code>	Insert values along the given axis before the given indices
<code>np.append(arr, values[, axis])</code>	Append values to the end of an array
<code>np.resize(a, new_shape)</code>	Return a new array with the specified shape
<code>np.trim_zeros(filt[, trim])</code>	Trim the leading and/or trailing zeros from a 1-D array or sequence
<code>np.unique(ar[, return_index, return_inverse, ...])</code>	Find the unique elements of an array
<i>Reranning elements</i>	
<code>np.flip(m[, axis])</code>	Reverse the order of elements in an array along the given axis
<code>np.fliplr(m)</code>	Reverse the order of elements along axis 1 (left/right)
<code>np.flipud(m)</code>	Reverse the order of elements along axis 0 (up/down)
<code>np.reshape(a, newshape[, order])</code>	Gives a new shape to an array without changing its data
<code>np.roll(a, shift[, axis])</code>	Roll array elements along a given axis
<code>np.rot90(m[, k, axes])</code>	Rotate an array by 90 degrees in the plane specified by axes

Notes: the statements included in the 'Routine' column assume NumPy is loaded with the `np` alias.

Appendix B

GitHub: The World-Leading Collaborative and Versioning Tool

B.1 GitHub in a Nuthshell

GitHub is one of the most prominent Internet hosting service for software development and version control using Git, the free and OSS for distributed version control. Mainly, GitHub allows to:

- Host and manage repositories containing source code
- Share a repository with other users, publicly or privately
- Track changes to the source code
- Collaborate with other users to develop the source code

B.2 GitHub in the Education Sector

Since the 2018/19 academic year, my Python-related teaching has been based on GitHub. According to recent research, GitHub has many distinctive features (see Figure B.1) that make it a valuable teaching tool in teaching quantitative business subjects. Particularly, Zagalsky and colleagues¹ identify the following benefits of using GitHub for education:

- Transparency of activities
- Encouraging learners' participation
- Industry relevance
- Ease of use
- Free academic licensing
- Shared spaces and course versioning

Considering the characteristics of many Bayes Business Analytics modules, I think GitHub offers several advantages. *First*, it allows lecturers to share a substantial volume of computer code efficiently and transparently without creating a sense of information overload for learners. Below is a quote regarding the module 'SMM638, Network Analytics' offered in the 2019-20 academic year, when I started to use GitHub in my teaching:

"I think GitHub is a very efficient platform for our study. It's mainly because our teacher always updates all information in advance, so we can easily find everything we want."

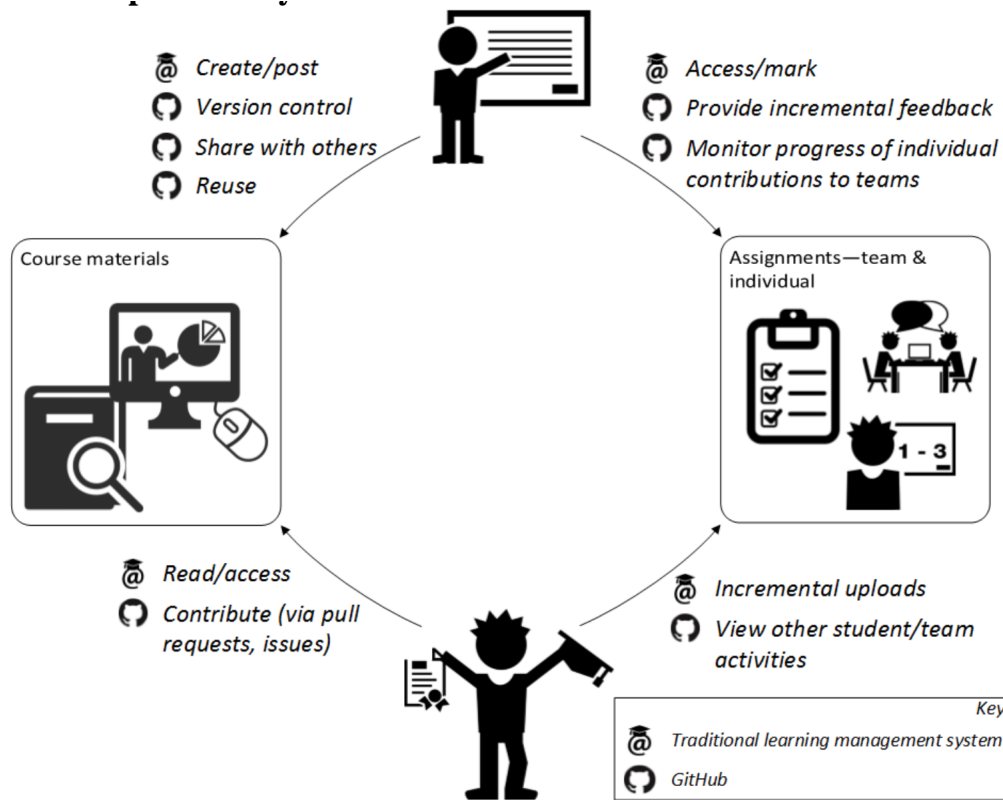


Figure B.1: The distinctive features of GitHub *vis a' vis* traditional LMS
Notes. — Source is Zagalsky et al. 2015.

Unlike a traditional Moodle page, GitHub has a very granular and public logging system and visualizations that highlight any change in the set of materials hosted in a repository (see, for example, Figure B.2). Furthermore, learners can decide how and to what extent to receive notifications on the changes affecting a GitHub repository.² Regarding ‘SMM694, Applied Natural Language Processing’ — delivered between May and July 2025 — 15 students out of 67 (22%) decided ‘to watch’ the GitHub repository hosting my teaching materials. Hence, they received real-time notifications on any edits of the 81 commits I made in the Summer Term;³ 43 students out of 67 (64%) added my GitHub repository to their library; 31 out of 67 (42%) reused my repository’s material for a new project.

Second, by engaging with GitHub, learners familiarise themselves with the industry-standard versioning software and collaborative platform. *Third*, GitHub is a piece of open-source and free software. As such, it presents no entry barriers for learners and can be deployed to sustain inclusive forms of teaching over and beyond the boundaries of individual academic institutions.⁴ According to the tracking system of GitHub, as of today, there are 154 projects (a.k.a., forks) that build on my teaching materials covering Python, network analytics, data visualization, and NLP. *Finally*, in the case of hybrid teaching, the public logging system of GitHub provides the group of face-to-face learners and the group of remote learners with equivalent information about the functioning of the module and teaching materials. In other words, GitHub can prevent the formation of information asymmetry that potentially penalize remote learners because of their lack of physical proximity to the lecturer and fellow learners.

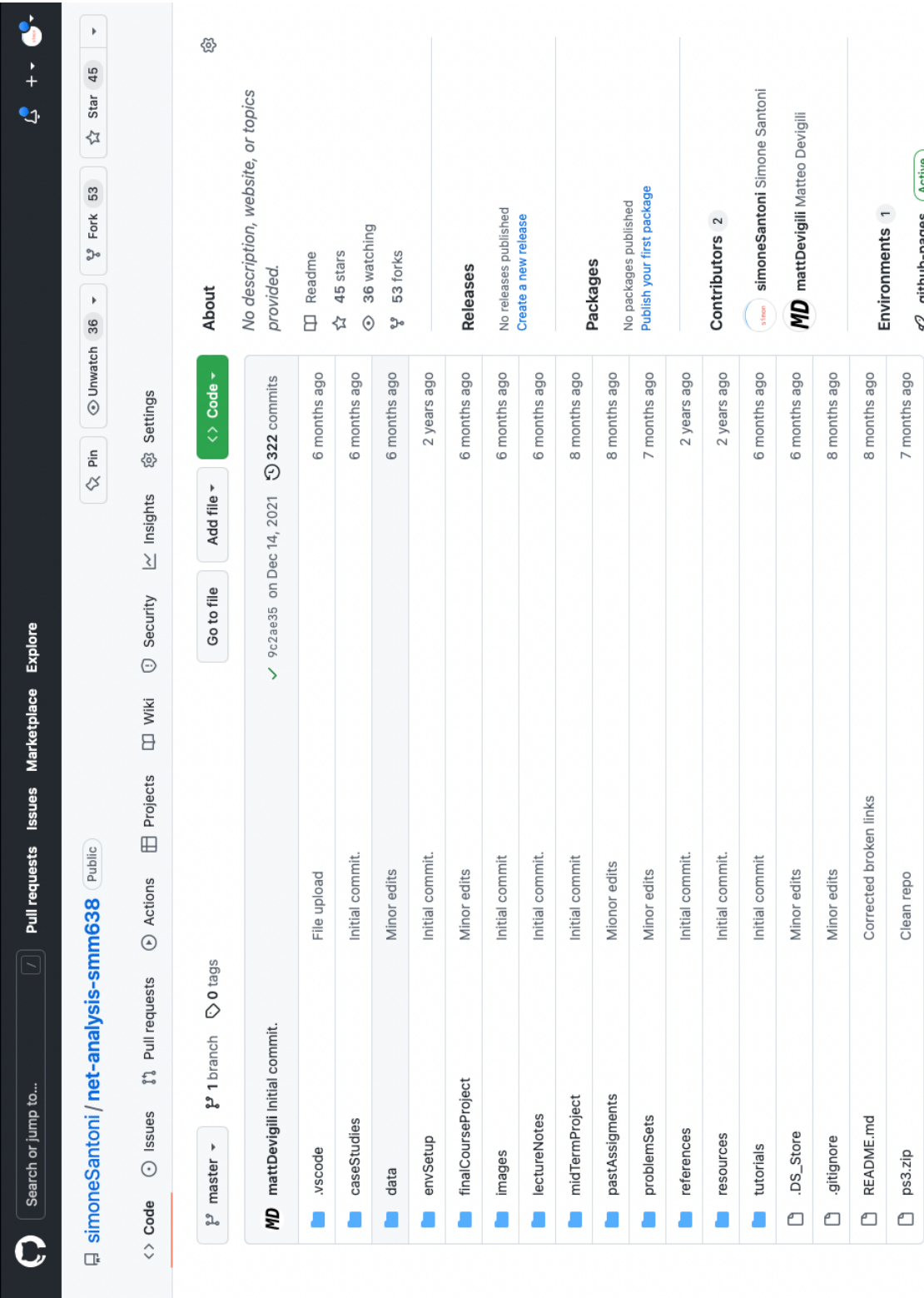


Figure B.2: A screenshot of the GitHub repository behind `SMM638`, Network analytics

Notes. — See <https://github.com/simoneSantoni/net-analysis-smm638>

B.3 Getting Started with GitHub

To work with GitHub, you need to have an account. You can create one for free at <https://github.com>. Programming newbies may want to interact with GitHub through GitHub Desktop (see Figure B.3) rather than through the command line. The graphical installer of GitHub Desktop is available at <https://desktop.github.com/downloads> and is free to use.

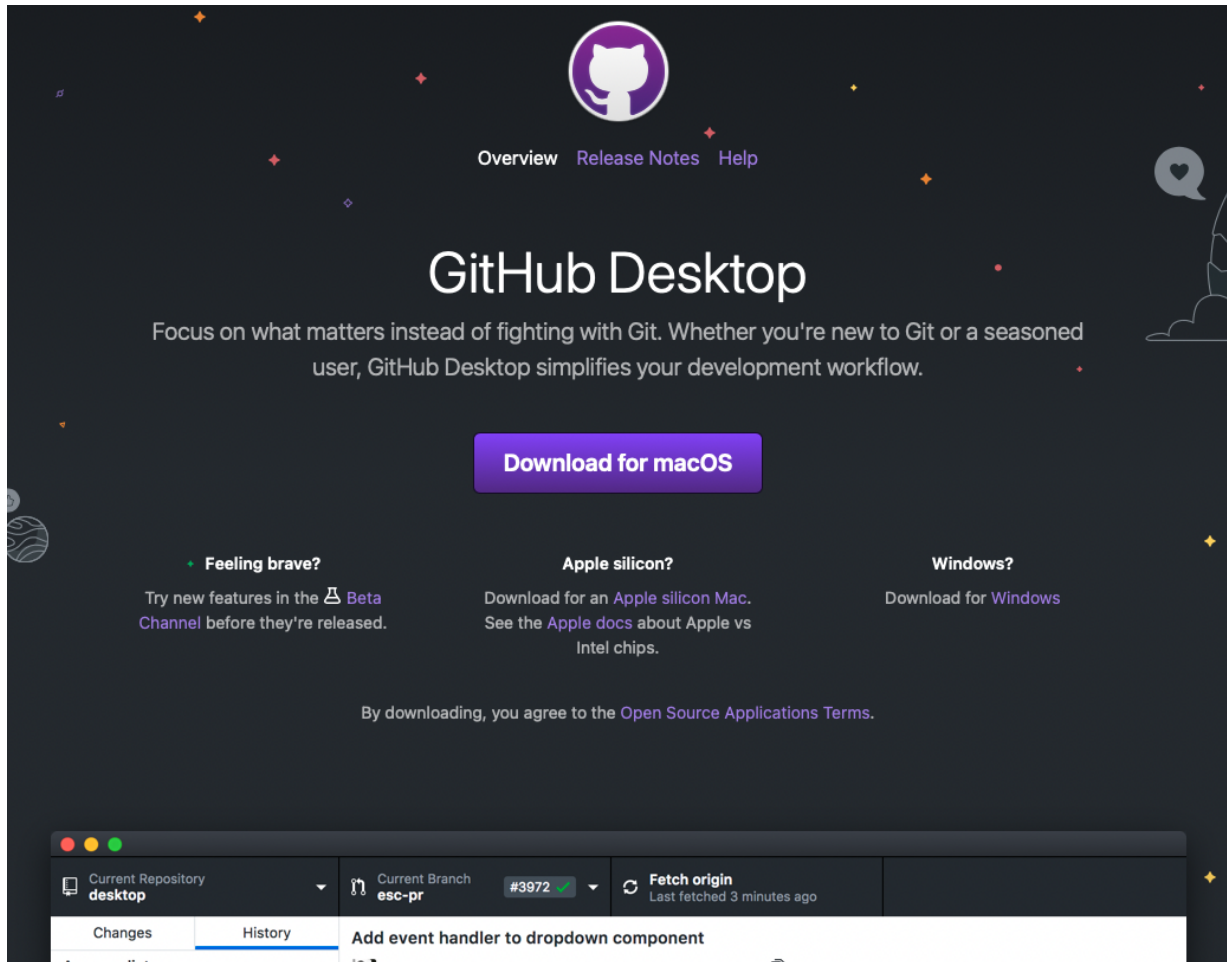


Figure B.3: A screenshot of the GitHub desktop application webpage

Once GitHub Desktop is installed on your computer, you have to log in to your account. You can do this via the 'Preferences' menu (see Figure B.4). At this point, it is possible to interact with GitHub. For example, you may 'clone' a repository hosted on GitHub. To do that, you search for a repository name (see Figure B.5) or pass the URL of the repository (see Figure B.6). Then, you have a local copy of the repository at the exact moment in time when you cloned it.

GithubDesktop has point-and-click structure (see Figure B.7):

- The left-hand side button, named 'Current Repository,' allows one to select the cloned repository one wants to work with
- The central button, named 'Current Branch,' allows one to select the branch one wants to work with. If you are not familiar with GitHub or similar software, you may want to know a branch is a version of a repository. For example, the 'master' branch is the default branch of a repository. It is the branch that contains the most recent version of the repository
- The right-hand side button, named 'Fetch Origin,' allows one to synchronize the local clone of the repository with the remote version of the repository
- The left-hand sidebar has two boxes:

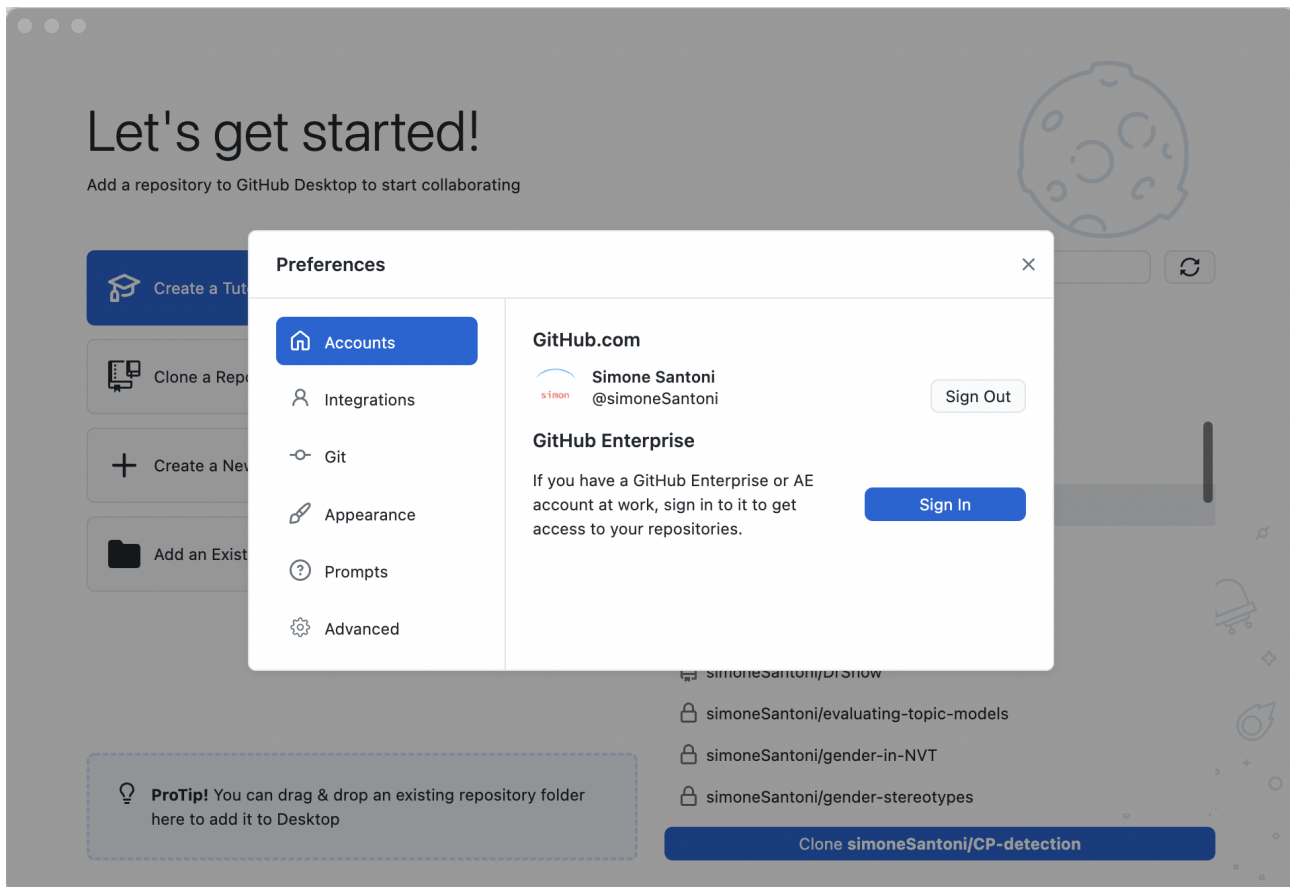


Figure B.4: A screenshot showing how to login into a GitHub account from within GitHub Desktop

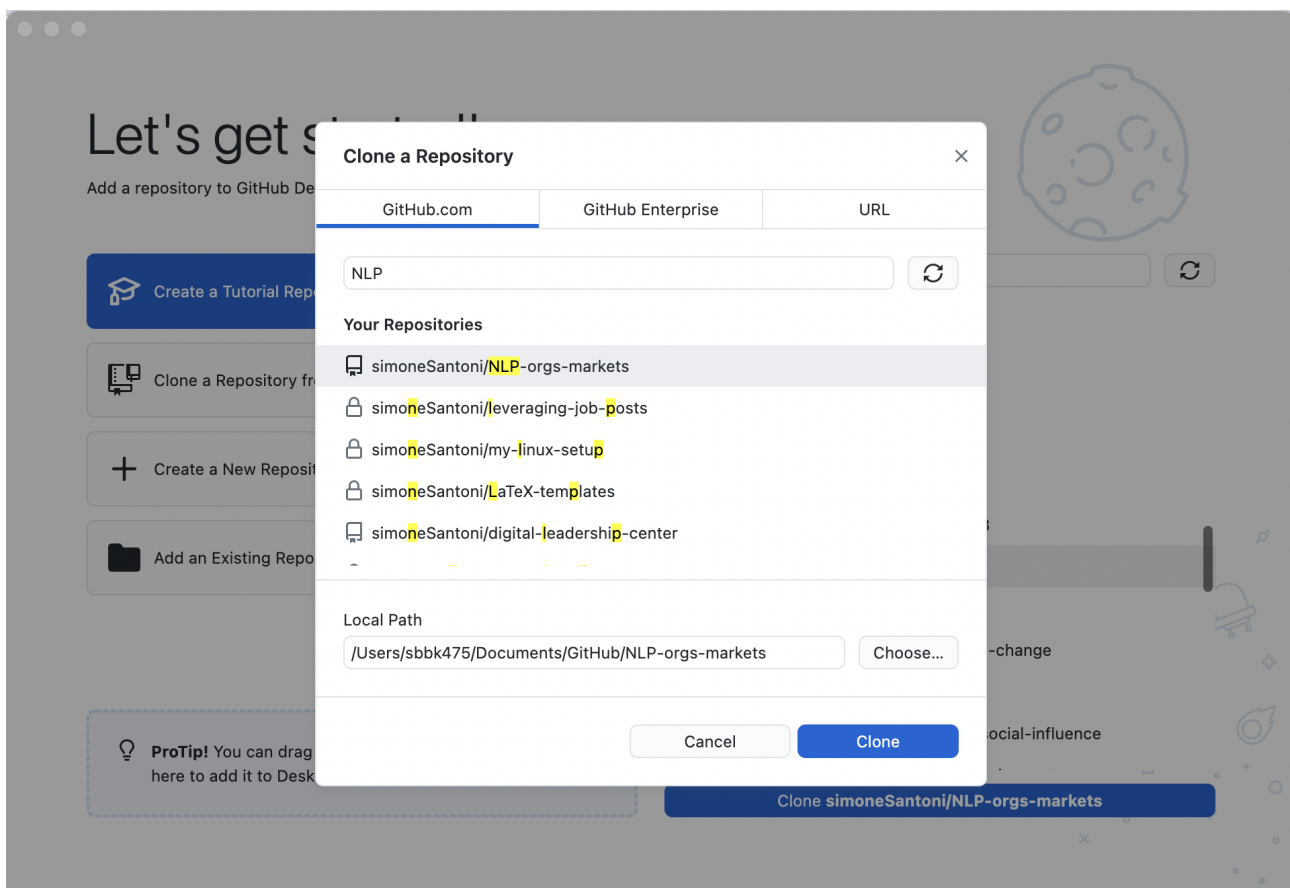


Figure B.5: A screenshot showing how to search for a repository name and then clone it

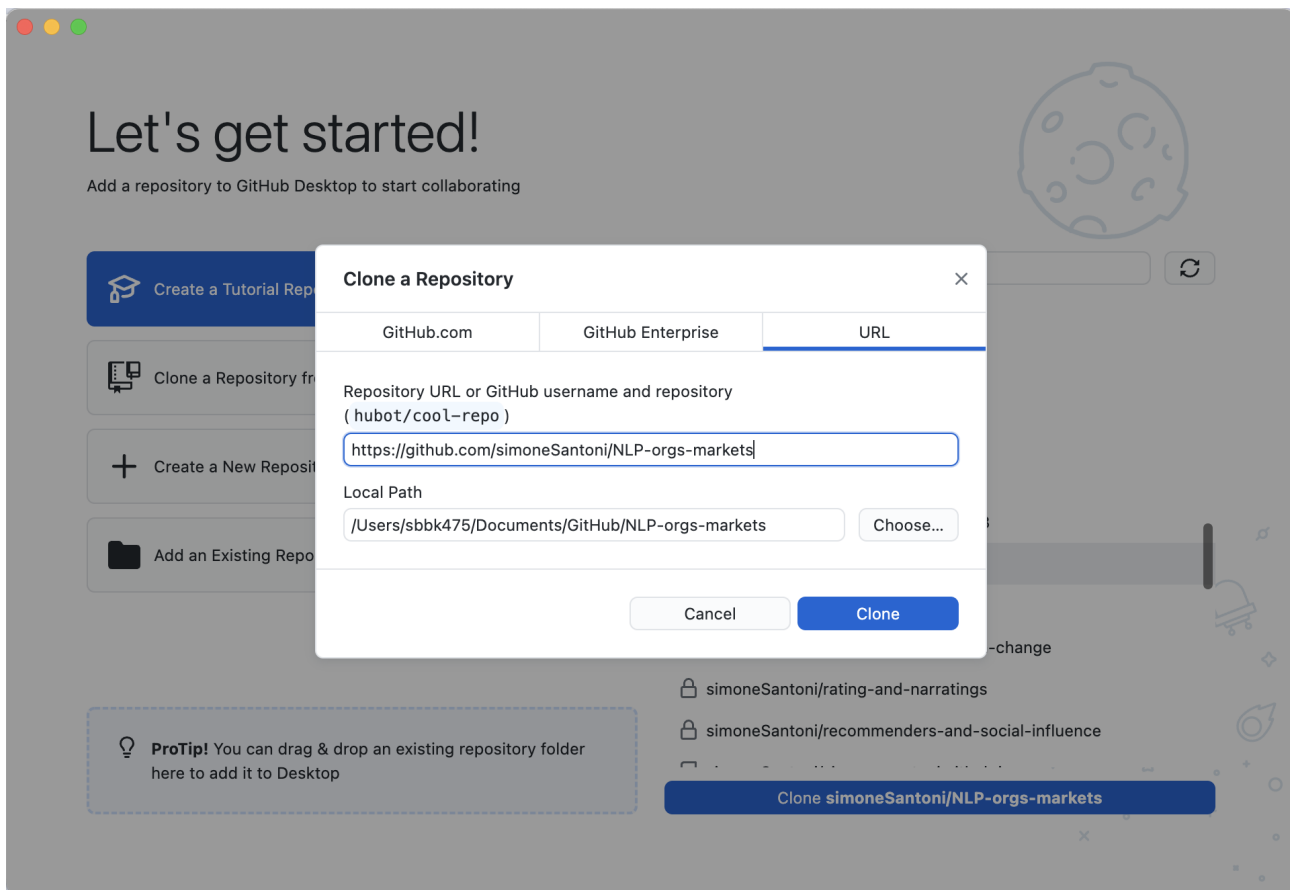


Figure B.6: A screenshot showing how to clone a repo using its URL

- The top box has two tabs:
 - * ‘History’ reports the history of the changes made to the repository
 - * ‘Changes’ reports the changes made to the local clone of the repository
- The bottom box contains a text input area in which one can write a commit message (e.g., ‘added a new file’) and the button to commit the changes made to the remote version of the repository (provided you are the owner of the repository or you have been granted the permission to do so)

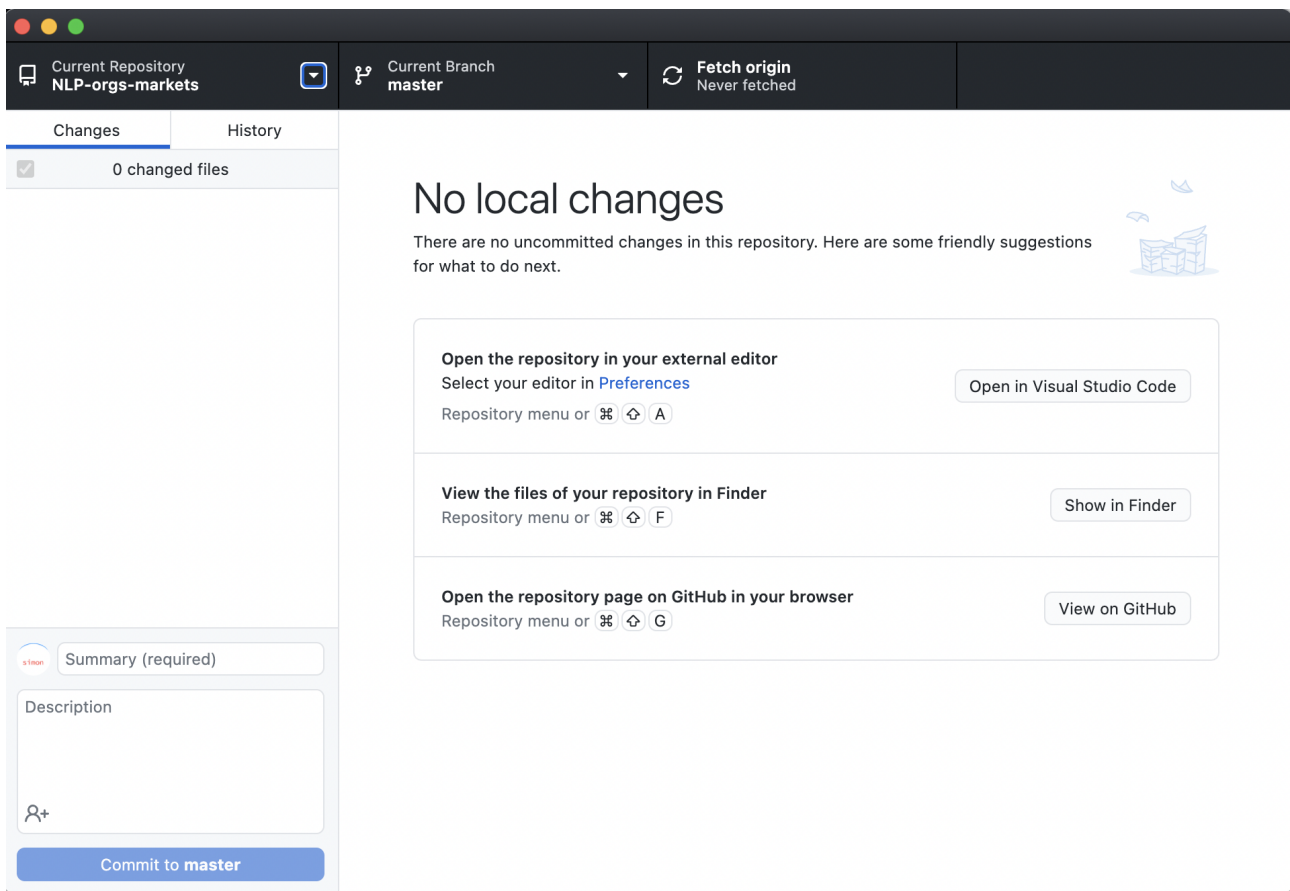


Figure B.7: A screenshot showing the actions available through GitHub Desktop

Notes

¹Zagalsky, Alexey, Joseph Feliciano, Margaret-Anne Storey, Yiyun Zhao, and Weiliang Wang. “The emergence of GitHub as a collaborative platform for education” In *Proceedings of the 18th ACM conference on computer supported cooperative work & social computing*, pp. 1906-1917. 2015.

²Especially, GitHub users can fine-tune the amount of information to receive by selecting the following options: i) by following the user owning a repository, a learner gets their GitHub homepage with the recent activities performed by that user; ii) by starring a repository, a learner has access to shortcuts to quickly and conveniently access a GitHub repository; iii) by watching a repository, a learner receives emails regarding any edit regarding a GitHub repository.

³The inter-temporal distribution of commits regarding SMM694 is available at
<https://github.com/simoneSantoni/NLP-orgs-markets/graphs/commit-activity>

⁴Iiyoshi, Toru, and M. S. V. Kumar. *Opening up education: The collective advancement of education through open technology, open content, and open knowledge*. The MIT Press, 2010.